ACM/ICPC

Rejected

2026 年 10 月 9 日

目录

1 不要忘记 **1**
- 1.1 土制 cph(仅 linux) . . . 1
- 1.2 土制 cph(全平台) . . . 1
- 1.3 随机数 . . . 1
- 1.4 对拍 . . . 2
- 1.5 解除 python 大数限制 . . . 2
- 1.6 常用函数 . . . 2
- 1.7 高精度 . . . 2
- 1.8 多进程评测 . . . 2
- 1.9 迭代器辅助函数 . . . 2
- 1.10 set 相关 . . . 2
- 1.11 vector 相关 . . . 3
- 1.12 整数域三分 . . . 3
- 1.13 运行时间 . . . 3
- 1.14 记得快读快输 . . . 3
- 1.15 !!! 数据范围 . . . 3
- 1.16 __int128 的使用 . . . 3
- 1.17 快读模板 . . . 3
- 1.18 数组离散化 . . . 3
- 1.19 随机数生成器 . . . 3
- 1.20 随机图 . . . 3
- 1.21 对拍器 . . . 4

2 数据结构 **4**
- 2.1 对顶 multiset . . . 4
- 2.2 分块 . . . 4
- 2.3 带权并查集 . . . 5
- 2.4 Trie . . . 5
- 2.5 ST 表 . . . 5
- 2.6 树状数组 . . . 6
- 2.7 线段树 . . . 6
- 2.8 替罪羊树 . . . 6
- 2.9 逆序对 . . . 9
- 2.10 笛卡尔树 . . . 9
- 2.11 可持久化 . . . 9
- 2.12 线段树模板 . . . 9
- 2.13 主席树（可持久化线段树） . . . 10
- 2.14 开点线段树 . . . 10
- 2.15 权值线段树 . . . 11
- 2.16 线段树上最值操作 . . . 11
- 2.17 线段树上序列操作 . . . 12
- 2.18 非经典线段树 . . . 13
- 2.19 Kruskal 重构树 . . . 13
- 2.20 左偏树 . . . 14
- 2.21 虚树的构建 . . . 15

3 图论 **15**
- 3.1 最短路 . . . 15
- 3.2 判奇环（无向图） . . . 15
- 3.3 最小生成树 . . . 16
- 3.4 拓扑排序 . . . 16
- 3.5 LCA . . . 16
- 3.6 树的直径与中心 . . . 16
- 3.7 点分治与树的重心 . . . 16
- 3.8 基环树 . . . 17
- 3.9 Hierholzer 求欧拉路 . . . 17
- 3.10 DSU on tree . . . 17
- 3.11 tarjan . . . 17
- 3.12 树哈希 . . . 18
- 3.13 Bellman-Ford / SPFA（判负环） . . . 18
- 3.14 Floyd 全源最短路 . . . 18
- 3.15 树上倍增 . . . 18
- 3.16 树的直径 . . . 18
- 3.17 树的重心 . . . 18
- 3.18 最大流 . . . 18
- 3.19 最近公共祖先（LCA） . . . 19
- 3.20 树链剖分 . . . 19
- 3.21 最大路匹配 . . . 20
- 3.22 Dijkstra 算法 . . . 20
- 3.23 分层图最短路 . . . 21
- 3.24 双向广搜 . . . 22
- 3.25 tarjan(强连通分量) . . . 22
- 3.26 边双连通分量 . . . 22
- 3.27 割边 . . . 22
- 3.28 点双连通分量 . . . 23
- 3.29 割点 . . . 23

4 数学 **23**
- 4.1 线性筛 . . . 23
- 4.2 因数 . . . 24
- 4.3 拉格朗日插值 . . . 24
- 4.4 快速幂与乘法逆元 . . . 24
- 4.5 中国剩余定理 . . . 24
- 4.6 扩展欧几里得 . . . 25
- 4.7 排列组合数 . . . 25
- 4.8 Lucas . . . 25
- 4.9 卡特兰数 . . . 25
- 4.10 数论分块 . . . 25
- 4.11 高斯消元 . . . 25
- 4.12 矩阵快速幂 . . . 25
- 4.13 自适应辛普森法 . . . 25
- 4.14 多项式 . . . 26
- 4.15 快速幂 . . . 26

4.16 乘法逆元 26
4.17 组合数求法 26
4.18 线性基 27
4.19 矩阵快速幂及高斯消元 27
4.20 线性筛求质因子个数（Omega 函数） 28
4.21 线性筛 (O(n)) 28
4.22 位运算的部分性质 28
4.23 builtin 29

5 字符串 29
5.1 前缀函数与 KMP 29
5.2 Manacher 29
5.3 字符串哈希 30
5.4 AC 自动机 30
5.5 后缀自动机 SAM 31
5.6 KMP 算法 31
5.7 Trie 树 32
5.8 string 常用函数 32

6 动态规划 33
6.1 数位 DP 33
6.2 单调队列优化（滑动窗口） 33
6.3 无向图上回路计数 33
6.4 SOSDP 33
6.5 树形 DP 33
6.6 最长不下降子序列 34

7 计算几何 34
7.1 define 34
7.2 向量 34
7.3 点线操作 35
7.4 多边形与凸包 35
7.5 计算几何模板 35

8 其它 41
8.1 二维差分与前缀和 41
8.2 CDQ 分治 41
8.3 矩阵加速 41
8.4 cdq 斜率优化 42
8.5 并查集 43
8.6 FFT（快速傅里叶变换） 44
8.7 NTT（数论变换） 44
8.8 扫描线算法 45
8.9 最长上升子序列 45
8.10 全排列输出 45
8.11 位运算用法 45
8.12 Map 的使用 46
8.13 优先队列（堆） 46
8.14 扫描线 + 线段树（矩形面积并） 46
8.15 建树参考 47
8.16 Hash 树 47

1 不要忘记

1.1 土制 cph(仅 linux)

```
from os import system as e, listdir as l
from sys import argv
_, q, f = argv  # q: 题号, f: 文件名

if "py" in f:
  r = f"python3 {f}"
else:
  e(f"g++ -std=c++23 -o2 -Wall {f}.cpp -o {f}")
  r = "./" + f

#
d = "samples-" + q.capitalize()
for i in l(d):
  if not "in" in i: continue
  print(i)
  p = d + "/" + i[:-2]
  if e(f"timeout 2 {r}<{p}in>{p}out"): print("TLE or RE")
  else:
    with open(f"{p}out") as o, open(f"{p}ans") as a:
      print("AC" if o.read().split()==a.read().split() else "
          WA")
```

1.2 土制 cph(全平台)

```
from os import listdir as l, system as e
from os.path import join as jp
from sys import argv as a
import subprocess as s
_,q,f = a
if "py" in f:
    cmd = ["python3", f]
else:
    e(f"g++ -std=gnu++20 -O2 -Wall {f}.cpp -o {f}")
    cmd = jp(".", f"{f}.exe")

d='samples-'+q.capitalize()
g,r,p,n='\033[32m','\033[31m','\033[35m','\033[0m'
for i in l(d):
    if not 'in' in i: continue
    t=jp(d,i[:-2])
    print(i,'测评结果 ',end='',flush=1)
    try:
        k = s.run(cmd, timeout=2, stdin=open(f"{t}in"),
                  stdout=open(f"{t}out", 'w'), stderr=s.PIPE,
                      text=1)
        c = lambda o,a : o.read().split() == a.read().split()
        # c = lambda o,a : all(abs(float(x)-float(y))/max(1,
            abs(float(y)))<=1e-4
        #               for x,y in zip(o.read().split(),a.read
            ().split()))
        print(f'{g}AC{n}' if c(open(f"{t}out"),open(f"{t}ans")
            ) else f'{r}WA{n}')
        if k.stderr:print(f'{r}{k.stderr}{n}')
    except s.TimeoutExpired as k:
        print(f'{p}TLE or RE{n}')
        if k.stderr: print(f'{r}{k.stderr}{n}')
```

1.3 随机数

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
     count());
int Rand(int x, int y) {
  return uniform_int_distribution<int>(x, y)(rng);
}
shuffle(a.begin(), a.end(), rng);
```

1.4 对拍

```
// checker.cpp
while (1) {
  system("A_gen > _data.in");  // linux 下换成 ./xxx
  system("A_bf < _data.in > A.ans");
  system("A < _data.in > A.ans");
  if (system("fc A.out A.ans")) {  // linux 下换成 diff
    cout << "WA\n"; return 0;
  } else {
    cout << "AC\n";
```

```
  }
}
```

1.5 解除 python 大数限制

```
import sys
sys.set_int_max_str_digits(100005)
```

1.6 常用函数

```
// 快读
inline int read()
{
  int x=0,f=1;char ch=getchar();
  while (ch<'0'||ch>'9'){if (ch=='-') f=-1;ch=getchar();}
  while (ch>='0'&&ch<='9'){x=x*10+ch-48;ch=getchar();}
  return x*f;
}

// 取模
const int MOD=998244353;
ll add(ll x, ll y) { return (x+y)%MOD; }
ll del(ll x, ll y) { return add(x, MOD-y); }
ll mul(ll x, ll y) { return (x*y)%MOD; }
ll qpow(ll a, ll b = MOD - 2) {
  ll res = 1;
  for (; b; b >>= 1) {
    if (b & 1) { res = mul(res, a); }
    a = mul(a, a);
  }
  return res;
}

// 计算一个整数的二进制表示中有多少个 1
__builtin_popcountll(i);
std::popcount((unsigned long long)i);  // c++20

// 计算一个整数的二进制表示中最高位的 1
__lg(i);  // i 最多 32 位
```

1.7 高精度

```
cout << fixed << setprecision(15);//输出精度

istream& operator>> (istream& is, __int128 &x)
{
  string s; is>>s;
  bool f=false;
  __int128 ans = 0;
  for (auto c: s) {
    if (c == '-') { f = true; }
    if (isdigit(c)) { ans = ans * 10 + (c-'0'); }
  }
  x = f? -ans: ans;
  return is;
}
ostream& operator<< (ostream& os, __int128 &x)
{
  string ans;
  function<void(__int128)> write = [&](__int128 x){
    if (x < 0) { ans += '-'; x=-x; }
    if (x > 9) { write(x/10); }
    ans += '0'+x%10;
  };
  write(x); return os << ans;
}
```

1.8 多进程评测

```
#include <cstdlib>
#include <iostream>
using namespace std;
int main(int argc, char *argv[]) {
    int id = argc > 1 ? atoi(argv[1]) : 0;
    int v;
    cin >> v;
    cout << v << endl;
    cout.flush();
    return 0;
}
```

1.9 迭代器辅助函数

```
函数  作用  是否修改原迭代器  set 上复杂度
std::next(it) 返回 it 后一个位置的迭代器  否  O(1)
std::next(it, n)  返回 it 后 n 步的迭代器  否  O(n)
std::prev(it) 返回 it 前一个位置的迭代器  否  O(1)
std::prev(it, n)  返回 it 前 n 步的迭代器  否  O(n)
std::advance(it, n) 把 it 本身移动 n 步 是  O(n)
std::distance(a, b) 计算 a 到 b 的步数  否  O(n)
++it / --it 前进一步 / 后退一步 是  O(1)
注意:
std::next、std::prev 返回的是新迭代器，原来的 it 不变。

std::advance 会直接改 it，没有返回值。

对 set 来说，std::distance 是 O(n)，不要拿它当区间长度用。区间
    长度应该用 key 的差值，比如 *next(it) - *prev(it)。
```

1.10 set 相关

```
// 默认：从小到大（std::less<int>）
set<int> s1;
// 从大到小（std::greater<int>）
set<int, greater<int>> s2;
// 自定义比较器（仿函数）
struct GreaterCmp {
    bool operator()(int a, int b) const {
        return a > b;   // 从大到小
    }
};
set<int, GreaterCmp> s3;
// 用 lambda 自定义比较器（注意：lambda 类型没有默认构造函数）
auto cmp = [](int a, int b) { return a > b; };
set<int, decltype(cmp)> s4(cmp);
// insert(value) 返回 pair<iterator, bool>
//   - first: 指向插入元素或已存在元素
//   - second: 是否插入成功
// insert(hint, value) 返回 iterator
// hint 正确时均摊 O(1)，否则 O(log n)
s1.insert(s1.end(), 20);
// 插入范围
vector<int> v = {5, 6, 7};
s1.insert(v.begin(), v.end());
// emplace 原地构造
s1.emplace(30);
//
    ------------------------------------------------------------


// 3. 删除
//
    ------------------------------------------------------------


// 按 key 删除，返回删除个数（set 中为 0 或 1）
size_t cnt = s1.erase(10);
auto it2 = s1.find(20);
if (it2 != s1.end()) {
    it2 = s1.erase(it2);   // it2 现在指向下一个元素
}

// 删除范围 [first, last)
// s1.erase(s1.begin(), s1.end());
// 清空
// s1.clear();
//
    ------------------------------------------------------------


// 4. 查找
//
    ------------------------------------------------------------


// find(key) 找到返回迭代器，否则返回 end()
auto it3 = s1.find(5);
if (it3 != s1.end()) {
    // 找到了
}
// count(key) set 中返回 0 或 1
if (s1.count(5)) {
    // 存在
}

// lower_bound(key) / upper_bound(key)
// 默认 less 时:
//   lower_bound: 第一个 >= key
```

```
//   upper_bound: 第一个 > key
// 使用 greater 时语义相反（降序）：
//   lower_bound: 第一个 <= key
//   upper_bound: 第一个 < key
auto lb = s1.lower_bound(5);
auto ub = s1.upper_bound(5);

// equal_range 返回 pair(lower_bound, upper_bound)
auto [lo, hi] = s1.equal_range(5);
//
     ------------------------------------------------------------


// 5. 迭代器与容量
//
     ------------------------------------------------------------

// s1.begin(), s1.end()
// s1.cbegin(), s1.cend()
// s1.rbegin(), s1.rend()
// s1.size(), s1.empty()
//
     ------------------------------------------------------------
```

1.11 vector 相关

```
// 多维 vector，需要 c++17，c++14 只能老老实实写 vector<vector<int>>
vector a(n+1, vector (n+1, vector<int>(n+1)));

// 最值
int mx = *min_element(vec.begin(), vec.end(), cmp);
int mn = *min_element(vec.begin(), vec.end(), cmp);

// 去重与离散化
for (int i = 1; i <= n; ++i) { vec.push_back(a[i]); }
sort(vec.begin(), vec.end());
vec.erase(unique(vec.begin(), vec.end()), vec.end());
for (int i = 1; i <= n; ++i) {
  idx[i]=lower_bound(vec.begin(), vec.end(), a[i])-vec.begin()+1;  // 下标从1开始
}

// 前缀和
vector <ll> a(n), s(n);
partial_sum(a.begin(), a.end(), s.begin());
ll sum = accumulate(a.begin(), a.end(), 0LL);

// 填充 1~n，令 a[i]=i
iota(a.begin(), a.end(), 0);
```

1.12 整数域三分

```
// 整数域三分求单峰函数最大值
int l, r;
while (l + 2 <= r) {
  int w = (r - l) / 3;
  int m1 = l + w; int v1 = f(m1);
  int m2 = r - w; int v2 = f(m2);
  if (v1 <= v2) {
    l = m1 + 1;
  } else {
    r = m2 - 1;
  }
}
cout << max(f(l), f(r)) << "\n";
```

1.13 运行时间

```
auto start = chrono::high_resolution_clock::now();
auto stop = chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(stop - start).count();
if (duration > 998) { cout << ans << "\n"; exit(0); }
```

1.14 记得快读快输

```
ios::sync_with_stdio(0);
cin.tie(0), cout.tie(0);
```

1.15 !!! 数据范围

```
注意爆int

    long long: 8字节，-9,223,372,036,854,775,808到9
         ,223,372,036,854,775,807
    unsigned long long: 8字节，0到18,446,744,073,709,551,615
    如果爆long long 就用__int128,但是注意输入输出

    尽量不要使用double输入，容易超时
```

1.16 __int128 的使用

```
#define int __int128
inline void read(int &n){
    int x=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9'){
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9'){
        x=(x<<1)+(x<<3)+(ch^48);
        ch=getchar();
    }
    n=x*f;
}
inline void print(int n){
    if(n<0){
        putchar('-');
        n*=-1;
    }
    if(n>9) print(n/10);
    putchar(n % 10 + '0');
}
#undef int
```

1.17 快读模板

```
#define re register
#define il inline
il int read()
{
    re int x=0,f=1;char c=getchar();
    while(c<'0'||c>'9'){if(c=='-') f=-1;c=getchar();}
    while(c>='0'&&c<='9') x=(x<<3)+(x<<1)+(c^48),c=getchar();
    return x*f;
}
```

1.18 数组离散化

```
sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
```

1.19 随机数生成器

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
int Rand(int x, int y) {
  return uniform_int_distribution<int>(x, y)(rng);
}
shuffle(a.begin(), a.end(), rng);
```

1.20 随机图

```
// n 为节点个数
// m 为边的个数，默认为 -1，表示建树
// root 为根节点编号，默认为 -1，
vector<array<int,2>> Graph(int n, int m=-1, int root=-1)
{
  assert(m==-1 || (n-1<=m && m<=1LL*n*(n-1)/2));
  assert(root==-1 || (1<=root && root<=n));
  vector<array<int,2>> E;
  set<array<int,2>> diff;
  // 先建一棵以 0 为根的树，然后选根偏移
  for (int i = 1; i < n; ++i) {
    E.push_back( { i, Rand(0, i-1) } );
  }
  if (root == -1) { root = Rand(0, n-1); }
```

```
  for (auto& [x, y]: E) {
    x = (x+root) % n + 1;
    y = (y+root) % n + 1;
    diff.insert( { x, y } );
    diff.insert( { y, x } );  // 无向图中需要添加这条反向边
  }
  // 从树建图
  if (m != -1) {
    for (int i = n-1; i < m; ++i) {
      while (1) {
        int x = Rand(1, n);
        int y = Rand(1, n);
        if (x == y || diff.count({x,y})) { continue; }
        E.push_back( { x, y } );
        diff.insert( { x, y } );
        diff.insert( { y, x } );  // 无向图中需要添加这条反向边
        break;
      }
    }
  }
  shuffle(E.begin(), E.end(), rng);
  return E;
}
```

1.21 对拍器

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int t=0;
    while(true){
    cout<<"test: "<<t++<<endl;
    system("data.exe>data.in");//data.exe 随机数生成器
    system("std.exe<data.in>data.out");//std.exe 标准程序
    system("solve.exe<data.in>solve.out");//solve.exe 暴力求解算法
    if(system("fc std.out solve.out>diff.log")){
        cout<<"WA"<<endl;
        break;
    }
}
cout<<"AC\n";
}
/*
// checker.cpp
while (1) {
  system("A_gen > _data.in");  // linux 下换成 ./xxx
  system("A_bf < _data.in > A.ans");
  system("A < _data.in > A.ans");
  if (system("fc A.out A.ans")) {  // linux 下换成 diff
    cout << "WA\n"; return 0;
  } else {
    cout << "AC\n";
  }
}*/
```

2 数据结构

2.1 对顶 multiset

```
//对顶 multiset 数据结构主要用于动态维护一组数的中位数（或第 k 小/大等顺序统计量）
struct PairingMultiset {
    int sz;
    multiset <int> minH, maxH;
    ll minS, maxS;
    void clear()
    {
        minH.clear(); maxH.clear();
        sz = minS = maxS = 0;
    }
    void move()
    {
        while ((int)minH.size() < (sz+1)/2) {
            minS += *maxH.rbegin();
            minH.insert(*maxH.rbegin());
            maxS -= *maxH.rbegin();
            maxH.erase(maxH.find(*maxH.rbegin()));
        }
        while ((int)minH.size() > (sz+1)/2) {
            maxS += *minH.begin();
            maxH.insert(*minH.begin());
            minS -= *minH.begin();
            minH.erase(minH.find(*minH.begin()));
        }
    }
    void add(int x)
    {  // 向对顶堆中插入一个元素
        ++sz;
        if (minH.empty() || x >= *minH.begin()) {
            minS += x;
            minH.insert(x);
        } else {
            maxS += x;
            maxH.insert(x);
        }
        move();
    }
    void del(int x)
    {  // 从对顶堆中删除一个x
        --sz;
        if (x >= *minH.begin()) {
            minS -= x;
            minH.erase(minH.find(x));
        } else {
            maxS -= x;
            maxH.erase(maxH.find(x));
        }
        move();
    }
    ll med()
    {  // 求中位数
        return *minH.begin();
    }
    ll dis_to_med()
    {  // 求对顶堆中的数字到中位数的距离和
        ll minSZ=(sz+1)/2, maxSZ=sz-minSZ, m=med();
        return (minS-minSZ*m)+(maxSZ*m-maxS);
    }
} p;
```

2.2 分块

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=5e5+10;
const int maxb=1e3+10;
int arr[maxn],sv[maxn],n,m;
int blen,bnum;
int bi[maxn],bl[maxb],br[maxb];
void build(){
    blen=sqrt(n);
    bnum=(n+blen-1)/blen;
    for(int i=1;i<=n;i++){
        bi[i]=(i-1)/blen+1;
    }
    for(int i=1;i<=bnum;i++){
        bl[i]=(i-1)*blen+1;
        br[i]=min(i*blen,n);
    }
    for(int i=1;i<=n;i++) sv[i]=arr[i];
    for(int i=1;i<=bnum;i++){
        sort(sv+bl[i],sv+br[i]+1);
    }
}
int getcnt(int i,int v){
    int l=bl[i],r=br[i],m,res=0;
    while(l<=r){
        m=(l+r)>>1;
        if(sv[m]>=v){
            res+=r-m+1;
            r=m-1;
        }else l=m+1;
    }
    return res;

}
void update(int ii,int v){
    int l=bl[bi[ii]];
    int r=br[bi[ii]];
    arr[ii]=v;
    for(int i=l;i<=r;i++) sv[i]=arr[i];
    sort(sv+l,sv+r+1);
}
```

```
int query(int l,int r,int v){
    int res=0;
    if(bi[l]==bi[r]){
        for(int i=l;i<=r;i++){
            if(arr[i]>=v) res++;
        }
    }else{
        for(int i=l;i<=br[bi[l]];i++){
            if(arr[i]>=v) res++;
        }
        for(int i=bl[bi[r]];i<=r;i++){
            if(arr[i]>=v) res++;
        }
        for(int i=bi[l];i<bi[r];i++){
            res+=getcnt(i,v);
        }
    }
    return res;
}
int main(){
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>arr[i];
    }
    build();
    cin>>m;
    for(int i=1;i<=m;i++){
        int op,a,b,c;
        cin>>op>>a>>b;
        if(op==0){
            cin>>c;
            cout<<query(a,b,c)<<endl;
        }else update(a,b);
    }
}
```

2.3 带权并查集

```
struct DSU {
    int n;
    vector<int> f, siz, pre;
    DSU(int n): n(n),
          f(vector<int>(n+1)),
          siz(vector<int>(n+1,1)),
          pre(vector<int>(n+1))
    {
        iota(f.begin(), f.end(), 0);
    }
    int find(int x) {
        if (f[x] == x) { return x; }
        int fa = find(f[x]);
        pre[x] += pre[f[x]];
        return f[x]=fa;
    }
    bool same(int x, int y) {
        return find(x) == find(y);
    }
    void merge(int x, int y) {
        x = find(x);
        y = find(y);
        f[x] = y;
        pre[x] += siz[y];
        siz[y] += siz[x];
        siz[x] = 0;
    }
    int query(int x, int y) {
        return abs(pre[x]-pre[y])-1;
    }
};
```

2.4 Trie

```
const string VAL="
     abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
     ";
const int S=3e6+5, V=62;

struct Trie {
    unordered_map<char,int> p;
    int tot, ch[S][V], pass[S];
    int newNode() {
        ++tot;
        memset(ch[tot], 0, sizeof(ch[tot]));
        pass[tot] = 0;
        return tot;
    }
    void init() {
        tot = -1;
        newNode();
    }
    Trie() {
        for (int i = 0; i < V; ++i) { p[VAL[i]] = i; }
        init();
    }
    void insert(string s) {
        int now = 0;
        for (auto c: s) {
            if (!ch[now][p[c]]) { ch[now][p[c]] = newNode(); }
            now = ch[now][p[c]];
            ++pass[now];
        }
    }
    int query(string s) {
        int now = 0;
        for (auto c: s) {
            if (!ch[now][p[c]]) { return 0; }
            now = ch[now][p[c]];
        }
        return pass[now];
    }
} trie;
```

2.5 ST 表

```
template <typename T>
struct ST {
    int n=0, I=0;
    vector<int> Log;
    vector<vector<T>> st;
    ST() { }
    ST(const vector<T>& a): n(a.size()), Log(vector<int>(n+1))
          {
        for (int i = 2; i <= n; ++i) {
          I=Log[i]=Log[i/2]+1;
        }
        st.assign(I+1, vector<T>(n));
        copy(a.begin(), a.end(), st[0].begin());
        for (int i = 1; i <= I; ++i) {
            for (int j = 0; j+(1<<(i-1)) < n; ++j) {
                st[i][j] = max( st[i-1][j], st[i-1][j+(1<<(i
                     -1))] );
            }
        }
    }
    T query(int l, int r) {
      assert(l <= r);  // l > r 时的返回值需要特殊定义
        int s = Log[r-l+1];
        return max( st[s][l], st[s][r-(1<<s)+1] );
    }
    int find(int l, T x) {  // 第一个区间 [l,r] 最值大于等于 x
          的 r
      if (l >= n) { return -1; }
        int rl=l-1, rr=n;
        while (rl != rr-1) {
            int mid = (rl + rr) >> 1;
            if (query(l, mid) >= x) { rr = mid; }
            else { rl = mid; }
        }
        if (rr < n) { return rr; }
        return -1;
    }
  int find(int l, int x) {  // 倍增写法
      assert(l < n);
      if (st[0][l] < x) { return l; }
      if (query(l, n-1) >= x) { return n; }
      int ans = l, g = st[0][l];
      for (int i = I; i >= 0; --i) {
          if (ans+(1<<i) < n && gcd(g, st[i][ans]) >= x) {
              g = max(g, st[i][ans]);
              ans += 1 << i;
          }
      }
      return ans;
  }
};
```

2.6 树状数组

```
struct BIT {
  int n;
    vector<int> t;
    BIT(int n): n(n), t(n+1) { }  // 注意值域树状数组中 n=tot
    int lowbit(int x) { return x&-x; }
    void modify(int x, int d) {
        for (; x <= n; x += lowbit(x)) { t[x] += d; }
    }
  // 前缀和
    int query(int x) {
        int res = 0;
        for (; x; x -= lowbit(x)) { res += t[x]; }
        return res;
    }
    // 求最小的前缀和等于x的位置
  int select(int x) {
      int ans = 0, sum = 0;
      for (int i = __lg(n); i >= 0; --i) {
          if (ans+(1<<i) <= n && sum+t[ans+(1<<i)] < x) {
              ans += 1<<i;
              sum += t[ans];
          }
      }
      return ans + 1;
  }
};
BIT T(n);
```

2.7 线段树

```
#define lson (p << 1)
#define rson (p << 1 | 1)
#define m ((l+r)>>1)
template<typename Info>
struct segmentTree {
    int n;
    vector<Info> a;

    void push_up(int p) {
        a[p] = a[lson] + a[rson];
    }

    template<typename F>
    Info dfs(int p, int l, int r, int x, int y, F&& op) {
        if (y <= l || r <= x) { return Info(); }
        if (x <= l && r <= y) { op(p,l,r); return a[p]; }
        Info res;
        res = res + dfs(lson, l, m, x, y, op);
        res = res + dfs(rson, m, r, x, y, op);
        push_up(p);
        return res;
    }

  template<typename F>
  pair<int,Info> findFirst(int p, int l, int r, int x, int y,
       F&& pred) {
      if (y <= l || r <= x) { return {-1,Info()}; }
      if (x <= l && r <= y && !pred(a[p])) { return {-1,Info()
          }; }
      if (l == r-1) { return {l,a[p]}; }
      pair<int,Info> res = findFirst(lson, l, m, x, y, pred);
      if (res.first == -1) {
          res = findFirst(rson, m, r, x, y, pred);
      }
      return res;
  }

  template<typename F>
  pair<int,Info> findLast(int p, int l, int r, int x, int y, F
       && pred) {
      if (y <= l || r <= x) { return {-1,Info()}; }
      if (x <= l && r <= y && !pred(a[p])) { return {-1,Info()
          }; }
      if (l == r-1) { return {l,a[p]}; }
      pair<int,Info> res = findLast(lson, m, r, x, y, pred);
      if (res.first == -1) {
          res = findLast(rson, l, m, x, y, pred);
      }
      return res;
  }

    segmentTree(int n = 0): n(n), a(4 << __lg(n)) { }
    segmentTree(vector<Info>&& b): segmentTree(b.size()) {
        function<void(int,int,int)> build = [&](int p, int l,
             int r) -> void {
            if (r - l == 1) { a[p] = Info(b[l]); return; }
            build(lson, l, m);
            build(rson, m, r);
            push_up(p);
        };
        build(1, 0, n);
    }

    void modify(int pos, int x) {
        dfs(1, 0, n, pos, pos+1, [&](int p, int l, int r){ a[p
            ].mx = x; });
    }

    int query(int l, int r) {
        return dfs(1, 0, n, l, r+1, [](int,int,int){}).mx;
    }

    void Debug(int p, int l, int r) {
        cerr << format("a[{}] = [ {}, {}, {} ]\n", p, l, r, a[
            p].mx);
        if (l == r - 1) { return; }
        Debug(lson, l, m);
        Debug(rson, m, r);
    }
    void Debug() { Debug(1, 0, n); }
};
#undef lson
#undef rson
#undef m
struct Info {
    int mx;
    Info(int mx = 0): mx(mx) { }
    friend Info operator+ (Info u, Info v) {
        return Info( max(u.mx, v.mx) );
    }
};
```

2.8 替罪羊树

```
/*替罪羊树 是一种依靠重构操作维持平衡的重量平衡树. 替罪羊树会
    在插入、删除操作后, 检测树是否发生失衡; 如果失衡, 将有针
    对性地进行重构以恢复平衡.
一般地, 替罪羊树不支持区间操作, 且无法完全持久化; 但它具有实现
    简单、常数较小的优点.
*/
//单次查询, 单次调整的代价均摊为log（n）, 平衡因子一般为0
    .7--0.8
//做词频压缩的
struct ScapegoatTree {
    static const int MAXN = 200005;
    static constexpr double ALPHA = 0.7;
    static const int INF = 1e9 + 10;

    int root, cnt;
    int key[MAXN], num[MAXN];
    int l[MAXN], r[MAXN];
    int sz[MAXN], dif[MAXN];
    int v[MAXN];

    int vc;
    int top, fa, side;

    ScapegoatTree() {
        clear();
    }

    int init(int x) {
        ++cnt;
        key[cnt] = x;
        num[cnt] = 1;
        l[cnt] = r[cnt] = 0;
        sz[cnt] = dif[cnt] = 1;
        return cnt;
    }

    void pushup(int p) {
        sz[p] = sz[l[p]] + sz[r[p]] + num[p];
        dif[p] = dif[l[p]] + dif[r[p]] + (num[p] > 0);
    }
```

```

    bool balance(int p) {
        return ALPHA * dif[p] >= max(dif[l[p]], dif[r[p]]);
    }

    void inorder(int p) {
        if (!p) return;

        inorder(l[p]);

        if (num[p] > 0)
            v[++vc] = p;

        inorder(r[p]);
    }

    int build(int L, int R) {
        if (L > R) return 0;

        int mid = (L + R) >> 1;
        int p = v[mid];

        l[p] = build(L, mid - 1);
        r[p] = build(mid + 1, R);

        pushup(p);

        return p;
    }

    void rebuild() {
        if (!top) return;

        vc = 0;
        inorder(top);

        if (!vc) {
            if (!fa) root = 0;
            else if (side == 1) l[fa] = 0;
            else r[fa] = 0;
            return;
        }

        int p = build(1, vc);

        if (!fa)
            root = p;
        else if (side == 1)
            l[fa] = p;
        else
            r[fa] = p;
    }

    void insert(int p, int f, int s, int x) {
        if (!p) {
            if (!f)
                root = init(x);
            else if (s == 1)
                l[f] = init(x);
            else
                r[f] = init(x);

            return;
        }

        if (key[p] == x)
            ++num[p];
        else if (key[p] > x)
            insert(l[p], p, 1, x);
        else
            insert(r[p], p, 2, x);

        pushup(p);

        if (!balance(p)) {
            top = p;
            fa = f;
            side = s;
        }
    }

    void insert(int x) {
        top = fa = side = 0;
        insert(root, 0, 0, x);
        rebuild();
    }

    int countLess(int p, int x) {
        if (!p) return 0;

        if (key[p] >= x)
            return countLess(l[p], x);

        return sz[l[p]] + num[p] + countLess(r[p], x);
    }

    // x 的排名：第一个 >= x 的位置
    int rank(int x) {
        return countLess(root, x) + 1;
    }

    int kth(int p, int k) {
        if (sz[l[p]] >= k)
            return kth(l[p], k);

        if (sz[l[p]] + num[p] < k)
            return kth(r[p], k - sz[l[p]] - num[p]);

        return key[p];
    }

    // 第 k 小
    int kth(int k) {
        if (k <= 0 || k > sz[root])
            return INF;

        return kth(root, k);
    }

    bool exist(int p, int x) {
        if (!p) return false;

        if (key[p] == x)
            return num[p] > 0;

        if (key[p] > x)
            return exist(l[p], x);

        return exist(r[p], x);
    }

    void erase(int p, int f, int s, int x) {
        if (!p) return;

        if (key[p] == x)
            --num[p];
        else if (key[p] > x)
            erase(l[p], p, 1, x);
        else
            erase(r[p], p, 2, x);

        pushup(p);

        if (!balance(p)) {
            top = p;
            fa = f;
            side = s;
        }
    }

    void erase(int x) {
        if (!exist(root, x)) return;

        top = fa = side = 0;
        erase(root, 0, 0, x);
        rebuild();
    }

    // < x 的最大值
    int pre(int x) {
        int k = rank(x);

        if (k == 1)
            return -INF;

        return kth(k - 1);
    }

```

```
    // > x 的最小值
    int nxt(int x) {
        int k = rank(x + 1);

        if (k == sz[root] + 1)
            return INF;

        return kth(k);
    }

    // 当前元素总数
    int size() {
        return sz[root];
    }

    // 是否为空
    bool empty() {
        return sz[root] == 0;
    }

    // 清空
    void clear() {
        root = 0;
        cnt = 0;
        vc = 0;
        top = fa = side = 0;
    }
};

//不做词频压缩版本
struct ScapegoatTree {
    static const int MAXN = 200005;
    static constexpr double ALPHA = 0.7;
    int root, cnt;
    int key[MAXN], l[MAXN], r[MAXN], sz[MAXN];
    int v[MAXN];
    int vc;
    int top, fa, side;
    ScapegoatTree() {
        clear();
    }
    int init(int x) {
        ++cnt;
        key[cnt] = x;
        l[cnt] = r[cnt] = 0;
        sz[cnt] = 1;
        return cnt;
    }
    void pushup(int p) {
        sz[p] = sz[l[p]] + sz[r[p]] + 1;
    }
    bool balance(int p) {
        return max(sz[l[p]], sz[r[p]]) <= ALPHA * sz[p];
    }
    void inorder(int p) {
        if (!p) return;
        inorder(l[p]);
        v[++vc] = p;
        inorder(r[p]);
    }
    int build(int L, int R) {
        if (L > R) return 0;
        int mid = (L + R) >> 1;
        int p = v[mid];
        l[p] = build(L, mid - 1);
        r[p] = build(mid + 1, R);
        pushup(p);
        return p;
    }
    void rebuild() {
        if (!top) return;
        vc = 0;
        inorder(top);
        int p = build(1, vc);
        if (!fa)
            root = p;
        else if (side == 1)
            l[fa] = p;
        else
            r[fa] = p;
    }
    void insert(int p, int f, int s, int x) {
        if (!p) {
            if (!f)
                root = init(x);
            else if (s == 1)
                l[f] = init(x);
            else
                r[f] = init(x);

            return;
        }
        if (x <= key[p])
            insert(l[p], p, 1, x);
        else
            insert(r[p], p, 2, x);
        pushup(p);
        if (!balance(p)) {
            top = p;
            fa = f;
            side = s;
        }
    }
    void insert(int x) {
        top = fa = side = 0;
        insert(root, 0, 0, x);
        rebuild();
    }
    // < x 的元素个数
    int countLess(int p, int x) {
        if (!p) return 0;
        if (key[p] >= x)
            return countLess(l[p], x);
        return sz[l[p]] + 1 + countLess(r[p], x);
    }
    // x 的排名：第一个 >= x 的位置
    int rank(int x) {
        return countLess(root, x) + 1;
    }
    int kth(int p, int k) {
        if (sz[l[p]] >= k)
            return kth(l[p], k);

        if (sz[l[p]] + 1 < k)
            return kth(r[p], k - sz[l[p]] - 1);

        return key[p];
    }
    // 第 k 小
    int kth(int k) {
        return kth(root, k);
    }
    bool exist(int p, int x) {
        if (!p) return false;
        if (key[p] == x)
            return true;
        if (x < key[p])
            return exist(l[p], x);
        return exist(r[p], x);
    }
    void erase(int p, int f, int s, int x) {
        if (key[p] == x) {
            // 找到以后直接把整个节点删除
            // 用左右子树合并的方法删除
            if (!l[p] || !r[p]) {
                int son = l[p] ? l[p] : r[p];

                if (!f)
                    root = son;
                else if (s == 1)
                    l[f] = son;
                else
                    r[f] = son;

                return;
            }
            // 两个儿子都存在：
            // 找右子树最小节点
            int q = r[p];
            while (l[q])
                q = l[q];

            key[p] = key[q];

            erase(r[p], p, 2, key[q]);
        }
        else if (x < key[p]) {
            erase(l[p], p, 1, x);
```

```
        }
        else {
            erase(r[p], p, 2, x);
        }
        pushup(p);
        if (!balance(p)) {
            top = p;
            fa = f;
            side = s;
        }
    }
    void erase(int x) {
        if (!exist(root, x)) return;

        top = fa = side = 0;
        erase(root, 0, 0, x);
        rebuild();
    }
    // < x 的最大值
    int pre(int x) {
        int k = rank(x);

        if (k == 1)
            return -INT_MAX;

        return kth(k - 1);
    }
    // > x 的最小值
    int nxt(int x) {
        int k = rank(x + 1);

        if (k == sz[root] + 1)
            return INT_MAX;

        return kth(k);
    }
    int size() {
        return sz[root];
    }
    bool empty() {
        return sz[root] == 0;
    }
    void clear() {
        root = 0;
        cnt = 0;
        vc = 0;
        top = fa = side = 0;
    }
};
```

2.9 逆序对

```
ll msort(vector<int> &a, int l, int r)
{
    if (l == r) { return 0; }
    int mid = (l + r) >> 1;
    ll ans=0;
    ans += msort(a, l, mid);
    ans += msort(a, mid+1, r);
    int i=l, j=mid+1;
    vector<int> b;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) { b.push_back(a[i++]); }
        else              { b.push_back(a[j++]); ans+=mid-i+1;
              }
    }
    while (i <= mid) { b.push_back(a[i++]); }
    while (j <= r)   { b.push_back(a[j++]); }
    copy(b.begin(), b.end(), a.begin()+l);
    return ans;
}
```

2.10 笛卡尔树

```
vector<int> stk;
for (int i = 1; i <= n; ++i) {
  int flag = 0;
  while (!stk.empty()) {
    if (a[stk.back()] > a[i]) {  // 小根堆
      flag = stk.back(); stk.pop_back();
    } else {
      break;
    }
  }
  if (flag) { L[i] = flag; }
  if (!stk.empty()) { R[stk.back()] = i; }
  stk.push_back(i);
}
function<void(int)> dfs = [&](int x){
  cerr << "now node " << x << "\n";
  cerr << ":: L = " << L[x] << "\n";
  cerr << ":: R = " << R[x] << "\n";
  if (L[x]) dfs(L[x]);
  if (R[x]) dfs(R[x]);
};
dfs(stk[0]);
```

2.11 可持久化

```
5 10
59 46 14 87 41
0 2 1
0 1 1 14
0 1 1 57
0 1 1 88
4 2 4
0 2 5
0 2 4
4 2 1
2 2 2
1 1 5 91
---
59
87
41
87
88
46
```

2.12 线段树模板

```
//区间乘法
#include<bits/stdc++.h>
using namespace std;
#define maxn 1000010
#define ll long long
int p;
ll a[maxn], w[maxn*4];
ll lzy_add[maxn*4], lzy_mul[maxn*4];

void pushup(const int u) {
    w[u] = (w[u<<1] + w[u<<1|1]) % p;
}

void build(const int u, int L, int R) {
    lzy_mul[u] = 1;
    lzy_add[u] = 0;
    if (L == R) {
        w[u] = a[L] % p;
        return;
    }
    int M = (L+R)>>1;
    build(u<<1, L, M);
    build(u<<1|1, M+1, R);
    pushup(u);
}

bool InRange(int L, int R, int l, int r) {
    return (l <= L) && (R <= r);
}

bool OutofRange(int L, int R, int l, int r) {
    return (R < l) || (L > r);
}

void maketag(int u, int L, int R, ll x, int type) {
    if (type == 1) {
        (lzy_mul[u] *= x) %= p;
        (lzy_add[u] *= x) %= p;
        (w[u] *= x) %= p;
    } else {
        (lzy_add[u] += x) %= p;
        (w[u] += (R-L+1) * x) %= p;
    }
```

```
}

void pushdown(int u, int L, int R) {
    int M = (L+R)>>1;
    // 左子节点
    maketag(u<<1, L, M, lzy_mul[u], 1);
    maketag(u<<1, L, M, lzy_add[u], 2);
    // 右子节点
    maketag(u<<1|1, M+1, R, lzy_mul[u], 1);
    maketag(u<<1|1, M+1, R, lzy_add[u], 2);
    lzy_mul[u] = 1;
    lzy_add[u] = 0;
}

ll query(int u, int L, int R, int l, int r) {
    if (InRange(L, R, l, r)) {
        return w[u];
    } else if (!OutofRange(L, R, l, r)) {
        int M = (L+R)>>1;
        pushdown(u, L, R);
        return (query(u<<1, L, M, l, r) + query(u<<1|1, M+1, R
            , l, r)) % p;
    } else {
        return 0;
    }
}

void update(int u, int L, int R, int l, int r, ll x, int type)
     {
    if (InRange(L, R, l, r)) {
        maketag(u, L, R, x, type);
    } else if (!OutofRange(L, R, l, r)) {
        int M = (L+R)>>1;
        pushdown(u, L, R);
        update(u<<1, L, M, l, r, x, type);
        update(u<<1|1, M+1, R, l, r, x, type);
        pushup(u);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ll n, m;
    cin >> n >> m >> p;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    build(1, 1, n);
    while (m--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1 || op == 2) {
            ll k;
            cin >> k;
            update(1, 1, n, x, y, k, op);
        } else {
            cout << query(1, 1, n, x, y) << endl;
        }
    }
    return 0;
}
```

2.13 主席树（可持久化线段树）

```
//对于单点历史修改与查询
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int maxn=(1e6+1);
int top,root[maxn],arr[maxn],n,m,cnt=0;
struct kkk{
    int l,r,val;
} a[maxn*30];
int build(int l,int r){
    int rt=++cnt;
    if(l==r)a[rt].val=arr[l];
    else {
        int mid=(l+r)>>1;
        a[rt].l=build(l,mid);
        a[rt].r=build(mid+1,r);
    }
    return rt;
}
int update(int ji,int jv,int l,int r,int i)//单点修改
{
    int rt=++cnt;
    a[rt].l=a[i].l;
    a[rt].r=a[i].r;
    if(l==r) a[rt].val=jv;
    else {
        int mid=(l+r)>>1;
        if(ji<=mid)a[rt].l=update(ji,jv,l,mid,a[rt].l);
        else a[rt].r=update(ji,jv,mid+1,r,a[rt].r);
    }
    return rt;
}
int query(int ji,int l,int r,int i){
    if(l==r) return a[i].val;
    int mid=(l+r)>>1;
    if(ji<=mid) return query(ji,l,mid,a[i].l);
    else return query(ji,mid+1,r,a[i].r);
}
signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>arr[i];
    }
    root[0]=build(1,n);
    for(int i=1,version,op,x,v;i<=m;i++){
        cin>>version>>op>>x;
        if(op==1){
            cin>>v;
            root[i]=update(x,v,1,n,root[version]);
        }else {
            root[i]=root[version];
            cout<<query(x,1,n,root[i])<<endl;
        }
    }
}
```

2.14 开点线段树

```
//适用于范围很大，但是操作较少
//用于减小空间复杂度
//如果操作过多，则与没有优化相同，因此可能被卡
//在线段树的合并与分裂中不可替代，树链剖分
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn=100001;
ll le[maxn],ri[maxn],sum[maxn],add[maxn],cnt;
void up(ll i, ll l, ll r){
    sum[i]=sum[l]+sum[r];
}
void lazy(ll i,ll v, ll n){
    sum[i]+=v*n;
    add[i]+=v;
}
void down(ll i, ll ln, ll rn){
    if(add[i]!=0){
     if(le[i]==0){
        le[i]=++cnt;
     }
    if(ri[i]==0){
        ri[i]=++cnt;
    }
    lazy(le[i],add[i],ln);
    lazy(ri[i],add[i],rn);
    add[i]=0;
    }
}
void Add(ll jl,ll jr,ll v,ll l,ll r,ll i){
     if(jl<=l&&r<=jr){
        lazy(i,v,r-l+1);
       return;
     }

    ll mid=(l+r)>>1;
     down(i,mid-l+1,r-mid);
     if(jl<=mid){
        if(le[i]==0){
            le[i]=++cnt;
        }
        Add(jl,jr,v,l,mid,le[i]);
```

```
     }
     if(jr>mid){
        if(ri[i]==0){
            ri[i]=++cnt;
        }
        Add(jl,jr,v,mid+1,r,ri[i]);
     }

    up(i,le[i],ri[i]);
}
ll query(ll jl,ll jr,ll l, ll r,ll i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }
    ll mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    ll ans=0;
    if(jl<=mid){
        if(le[i]!=0){
            ans+=query(jl,jr,l,mid,le[i]);
        }
    }
    if(jr>mid){
        if(ri[i]!=0){
            ans+=query(jl,jr,mid+1,r,ri[i]);
        }
    }
    return ans;
}
int main(){
    int n,m;
    cin>>n>>m;
    cnt=1;
    while(m--){
        int opt;
        cin>>opt;
        if(opt==1){
            int l,r,v;
            cin>>l>>r>>v;
            Add(l,r,v,1,n,1);
        }
    else {
        int l,r;
        cin>>l>>r;
        cout<<query(l,r,1,n,1)<<endl;
    }
}
}
```

2.15 权值线段树

```
//权值线段树模板
#define lson pos<<1
#define rson pos<<1|1
void build(int pos,int l,int r)
{
  int mid=(l+r)>>1;
  if(l==r)
  {
    tree[pos]=a[l];//a[l]表示数l有多少个
    return;
  }
  build(lson,l,mid);
  build(rson,mid+1,r);
  tree[pos]=tree[lson]+tree[rson];
}
void update(int pos,int l,int r,int k,int cnt)//表示数k的个数多cnt个
{
  int mid=(l+r)>>1;
  if(l==r)
  {
    tree[pos]+=cnt;
    return;
  }
  if(k<=mid)
    update(lson,l,mid,k,cnt);
  else
    update(rson,mid+1,r,k,cnt);
  tree[pos]=tree[lson]+tree[rson];
}
int query(int pos,int l,int r,int k)//查询数k有多少个
{
  int mid=(l+r)>>1;
  if(l==r)
    return tree[pos];
  if(k<=mid)
    return query(lson,l,mid,k);
  else
    return query(rson,mid+1,r,k);
}
int kth(int pos,int l,int r,int k)//查询第k大值是多少
{
  int mid=(l+r)>>1;
  if(l==r)
    return l;
  if(k<mid)
    return kth(rson,mid+1,r,k);
  else
    return kth(lson,l,mid,k-mid);
}
```

2.16 线段树上最值操作

```
//基本的实现
#include<bits/stdc++.h>
using namespace std;
const int maxn=1000001;
const int minest=-1000001;
long long arr[maxn<<2],sum[maxn<<2],mx[maxn<<2],sem[maxn<<2],
     cnt[maxn<<2];
void up(int i){
    int l=i<<1;
    int r=i<<1|1;
    sum[i]=sum[l]+sum[r];
    mx[i]=max(mx[l],mx[r]);
    if(mx[l]>mx[r]){
        cnt[i]=cnt[l];
        sem[i]=max(sem[l],mx[r]);
    }else if(mx[r]>mx[l]){
        cnt[i]=cnt[r];
        sem[i]=max(sem[r],mx[l]);
    }else {
        sem[i]=max(sem[l],sem[r]);
        cnt[i]=cnt[l]+cnt[r];
    }
}
void Lazy(int i,int v){
    if(v<mx[i]){
        sum[i]-=(mx[i]-v)*cnt[i];
        mx[i]=v;
    }
}
void down(int i){
    Lazy(i<<1,mx[i]);
    Lazy(i<<1|1,mx[i]);
}
void setmin(int jl,int jr,int v,int l,int r,int i){
    if(v>mx[i]) return;
    if(jl<=l&&r<=jr&&v>sem[i]){
        Lazy(i,v);
        return;
    }else {
        down(i);
        int mid=(l+r)>>1;
        if(jl<=mid) setmin(jl,jr,v,l,mid,i<<1);
        if(jr>mid) setmin(jl,jr,v,mid+1,r,i<<1|1);
    }
    up(i);
}
long long querymax(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return mx[i];
    }
    down(i);
    int mid=(l+r)>>1;
    long long lm=minest,rm=minest;
    if(jl<=mid) lm=querymax(jl,jr,l,mid,i<<1);
    if(jr>mid)  rm=querymax(jl,jr,mid+1,r,i<<1|1);
    return max(lm,rm);
}
long long querysum(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }
    down(i);
```

```
    int mid=(l+r)>>1;
    long long ans=0;
    if(jl<=mid) ans+=querysum(jl,jr,l,mid,i<<1);
    if(jr>mid)  ans+=querysum(jl,jr,mid+1,r,i<<1|1);
    return ans;
}
void build(int l,int r,int i){
    if(l==r){
        sum[i]=arr[l];
        mx[i]=arr[l];
        sem[i]=minest;
        cnt[i]=1;
        return;
    }
    int mid=(l+r)>>1;
    build(l,mid,i<<1),build(mid+1,r,i<<1|1);
    up(i);
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin>>t;
    while(t--){
        int n,m;
        cin>>n>>m;
        for(int i=1;i<=n;i++){
            cin>>arr[i];
        }
        build(1,n,1);
        while(m--){
            int opt;
            cin>>opt;
            if(opt==0){
                int l,r;
                long long mi;
                cin>>l>>r>>mi;
                setmin(l,r,mi,1,n,1);
            }else if(opt==1){
                int l,r;
                cin>>l>>r;
                cout<<querymax(l,r,1,n,1)<<endl;
            }else {
                int l,r;
                cin>>l>>r;
                cout<<querysum(l,r,1,n,1)<<endl;
            }
        }
    }
}
```

2.17 线段树上序列操作

```
//线段树上的序列操作，稍难一点
//求连续1的最长子列，有点难度，容易打错

#include<bits/stdc++.h>
using namespace std;
const int maxn=100001;
int arr[maxn],sum[maxn<<2],pre0[maxn<<2],len0[maxn<<2],suf0[
     maxn<<2],pre1[maxn<<2],len1[maxn<<2],suf1[maxn<<2];
int lazy[maxn<<2];
bool update[maxn<<2];
bool rever[maxn<<2];
void up(int i,int ln,int rn){
    int l=i<<1;
    int r=i<<1|1;
    sum[i]=sum[l]+sum[r];
    len0[i]=max(max(len0[l],len0[r]),suf0[l]+pre0[r]);
    pre0[i]=len0[l]<ln?pre0[l]:pre0[l]+pre0[r];
    suf0[i]=len0[r]<rn?suf0[r]:suf0[l]+suf0[r];
    len1[i]=max(max(len1[l],len1[r]),suf1[l]+pre1[r]);
    pre1[i]=len1[l]<ln?pre1[l]:pre1[l]+pre1[r];
    suf1[i]=len1[r]<rn?suf1[r]:suf1[l]+suf1[r];
}
void updateLazy(int i,int v,int n){
    sum[i]=v*n;
    pre0[i]=suf0[i]=len0[i]=v==0?n:0;
    pre1[i]=suf1[i]=len1[i]=v==1?n:0;
    lazy[i]=v;
    update[i]=true;
    rever[i]=false;
}
void reverseLazy(int i,int n){
    sum[i]=n-sum[i];
    swap(pre0[i],pre1[i]);
    swap(len0[i],len1[i]);
    swap(suf0[i],suf1[i]);
    rever[i]=!rever[i];
}
void down(int i,int ln,int rn){
    if(update[i]){
        updateLazy(i<<1,lazy[i],ln);
        updateLazy(i<<1|1,lazy[i],rn);
        update[i]=false;
    }
    if(rever[i]){
        reverseLazy(i<<1,ln);
        reverseLazy(i<<1|1,rn);
        rever[i]=false;
    }
}
void build(int l,int r,int i){
    if(l==r){
        sum[i]=arr[l];
        pre0[i]=arr[l]?0:1;
        suf0[i]=arr[l]?0:1;
        len0[i]=arr[l]?0:1;
        pre1[i]=arr[l]?1:0;
        suf1[i]=arr[l]?1:0;
        len1[i]=arr[l]?1:0;
        return;
    }
    int mid=(l+r)>>1;
    build(l,mid,i<<1),build(mid+1,r,i<<1|1);
    up(i,mid-l+1,r-mid);
    rever[i]=0;
    update[i]=0;
}
void Update(int jl,int jr,int v,int l,int r,int i){
    if(jl<=l&&r<=jr){
        updateLazy(i,v,r-l+1);
    }else {
        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) Update(jl,jr,v,l,mid,i<<1);
        if(jr>mid) Update(jl,jr,v,mid+1,r,i<<1|1);
        up(i,mid-l+1,r-mid);
    }
}
void Reverse(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        reverseLazy(i,r-l+1);
    }else {
        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) Reverse(jl,jr,l,mid,i<<1);
        if(jr>mid) Reverse(jl,jr,mid+1,r,i<<1|1);
        up(i,mid-l+1,r-mid);
    }
}
tuple<int,int,int> querylength(int jl,int jr,int l,int r,int i
     ){
    if(jl<=l&&r<=jr){
        return make_tuple(len1[i],pre1[i],suf1[i]);
    }
    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    if(jr<=mid) return querylength(jl,jr,l,mid,i<<1);
    if(jl>mid) return querylength(jl,jr,mid+1,r,i<<1|1);
    tuple<int,int,int> l1=querylength(jl,jr,l,mid,i<<1);
    tuple<int,int,int> r1=querylength(jl,jr,mid+1,r,i<<1|1);
    int llen=get<0>(l1),lpre=get<1>(l1),lsuf=get<2>(l1);
    int rlen=get<0>(r1),rpre=get<1>(r1),rsuf=get<2>(r1);
    int len=max(max(llen,rlen),lsuf+rpre);
    int pre=lpre<mid-max(l,jl)+1?lpre:lpre+rpre;
    int suf=rsuf<min(r,jr)-mid?rsuf:rsuf+lsuf;
    return make_tuple(len,pre,suf);
}
int query(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }
    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    int ans=0;
    if(jl<=mid) ans+=query(jl,jr,l,mid,i<<1);
```

```
    if(jr>mid)  ans+=query(jl,jr,mid+1,r,i<<1|1);
    return ans;
}
int main(){
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++) cin>>arr[i];
    build(1,n,1);
    while(m--){
        int opt,l,r;
        cin>>opt>>l>>r;
        l+=1;
        r+=1;
        if(opt==0){
            Update(l,r,0,1,n,1);
        }else if(opt==1){
            Update(l,r,1,1,n,1);
        }else if(opt==2){
            Reverse(l,r,1,n,1);
        }else if(opt==3){
            cout<<query(l,r,1,n,1)<<endl;
        }else {
            tuple<int,int,int>a=querylength(l,r,1,n,1);
            cout<<get<0>(a)<<endl;
        }
    }
}
```

2.18 非经典线段树

```
// 这类题目一般是可以看出来使用线段树，但是不能满足一般的懒更新的条件
//简单的一般需要进行势能分析，再减枝就可以解决
//例如 洛谷 P4145，此题是开根号，换为取模同理
//P1471 求方差
#include<bits/stdc++.h>
using namespace std;
const int maxn=100001;
double a[maxn],sum[maxn<<2],pf[maxn<<2],lazy[maxn<<2];
void up(int i){
    sum[i]=sum[i<<1]+sum[i<<1|1];
    pf[i]=pf[i<<1]+pf[i<<1|1];
}

void Lazy(int i,double v,int n){
    lazy[i]+=v;
    pf[i]+=2*v*sum[i]+v*v*n;
    sum[i]+=v*n;
}
void down(int i,int ln,int rn){
    if(lazy[i]!=0){
        Lazy(i<<1,lazy[i],ln);
        Lazy(i<<1|1,lazy[i],rn);
        lazy[i]=0;
    }
}
void add(int jl,int jr,double v,int l,int r,int i){
    if(jl<=l&&jr>=r){
        Lazy(i,v,r-l+1);
        return;
    }
    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    if(jl<=mid){
        add(jl,jr,v,l,mid,i<<1);
    }
    if(jr>mid) add(jl,jr,v,mid+1,r,i<<1|1);
    up(i);
}
double query(int jl,int jr,int jt,int l,int r,int i){
    if(jl<=l&&jr>=r){
        if(jt){
            return pf[i];
        }else return sum[i];
    }
    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    double ans=0;
    if(jl<=mid) ans+=query(jl,jr,jt,l,mid,i<<1);
    if(jr>mid) ans+=query(jl,jr,jt,mid+1,r,i<<1|1);
    return ans;
}
void build(int l,int r,int i){
    if(l==r){
        sum[i]=a[l];
        pf[i]=a[l]*a[l];

    }
    else {
    int mid=(l+r)>>1;
    build(l,mid,i<<1);
    build(mid+1,r,i<<1|1);
    up(i);
    }
    lazy[i]=0;
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>a[i];
    }
    build(1,n,1);
    while(m--){
        int opt;
        cin>>opt;
        if(opt==1){
            int l,r;
            double v;
            cin>>l>>r>>v;
            add(l,r,v,1,n,1);
        }else if(opt==2){
            int l,r;
            cin>>l>>r;
            printf("%.4lf\n",query(l,r,0,1,n,1)/(r-l+1));
        }else {
            int l,r;
            cin>>l>>r;
            printf("%.4lf\n",query(l,r,1,1,n,1)/(r-l+1)-query(
                l,r,0,1,n,1)/(r-l+1)*query(l,r,0,1,n,1)/(r-l
                +1));
        }
    }
}
```

2.19 Kruskal 重构树

```
//Kruskal重构树：最小生成树上任意两点路径最大值=新节点LCA的权值
const int maxn=1e5+10,maxm=2*maxn,maxk=3*maxn,maxh=20;
struct edge{ int u,v,w; }e[maxk];
int n,m,fa[maxk],head[maxk],tot=1;
struct graph{ int t,n; }g[maxk];
void add(int u,int v){ g[++tot].n=head[u]; g[tot].t=v; head[u]=tot; }
int nodekey[maxk],cntu,dep[maxk],st[maxk][maxh];
int find(int x){ return x==fa[x]?x:fa[x]=find(fa[x]); }
bool cmp(edge x,edge y){ return x.w<y.w; }
void kruskalbuild(){
    for(int i=1;i<=n;i++) fa[i]=i;
    sort(e+1,e+m+1,cmp); cntu=n;
    for(int i=1,fx,fy;i<=m;i++){
        fx=find(e[i].u); fy=find(e[i].v);
        if(fx!=fy){
            fa[fx]=fa[fy]=++cntu; fa[cntu]=cntu;
            nodekey[cntu]=e[i].w;
            add(cntu,fx); add(cntu,fy);
        }
    }
}
void dfs(int u,int f){
    st[u][0]=f; dep[u]=dep[f]+1;
    for(int p=1;p<maxh;p++) st[u][p]=st[st[u][p-1]][p-1];
    for(int e=head[u];e;e=g[e].n) dfs(g[e].t,u);
}
int lca(int u,int v){
    if(dep[u]<dep[v]) swap(u,v);
    for(int p=maxh-1;p>=0;p--) if(dep[st[u][p]]>=dep[v]) u=st[u][p];
    if(u==v) return u;
    for(int p=maxh-1;p>=0;p--) if(st[u][p]!=st[v][p]){ u=st[u][p]; v=st[v][p]; }
    return st[u][0];
}
```

2.20 左偏树

```
//可并堆结构，两个堆的合并时间复杂度为Log（n）
//查询操作需要并查集的优化
//P3377,模板,删除一个堆中最小值
#include<bits/stdc++.h>
using namespace std;
const int maxn=100001;
int n,m;
int num[maxn],lef[maxn],righ[maxn],dis[maxn];
int father[maxn];
//father并不是代表该节点的父亲节点，而是代表并查集所对应的路径信息
//需要保证，并查集最上方代表节点=左偏树的头，也就是堆顶
void prepare(){
    dis[0]=-1;
    for(int i=1;i<=n;i++){
        lef[i]=0;
        righ[i]=0;
        dis[i]=0;
        father[i]=i;
    }
}
int find(int i){
    father[i]=father[i]==i?i:find(father[i]);
    return father[i];
}
int merge(int i,int j){
    if(i==0||j==0) return i+j;
    //维护小根堆，如果值一样，编号小的节点做头,大根堆同理
    if(num[i]>num[j]||(num[i]==num[j]&&i>j)){
        swap(i,j);
    }
    righ[i]=merge(righ[i],j);
    if(dis[lef[i]]<dis[righ[i]]){
        swap(lef[i],righ[i]);
    }
    dis[i]=dis[righ[i]]+1;
    father[lef[i]]=father[righ[i]]=i;
    return i;
}
//节点i一定是左偏树的头，在左偏树上删除节点i，返回新树的头结点编号
int pop(int i){
    father[lef[i]]=lef[i];
    father[righ[i]]=righ[i];
    father[i]=merge(lef[i],righ[i]);//保证删除节点后，能够正确再次查询
    lef[i]=righ[i]=dis[i]=0;
    return father[i];
}
int main(){
    cin>>n>>m;
    prepare();
    for(int i=1;i<=n;i++) cin>>num[i];
    for(int i=1;i<=m;i++){
        int op;
        cin>>op;
        if(op==1){
            int l,r;
            cin>>l>>r;
            if(num[l]!=-1&&num[r]!=-1){
                l=find(l),r=find(r);
                if(l!=r) merge(l,r);
            }
        }else {
            int x;
            cin>>x;
            if(num[x]==-1) cout<<-1<<endl;
            else {
            int ans=find(x);
            cout<<num[ans]<<endl;
            pop(ans);
            num[ans]=-1;
        }
    }
    }
}

//进一步，删除任意一个节点
//P4971
#include<bits/stdc++.h>
using namespace std;
#define  int long long
const int maxn=2000100;
int n,m;
int num[maxn],father[maxn],lef[maxn],righ[maxn],dis[maxn],up[maxn];
int t,w,k;
// up数组存储的就是节点真正的父节点
void prepare(){
    dis[0]=-1;
    for(int i=1;i<=n;i++){
        dis[i]=lef[i]=righ[i]=up[i]=0;
        father[i]=i;
    }
}
int find(int i){
    father[i]=father[i]==i?i:find(father[i]);
    return father[i];
}
int merge(int i,int j){
    if(i==0||j==0){
        return i+j;
    }
    //这段代码是维护的大根堆
    if(num[i]<num[j]||(num[i]==num[j]&&i>j)){
        swap(i,j);
    }
    righ[i]=merge(righ[i],j);
    up[righ[i]]=i;//多了一句
    if(dis[lef[i]]<dis[righ[i]]){
        swap(lef[i],righ[i]);
    }
    dis[i]=dis[righ[i]]+1;
    father[lef[i]]=father[righ[i]]=i;
    return i;
}
//删除任意节点，返回整棵树的头节点
//这是主要的改动
int remove(int i){
    int h=find(i);
    father[lef[i]]=lef[i];
    father[righ[i]]=righ[i];
    int s=merge(lef[i],righ[i]);
    int f=up[i];
    father[i]=s;
    up[s]=f;
    if(h!=i){
        father[s]=h;
        if(lef[f]==i) lef[f]=s;
        else righ[f]=s;
        for(int d=dis[s];dis[f]>d+1;f=up[f],d++){// log(n)
            if(dis[lef[f]]<dis[righ[f]]){
                swap(lef[f],righ[f]);
            }
            dis[f]=d+1;
        }
    }
    up[i]=lef[i]=righ[i]=dis[i]=0;
    return father[s];
}
void reduce(int i,int v){
    int h=remove(i);
    num[i]=max((int)0,num[i]-v);
    father[h]=father[i]=merge(h,i);
}
int compute(){
    int ans=0,mx=0;
    for(int i=1;i<=n;i++){
        if(father[i]==i) {
            ans+=num[i];
            mx=max(num[i],mx);
        }
    }
    if(w==2) ans-=mx;
    else if(w==3) ans+=mx;
    return ans;
}
void solve(){
    cin>>n>>m;
    prepare();
    for(int i=1;i<=n;i++){
        cin>>num[i];
    }
    for(int i=1;i<=m;i++){
        int op;
        cin>>op;
```

```
        if(op==2){
            int x;
            cin>>x;
            reduce(x,num[x]);
        }else if(op==3){
            int x,y;
            cin>>x>>y;
            reduce(find(x),y);
        }else {
            int x,y;
            cin>>x>>y;
            x=find(x),y=find(y);
            if(x!=y) merge(x,y);
        }
    }
    int ans=compute();
    if(ans==0) cout<<"Gensokyo"<<' ';
    else if(ans>k) cout<<"Hell"<<' ';
    else cout<<"Heaven"<<' ';
    cout<<ans<<endl;
}
signed main(){
ios::sync_with_stdio(0);
cin.tie(0), cout.tie(0);
    cin>>t>>w>>k;
    while(t--){
        solve();
    }
}
```

2.21 虚树的构建

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+10;
int h[maxn<<1],ne[maxn<<1],to[maxn<<1],tot1;//初始树
int n,q,k;
int hn[maxn<<1],nn[maxn<<1],tn[maxn<<1],tot2;//新树
int dep[maxn],st[maxn][20],dfn[maxn],dfncnt;
vector<int>arr,tmp;//关键点
bool iskey[maxn];
int cost[maxn],siz[maxn];
bool cmp(int x,int y){
    return dfn[x]<dfn[y];
}
void add1(int u,int v){
    ne[++tot1]=h[u];
    to[tot1]=v;
    h[u]=tot1;
}
void add2(int u,int v){
    nn[++tot2]=hn[u];
    tn[tot2]=v;
    hn[u]=tot2;
}
void dfs(int u,int f){
    dep[u]=dep[f]+1;
    dfn[u]=++dfncnt;
    st[u][0]=f;
    for(int i=1;i<20;i++){
        st[u][i]=st[st[u][i-1]][i-1];
    }
    for(int e=h[u];e;e=ne[e]){
        int v=to[e];
        if(v==f) continue;
        dfs(v,u);
    }
}
int getlca(int u,int v){
    if(dep[u]!=dep[v]){
        if(dep[u]<dep[v]) swap(u,v);
        for(int p=19;p>=0;p--){
            if(dep[st[u][p]]>=dep[v]){
                u=st[u][p];
            }
        }
    }
    if(u==v) return u;
    for(int p=19;p>=0;p--){
        if(st[u][p]!=st[v][p]){
            u=st[u][p];
            v=st[v][p];
        }
    }
    return st[u][0];
}
int buildvtree(){
    sort(arr.begin(),arr.end(),cmp);
    tmp=arr;
    for(int i=0;i<(int)arr.size()-1;i++){
        tmp.push_back(getlca(arr[i],arr[i+1]));
    }
    sort(tmp.begin(),tmp.end(),cmp);
    tmp.erase(unique(tmp.begin(),tmp.end()),tmp.end());
    tot2=0;
    for(int i=0;i<(int)tmp.size();i++){
        hn[tmp[i]]=0;
    }
    for(int i=0;i<tmp.size()-1;i++){
        add2(getlca(tmp[i],tmp[i+1]),tmp[i+1]);
    }
    return tmp[0];
}
```

3 图论

3.1 最短路

```
bitset<N> b[N];
for (int j = 1; j <= n; ++j) {  // 注意中转点在最外层
    for (int i = 1; i <= n; ++i) {
        if (b[i][j]) { b[i] |= b[j]; }
    }
}
```

3.2 判奇环（无向图）

```
#include <iostream>
#include <vector>
using namespace std;
bool dfs(int u, int c, vector<vector<int>>& adj, vector<int>&
     color) {
    color[u] = c;
    for (int v : adj[u]) {
        if (color[v] == -1) {               // 未染色，染相反
             色并递归
            if (!dfs(v, 1 - c, adj, color))
                return false;
        } else if (color[v] == c) {         // 邻接点同色，出
             现奇环
            return false;
        }
    }
    return true;
}

bool hasOddCycle(int n, vector<vector<int>>& adj) {
    vector<int> color(n, -1);
    for (int i = 0; i < n; ++i) {
        if (color[i] == -1) {
            if (!dfs(i, 0, adj, color))
                return true;
        }
    }
    return false;
}

int main() {
    int n, m;
    cout << "请输入顶点数和边数：";
    cin >> n >> m;

    vector<vector<int>> adj(n);
    cout << "请输入每条边的两个端点（编号从0开始）：" << endl;
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    if (hasOddCycle(n, adj))
        cout << "图中存在奇环" << endl;
    else
        cout << "图中不存在奇环（是二分图）" << endl;
```

```

    return 0;
}
```

3.3 最小生成树

```
int n, m;
vector<pii> G[N];
int kruskal()
{   // 默认是一张连通图
  vector <array<int,3>> e(m);
  for (auto& [w, u, v]: e) { cin >> u >> v >> w; }
  sort(e.begin(), e.end());
  int ans = 0, cnt = 0;
  d = dsu(n);  // 见并查集模板
  for (auto& [w, u, v]: e) {
    if (d.find(u) != d.find(v)) {
      d.merge(u, v);
      ans += w;
      ++cnt;
      // 以下两行为建图，完成后 G 中存放了最小生成树
      G[u].push_back( { v, w } );
      G[v].push_back( { u, w } );
    }
    if (cnt == n-1) { break; }
  }
  return ans;
}
```

3.4 拓扑排序

```
queue<int> q;
for (int i = 1; i <= n; ++i) {
  if (deg[i] == 0) { q.push(i); }
}
vector<int> topo;
while (!q.empty()) {
  int u = q.front(); q.pop();
  topo.push_back(u);
  for (int v: G[u]) {
    if (--deg[v] == 0) { q.push(v); }
  }
}
```

3.5 LCA

```
const int N=5e5+5, I=20;
int n, m, s, dep[N], f[I+5][N], mn[I+5][N];
vector <pii> G[N];

void dfs(int u, int fa)
{   // 预处理父节点、深度、路径上的最小值
    f[0][u] = fa;
    dep[u] = dep[fa] + 1;
    for (int i = 1; i <= I; ++i) {
        f[i][u] = f[i-1][ f[i-1][u] ];
        mn[i][u] = min( mn[i-1][u], mn[i-1][ f[i-1][u] ] );
    }
    for (auto [v, w]: G[u]) {
        if (v != fa) {
            mn[0][v] = w;
            dfs(v, u);
        }
    }
}

int LCA(int u, int v)
{   // 求 u,v 到其 LCA 路径上的最小值
    int ans = numeric_limits<int>::max();
    function<void(int&,int)> jump = [&](int &u, int i) {
        ans = min(ans, mn[i][u]);
        u = f[i][u];
    };
    if (dep[u] < dep[v]) { swap(u, v); }
    for (int i = I; i >= 0; --i) {
        if (dep[f[i][u]] >= dep[v]) { jump(u, i); }
    }
    if (u == v) { return ans; }
    for (int i = I; i >= 0; --i) {
        if (f[i][u] != f[i][v]) {
            jump(u, i); jump(v, i);
        }
    }
    jump(u, 0); jump(v, 0);
    return ans;
}
```

3.6 树的直径与中心

```
int d[N], mxd;
void dfs(int u, int fa)
{
  for (auto v: G[u]) {
    if (v != fa) {
      dfs(v, u);
      mxd = max(mxd, d[u]+d[v]+1);
      d[u] = max(d[u], d[v]+1);
    }
  }
}
```

3.7 点分治与树的重心

```
int n, siz[N], dis[N];
bool vis[N];
vector<pii> G[N];

void calcsiz(int u, int fa, int sum, int &core)
{   // 求树的重心，sum 表述当前树的大小
    siz[u] = 1;
    int mxz = 0;
    for (auto [v, w]: G[u]) {
        if (v != fa && !vis[v]) {
            calcsiz(v, u, sum, core);
            siz[u] += siz[v];
            mxz = max(mxz, siz[v]);
        }
    }
    mxz = max(mxz, sum-siz[u]);
    if (mxz * 2 <= sum) { core = u; }
}

void calcdis(int u, int fa)
{
    for (auto [v, w]: G[u]) {
        if (v != fa && !vis[v]) {
            dis[v] = dis[u] + w;
            calcdis(v, u);
        }
    }
}

void work(int u, int fa)
{
    vis[u] = true;
    for (auto [v, w]: G[u]) {
        if (v != fa && !vis[v]) {
            dis[v] = w;
            calcdis(v, u);
        }
    }
    for (auto [v, w]: G[u]) {
        if (v != fa && !vis[v]) {
            int sum = siz[v];
            int core = 0;
            calcsiz(v, u, sum, core);
            calcsiz(core, u, sum, core);  // 第二次调用更新 siz[]
            work(core, u);
        }
    }
}

int main()
{
    int core = 0;
    calcsiz(1, 0, n, core);
    calcsiz(core, 0, n, core);
    work(core, 0);
}
```

3.8 基环树

```
vector<int> G[N], cycle;
void dfs(int u, int fa)
{
    cycle.push_back(u);
    if (u == t) {
        sort(cycle.begin(), cycle.end());
        for (int x: cycle) { cout << x << " "; }
        exit(0);
    }
    for (int v: G[u]) {
        if (v != fa) { dfs(v, u); }
    }
    cycle.pop_back();
}
int main()
{
  dsu d(n);  // 见并查集板子
  for (int i = 1; i <= n; ++i) {
      int u, v;
      cin >> u >> v;
      G[u].push_back(v);
      G[v].push_back(u);
      if (d.same(u, v)) { s=u; t=v; }
      d.merge(u, v);
  }
  dfs(s, 0);
}
```

3.9 Hierholzer 求欧拉路

```
int n, deg[N<<1], cur[N<<1];
bool vis[N];
vector<int> ans;
vector<pair<int,int>> G[N<<1];

void dfs(int u)
{
    for (int &j = cur[u]; j < G[u].size(); ++j) {
        auto [v, i] = G[u][j];
        if (!vis[i]) {
            vis[i] = true;
            dfs(v);
            ans.push_back(i);
        }
    }
}

int main()
{
  int s = 1;
  while (s <= 2*n && deg[s] % 2 == 0) { ++s; }
  if (s > 2 * n) {
      s = 1;
      while (!deg[s]) { ++s; }
  }

  int cnt = 0;
  for (int i = 1; i <= 2*n; ++i) {
      cnt += deg[i] % 2;
  }
  if (cnt > 2) {
      cout << "NO\n"; return;
  }

  dfs(s);

  if (ans.size() != n) {
      cout << "NO\n";
  } else {
      cout << "YES\n";
      for (auto x: ans) {
          cout << x << " \n"[x == ans.back()];
      }
  }
}
```

3.10 DSU on tree

```
const int N=2e5+5;
int n, c[N], siz[N], hvs[N];
int L[N], R[N], dfn[N], totdfn;
vector<int> G[N];

void dfs_init(int u, int fa)
{   // 预处理 dfn 和重儿子
  siz[u] = 1;
  L[u] = ++totdfn;
  dfn[totdfn] = u;
  for (int v: G[u]) {
    if (v != fa) {
      dfs_init(v, u);
      siz[u] += siz[v];
      if (hvs[u] == 0 || siz[v] > siz[hvs[u]]) {
        hvs[u] = v;
      }
    }
  }
  R[u] = totdfn;
}

void add(int u, int dt)
{
  --ccnt[cnt[c[u]]];
  cnt[c[u]] += dt;
  ++ccnt[cnt[c[u]]];
}

void dfs_solve(int u, int fa, bool keep)
{
  for (auto v: G[u]) {  // 先做轻儿子，不保留
    if (v != fa && v != hvs[u]) {
      dfs_solve(v, u, false);
    }
  }
  if (hvs[u]) {  // 再做重儿子，保留
    dfs_solve(hvs[u], u, true);
  }
  for (auto v: G[u]) {  // 再把轻儿子的信息加上
    if (v != fa && v != hvs[u]) {
      for (int i = L[v]; i <= R[v]; ++i) {
        add(dfn[i], 1);
      }
    }
  }
  add(u, 1);
  ans += (cnt[c[u]] * ccnt[cnt[c[u]]] == siz[u]);
  if (keep) { return; }
  for (int i = L[u]; i <= R[u]; ++i) {
    add(dfn[i], -1);
  }
}
```

3.11 tarjan

```
struct SCC {
  int n, totdfn=0, colcnt=0;
  stack<int> stk;
  vector<int> dfn, low, col;
  vector<vector<int>> G;
  SCC(int n): n(n) {
    G.assign(n+1, {});
    dfn.assign(n+1, 0);
    low.assign(n+1, 0);
    col.assign(n+1, 0);
  }
  void add_edge(int u, int v) {
    G[u].push_back(v);
  }
  void dfs(int u) {
    stk.push(u);
    dfn[u] = low[u] = ++totdfn;
    for (auto v: G[u]) {
      if (!dfn[v]) {
        dfs(v);
        low[u] = min(low[u], low[v]);
      } else if (!col[v]) {
        low[u] = min(low[u], dfn[v]);
      }
    }
    if (low[u] == dfn[u]) {
      col[u] = ++colcnt;
      int v;
      do {
```

```
        v = stk.top(); stk.pop();
        col[v] = colcnt;
      } while(v != u);
    }
  }
  vector<int> work() {
    for (int i = 1; i <= n; ++i) {
      if (!dfn[i]) { dfs(i); }
    }
    return col;
  }
};
```

3.12 树哈希

```
using ull = unsigned long long;
const ull msk = chrono::steady_clock::now().time_since_epoch().count();

// ull h(ull x) { return x * x * x * 1237123 + 19260817; }
// ull f(ull x) { return h(x & ((1 << 31) - 1)) + h(x >> 31); }
ull shift(ull x)
{
    x ^= msk;
    x ^= x << 13;
    x ^= x >> 7;
    x ^= x << 17;
    x ^= msk;
    return x;
}

ull has[N];
void getHash(int u, int fa)
{
    has[u] = 1;
    for (auto v: G[u]) {
        if (v == fa) { continue; }
        getHash(v, u);
        has[u] += shift(has[v]);
    }
}
```

3.13 Bellman-Ford / SPFA（判负环）

```
//Bellman-Ford + SPFA优化, O(n*m), 可用于判负环
#include<bits/stdc++.h>
using namespace std;
const int maxn=2010,maxm=3010,inf=1e8;
int head[maxm<<1],n,m,cnt,upfill[maxn],dis[maxn];
queue<int>q; bool enter[maxn];
struct graph{ int next,to,w; }g[maxm<<1];
void add(int u,int v,int w){ g[cnt].next=head[u]; g[cnt].to=v; g[cnt].w=w; head[u]=cnt++; }
void build(int n){ cnt=1; while(!q.empty())q.pop();
    for(int i=1;i<=n;i++){ head[i]=0; upfill[i]=0; dis[i]=inf; enter[i]=0; } }
bool spfa(){ dis[1]=0; upfill[1]++; q.push(1); enter[1]=true;
    while(!q.empty()){ int x=q.front(); q.pop(); enter[x]=false;
        for(int ei=head[x];ei;ei=g[ei].next){ int v=g[ei].to,w=g[ei].w;
            if(dis[x]+w<dis[v]){ dis[v]=dis[x]+w;
                if(!enter[v]){ if(upfill[v]++==n)return true; enter[v]=true; q.push(v); } } } }
    return false; }
```

3.14 Floyd 全源最短路

```
//全源最短路 O(n^3), 允许负边但不能有负环
const int maxn=1001,inf=1e8; int dis[maxn][maxn];
void floyd(){ for(int b=1;b<=n;b++)for(int i=1;i<=n;i++)for(int j=1;j<=n;j++)
    if(dis[i][b]!=inf&&dis[b][j]!=inf&&dis[i][j]>dis[i][b]+dis[b][j])
        dis[i][j]=dis[i][b]+dis[b][j]; }
```

3.15 树上倍增

```
//快速查询从节点往上走k步的祖先, 预处理O(n log n), 查询O(log n)
const int maxn=5e5+10; int st[maxn][32],deep[maxn],power;
void dfs(int i,int f){ deep[i]=(i==0?1:deep[f]+1); st[i][0]=f;
    for(int p=1;(1<<p)<=deep[i];p++) st[i][p]=st[st[i][p-1]][p-1];
    for(int ei=head[i];ei;ei=tr[ei].n) dfs(tr[ei].t,i); }
int getkthAncestor(int i,int k){ if(deep[i]<=k)return -1; int s=deep[i]-k;
    for(int p=power;p>=0;p--) if(deep[st[i][p]]>=s) i=st[i][p]; return i; }
```

3.16 树的直径

```
//两次DFS求树的直径
int maxd=0,far=1;
void dfs(int u,int fa,int d){ if(d>maxd){maxd=d;far=u;} for(int v:g[u])if(v!=fa)dfs(v,u,d+1); }
// dfs(1,0,0); dfs(far,0,0); maxd即为直径
```

3.17 树的重心

```
void dfs(int u, int fa, int &centroid) {
    sz[u] = 1;
    bool ok = true;
    for (int v : g[u]) {
        if (v == fa) continue;
        dfs(v, u, centroid);
        sz[u] += sz[v];
        if (sz[v] > n / 2) ok = false;
    }
    if (n - sz[u] > n / 2) ok = false;
    if (ok) centroid = u;
}
```

3.18 最大流

```
const ll INF = 9e18;
struct Flow {
    vector<tuple<int,ll,int>> G[N];  // 终点, 流量, 反边
    int dis[N], now[N];  // now[u] 为当前弧
    void add_edge(int u, int v, ll w) {
        G[u].push_back( { v, w, G[v].size() } );
        G[v].push_back( { u, 0, G[u].size()-1 } );
    }
    void bfs(int st) {  // 求层次图
        queue<int> q;
        memset(dis, 0, sizeof(dis));
        q.push(st); dis[st] = 1;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto& [v, w, inv]: G[u]) {
                if (w != 0 && dis[v] == 0) {
                    dis[v] = dis[u] + 1;
                    q.push(v);
                }
            }
        }
    }
    ll dfs(int u, int t, ll flow) {  // 求阻塞流
        if (u == t) { return flow; }
        for (int &i=now[u]; i < (int)G[u].size(); ++i) {
            auto& [v, w, inv] = G[u][i];
            if (w != 0 && dis[v] > dis[u]) {
                ll d = dfs(v, t, min(flow, w));
                if (d > 0) {
                    w -= d;
                    get<1>(G[v][inv]) += d;
                    return d;
                }
            }
        }
        return 0;
    }
    ll dicnic(int st, int ed) {
        for (ll flow=0, res;;) {
            bfs(st);
            if (dis[ed] == 0) { return flow; }
            memset(now, 0, sizeof(now));
            while ((res=dfs(st,ed,INF)) > 0) {
                flow += res;
            }
```

```
        }
    }
};
```

3.19 最近公共祖先（LCA）

```
//利用树链剖分的方法
#include<bits/stdc++.h>
using namespace std;
const int N=5e5+5;
struct node{
  int next,to;
}e[N<<1];
int tot,head[N];
int dep[N],siz[N],hson[N],fa[N];
int top[N];
int n,m,s;
void add(int x,int y){
  e[++tot].to=y;
  e[tot].next=head[x];
  head[x]=tot;
}
void dfs1(int x,int f){
  siz[x]=1;
  fa[x]=f;
  dep[x]=dep[f]+1;
  for(int i=head[x];i;i=e[i].next){
    int y=e[i].to;
    if(y!=f){
      dfs1(y,x);
      siz[x]+=siz[y];
      if(siz[hson[x]]<siz[y] || !hson[x]){
        hson[x]=y;
      }
    }
  }
}
void dfs2(int x,int t){
  top[x]=t;
  if(!hson[x])return ;
  dfs2(hson[x],t);
  for(int i=head[x];i;i=e[i].next){
    int y=e[i].to;
    if(y!=fa[x] && y!=hson[x]){
      dfs2(y,y);
    }
  }
}
int LCA(int x,int y){
  while(top[x]!=top[y]){
    if(dep[top[x]]<dep[top[y]]){
      swap(x,y);
    }
    x=fa[top[x]];
  }
  return dep[x]<dep[y]?x:y;
}
int main(){
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  cout.tie(NULL);
  cin>>n>>m>>s;
  for(int i=1;i<n;i++){
    int x,y;
    cin>>x>>y;
    add(x,y);
    add(y,x);
  }
  dfs1(s,0);
  dfs2(s,s);
  for(int i=1;i<=m;i++){
    int a,b;
    cin>>a>>b;
    cout<<LCA(a,b)<<'\n';
  }
  return 0;
}
```

3.20 树链剖分

```
#include<algorithm>
#include<iostream>
#include<cstdlib>
#include<cstring>
#include<cstdio>
#define Rint register int
#define mem(a,b) memset(a,(b),sizeof(a))
#define Temp template<typename T>
using namespace std;
typedef long long LL;
Temp inline void read(T &x){
    x=0;T w=1,ch=getchar();
    while(!isdigit(ch)&&ch!='-')ch=getchar();
    if(ch=='-')w=-1,ch=getchar();
    while(isdigit(ch))x=(x<<3)+(x<<1)+(ch^'0'),ch=getchar();
    x=x*w;
}

#define mid ((l+r)>>1)
#define lson rt<<1,l,mid
#define rson rt<<1|1,mid+1,r
#define len (r-l+1)

const int maxn=200000+10;
int n,m,r,mod;
//见题意
int e,beg[maxn],nex[maxn],to[maxn],w[maxn],wt[maxn];
//链式前向星数组，w[]、wt[]初始点权数组
int a[maxn<<2],laz[maxn<<2];
//线段树数组、lazy操作
int son[maxn],id[maxn],fa[maxn],cnt,dep[maxn],siz[maxn],top[
    maxn];
//son[]重儿子编号,id[]新编号,fa[]父亲节点,cnt dfs_clock/dfs序,
    dep[]深度,siz[]子树大小,top[]当前链顶端节点
int res=0;
//查询答案

inline void add(int x,int y){//链式前向星加边
    to[++e]=y;
    nex[e]=beg[x];
    beg[x]=e;
}
//------------------------------------ 以下为线段树
inline void pushdown(int rt,int lenn){
  laz[rt<<1]+=laz[rt];
  laz[rt<<1|1]+=laz[rt];
    a[rt<<1]+=laz[rt]*(lenn-(lenn>>1));
    a[rt<<1|1]+=laz[rt]*(lenn>>1);
    a[rt<<1]%=mod;
    a[rt<<1|1]%=mod;
    laz[rt]=0;
}

inline void build(int rt,int l,int r){
    if(l==r){
        a[rt]=wt[l];
        if(a[rt]>mod)a[rt]%=mod;
        return;
    }
    build(lson);
    build(rson);
    a[rt]=(a[rt<<1]+a[rt<<1|1])%mod;
}

inline void query(int rt,int l,int r,int L,int R){
  if(L<=l&&r<=R){res+=a[rt];res%=mod;return;}
  else{
    if(laz[rt])pushdown(rt,len);
    if(L<=mid)query(lson,L,R);
    if(R>mid)query(rson,L,R);
  }
}

inline void update(int rt,int l,int r,int L,int R,int k){
    if(L<=l&&r<=R){
    laz[rt]+=k;
    a[rt]+=k*len;
  }
  else{
    if(laz[rt])pushdown(rt,len);
    if(L<=mid)update(lson,L,R,k);
    if(R>mid)update(rson,L,R,k);
    a[rt]=(a[rt<<1]+a[rt<<1|1])%mod;
  }
}
//--------------------------------以上为线段树
```

```
inline int qRange(int x,int y){
  int ans=0;
  while(top[x]!=top[y]){//当两个点不在同一条链上
    if(dep[top[x]]<dep[top[y]])swap(x,y);//把x点改为所在链顶端的深度更深的那个点
    res=0;
    query(1,1,n,id[top[x]],id[x]);//ans加上x点到x所在链顶端 这一段区间的点权和
    ans+=res;
    ans%=mod;//按题意取模
    x=fa[top[x]];//把x跳到x所在链顶端的那个点的上面一个点
  }
  //直到两个点处于一条链上
  if(dep[x]>dep[y])swap(x,y);//把x点深度更深的那个点
  res=0;
  query(1,1,n,id[x],id[y]);//这时再加上此时两个点的区间和即可
  ans+=res;
  return ans%mod;
}

inline void updRange(int x,int y,int k){//同上
  k%=mod;
  while(top[x]!=top[y]){
    if(dep[top[x]]<dep[top[y]])swap(x,y);
    update(1,1,n,id[top[x]],id[x],k);
    x=fa[top[x]];
  }
  if(dep[x]>dep[y])swap(x,y);
  update(1,1,n,id[x],id[y],k);
}

inline int qSon(int x){
  res=0;
  query(1,1,n,id[x],id[x]+siz[x]-1);//子树区间右端点为id[x]+siz[x]-1
  return res;
}

inline void updSon(int x,int k){//同上
  update(1,1,n,id[x],id[x]+siz[x]-1,k);
}

inline void dfs1(int x,int f,int deep){//x当前节点，f父亲，deep深度
  dep[x]=deep;//标记每个点的深度
  fa[x]=f;//标记每个点的父亲
  siz[x]=1;//标记每个非叶子节点的子树大小
  int maxson=-1;//记录重儿子的儿子数
  for(Rint i=beg[x];i;i=nex[i]){
    int y=to[i];
    if(y==f)continue;//若为父亲则continue
    dfs1(y,x,deep+1);//dfs其儿子
    siz[x]+=siz[y];//把它的儿子数加到它身上
    if(siz[y]>maxson)son[x]=y,maxson=siz[y];//标记每个非叶子节点的重儿子编号
  }
}

inline void dfs2(int x,int topf){//x当前节点，topf当前链的最顶端的节点
  id[x]=++cnt;//标记每个点的新编号
  wt[cnt]=w[x];//把每个点的初始值赋到新编号上来
  top[x]=topf;//这个点所在链的顶端
  if(!son[x])return;//如果没有儿子则返回
  dfs2(son[x],topf);//按先处理重儿子，再处理轻儿子的顺序递归处理
  for(Rint i=beg[x];i;i=nex[i]){
    int y=to[i];
    if(y==fa[x]||y==son[x])continue;
    dfs2(y,y);//对于每一个轻儿子都有一条从它自己开始的链
  }
}

int main(){
    read(n);read(m);read(r);read(mod);
    for(Rint i=1;i<=n;i++)read(w[i]);
    for(Rint i=1;i<n;i++){
        int a,b;
        read(a);read(b);
        add(a,b);add(b,a);
    }
    dfs1(r,0,1);
    dfs2(r,r);
    build(1,1,n);
  while(m--){
        int k,x,y,z;
        read(k);
        if(k==1){
            read(x);read(y);read(z);
            updRange(x,y,z);
        }
        else if(k==2){
            read(x);read(y);
            printf("%d\n",qRange(x,y));
        }
        else if(k==3){
            read(x);read(y);
            updSon(x,y);
        }
        else{
            read(x);
            printf("%d\n",qSon(x));
      }
    }
}
```

3.21 最大路匹配

```
#include<bits/stdc++.h>
using namespace std;
const int N=510;
vector<int>g[N];
bool vis[N];
int match[N];
bool dfs(int u){
    for(int v:g[u]){
        if(!vis[v]){
            vis[v]=1;
            if(match[v]==0||dfs(match[v])){
                match[v]=u;
                return true;
            }
        }
    }
    return false;
}
int main(){
    int n,m,ans=0;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        int x;
        cin>>x;
        for(int j=1;j<=x;j++){
            int y;
            cin>>y;
            g[i].push_back(y);
        }
    }
    for(int i=1;i<=n;i++){
        memset(vis,0,sizeof(vis));
        if(dfs(i)) ans++;
    }
    cout<<ans;
}
```

3.22 Dijkstra 算法

```
//小根堆实现较为简单，这里采用以点为复杂度的反向索引堆的优化，一般不会用
#include<bits/stdc++.h>
using namespace std;
const int maxn=100010;
const int inf=1e8;
struct graph{
    int next,to,w;
}g[maxn*2];
int head[maxn],cnt,n,m;
int heap[maxn];//反向索引堆
int where[maxn];//表示点在堆上位置，初始化为-1
int heapSize;
int dis[maxn];
void build(int n){
    cnt=1;
    heapSize=0;
    for(int i=1;i<=n;i++){
        heap[i]=0;
```

```
        where[i]=-1;
        head[i]=0;
        dis[i]=inf;
    }
}
void add(int u,int v,int w){
    g[cnt].next=head[u];
    g[cnt].to=v;
    g[cnt].w=w;
    head[u]=cnt++;
}
void Swap(int i,int j){
    swap(heap[i],heap[j]);
    where[heap[i]]=i;
    where[heap[j]]=j;
}
void heapInsert(int i){
    while(dis[heap[i]]<dis[heap[(i-1)/2]]){
        Swap(i,(i-1)/2);
        i=(i-1)/2;
    }
}
void addorupdateoringnore(int v,int c){
    if(where[v]==-1){
        heap[heapSize]=v;
        where[v]=heapSize++;
        dis[v]=c;
        heapInsert(where[v]);
    }else if(where[v]>=0){
        dis[v]=min(dis[v],c);
        heapInsert(where[v]);
    }
}
void heapify(int i){
    int l=1;
    while(l<heapSize){
        int best=l+1<heapSize&&dis[heap[l+1]]<dis[heap[l]]?l
            +1:l;
        best=dis[heap[best]]<dis[heap[i]]?best:i;
        if(best==i) break;
        Swap(best,i);
        i=best;
        l=i*2+1;
    }
}
bool isEmpty(){
    return heapSize==0;
}
int pop(){
    int ans=heap[0];
    swap(heap[0],heap[--heapSize]);
    heapify(0);
    where[ans]=-2;
    return ans;
}
int main(){
    cin>>n>>m;
    int s;
    cin>>s;
    build(n);
    for(int i=1;i<=m;i++){
        int u,v,w;
        cin>>u>>v>>w;
        add(u,v,w);
    }
    addorupdateoringnore(s,0);
    while(!isEmpty()){
        int u=pop();
        for(int ei=head[u];ei>0;ei=g[ei].next){
            addorupdateoringnore(g[ei].to,dis[u]+g[ei].w);
        }
    }
    int ans=-inf;
    for(int i=1;i<=n;i++){
        /*if(dis[i]==inf){
            cout<<-1<<endl;
        }
        ans=max(ans,dis[i]);*/
        cout<<dis[i]<<' ';
    }
    //cout<<ans<<endl;
}
```

3.23 分层图最短路

```
//无论是在bfs上分层还是dijkstra上，本质其实就是把原图上的点与
    其的不同状态整合出更多的点
//难点在于如何将点上的状态给储存下来，以及如何转移
#include<bits/stdc++.h>
using namespace std;
const int maxn=50010;
const int inf=1e8;
struct dot{
    int x,cost,k;
};
struct graph{
    int next,to,w;
}g[maxn<<1];
struct compare{
   bool operator()(dot x,dot y){
        return x.cost>y.cost;
        // 小根堆
    }
};
int head[maxn<<1],cnt=1,n,m,k,dis[maxn][20];
bool vis[maxn][20];
priority_queue<dot,vector<dot>,compare>q;
void add(int u,int v,int w){
    g[cnt].next=head[u];
    g[cnt].to=v;
    g[cnt].w=w;
    head[u]=cnt++;
}
int main(){
    cin>>n>>m>>k;
    int s,t;
    cin>>s>>t;
    for(int i=1;i<=m;i++){
        int u,v,w;
        cin>>u>>v>>w;
        add(u,v,w);
        add(v,u,w);
    }
    for(int i=0;i<=n;i++){
        for(int j=0;j<=k;j++){
            dis[i][j]=inf;
        }
    }
    dis[s][0]=0;
    q.push({s,0,0});
    while(!q.empty()){
        int xn=q.top().x;
        int cost=q.top().cost;
        int kn=q.top().k;
        q.pop();
        if(vis[xn][kn]) continue;
        vis[xn][kn]=1;
        if(xn==t){
            cout<<cost<<endl;
            return 0;
        }
        for(int i=head[xn];i>0;i=g[i].next){
            int v=g[i].to;
            if(kn<k&&cost<dis[v][kn+1]){
                dis[v][kn+1]=cost;
                q.push({v,cost,kn+1});
            }
            if(cost+g[i].w<dis[v][kn]){
                dis[v][kn]=cost+g[i].w;
                q.push({v,cost+g[i].w,kn});
            }
        }
    }
}
/*
struct Compare {
    bool operator()(const State& a, const State& b) {
        return a.cost > b.cost; // 小根堆
    }
};
    priority_queue<State, vector<State>, Compare> q;
    这样子定义也可以
*/
```

3.24 双向广搜

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll n,m;
ll an[45];
ll l[1<<20+1],r[1<<20+1];
ll f(int i,int e,ll s,ll w,ll a[],ll j){
    if(s>w) return j;
    if(i==e) a[j++]=s;
    else {
        j=f(i+1,e,s,w,a,j);
        j=f(i+1,e,s+an[i],w,a,j);
    }
    return j;
}
ll compute(){
    ll lsize=f(0,n>>1,0,m,l,0);
    ll rsize=f(n>>1,n,0,m,r,0);
    sort(l,l+lsize);
    sort(r,r+rsize);
    ll ans=0;
    for(int i=lsize-1,j=0;i>=0;i--){
        while(j<rsize&&l[i]+r[j]<=m){
            j++;
        }
        ans+=j;
    }
    return ans;
}
int main(){
    cin>>n>>m;
  for(int i=0;i<n;i++) cin>>an[i];
  sort(an,an+n);
  cout<<compute();
}
```

3.25 tarjan(强连通分量)

```cpp
//强连通分量求解
int dfn[N], low[N], dfncnt, s[N], in_stack[N], tp;
int scc[N], sc;  // 结点 i 所在 SCC 的编号
int sz[N];       // 强连通 i 的大小

void tarjan(int u) {
  low[u] = dfn[u] = ++dfncnt, s[++tp] = u, in_stack[u] = 1;
  for (int i = h[u]; i; i = e[i].nex) {
    const int &v = e[i].t;
    if (!dfn[v]) {
      tarjan(v);
      low[u] = min(low[u], low[v]);
    } else if (in_stack[v]) {
      low[u] = min(low[u], dfn[v]);
    }
  }
  if (dfn[u] == low[u]) {
    ++sc;
    do {
      scc[s[tp]] = sc;
      sz[sc]++;
      in_stack[s[tp]] = 0;
    } while (s[tp--] != u);
  }
}
```

3.26 边双连通分量

```cpp
#include <algorithm>
#include <iostream>
#include <stack>
#include <vector>

using namespace std;
constexpr int N = 5e5 + 5, M = 2e6 + 5;
int n, m;

struct edge {
  int to, nt;
} e[M << 2];

int hd[N << 1], tot = 1;

void add(int u, int v) { e[++tot] = edge{v, hd[u]}, hd[u] =
    tot; }

void uadd(int u, int v) { add(u, v), add(v, u); }

int bfncnt, sum;
int dfn[N], low[N],bcc[N],bcc_cnt;
bool vis[N];
vector<vector<int>> ans;
stack<int> st;
void tarjan(int u, int in) {
  low[u] = dfn[u] = ++dfncnt;
  st.push(u), vis[u] = true;
  for (int i = hd[u]; i; i = e[i].nt) {
    int v = e[i].to;
    if (i == (in ^ 1)) continue;
    if (!dfn[v])
      tarjan(v, i), low[u] = min(low[u], low[v]);
    else if (vis[v])
      low[u] = min(low[u], dfn[v]);
  }
  if (dfn[u] == low[u]) {
    bcc_cnt++;
    bcc[u]=bcc_cnt;
    vector<int> t;
    t.push_back(u);
    while (st.top() != u)
      t.push_back(st.top()), bcc[st.top()]=bcc_cnt,vis[st.top
          ()] = false, st.pop();
    st.pop(), ans.push_back(t);
  }
}
```

3.27 割边

```cpp
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+10;
int n,m;
int dfn[maxn],low[maxn],dfncnt,cutedge[maxn<<1],tot=1,head[
    maxn<<1],ans1;
struct graph{
    int t,nex;
}e[maxn<<1];
void add(int u,int v){
    e[++tot].t=v;
    e[tot].nex=head[u];
    head[u]=tot;
}
void tarjan(int u,int pre) {
  low[u] = dfn[u] = ++dfncnt;
  for (int ei = head[u]; ei; ei = e[ei].nex) {
    if((ei^1)==pre) continue;
    const int &v = e[ei].t;
    if (!dfn[v]) {
      tarjan(v,ei);

      low[u] = min(low[u], low[v]);
      if(low[v]>dfn[u]){
        cutedge[ei>>1]=1;//无向图割边
        ans1++;
      }
    } else  {
      low[u] = min(low[u], dfn[v]);
    }
  }
}
int main(){
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        int u,v;
        cin>>u>>v;
        add(u,v);
        add(v,u);
    }
    for(int i=1;i<=n;i++){
        if(!dfn[i]) tarjan(i,0);
    }
    cout<<ans1<<endl;
    for(int i=1;i<=m;i++){
        if(cutedge[i]) cout<<i<<' ';
    }
}
```

3.28 点双连通分量

```cpp
#include <iostream>
#include <vector>
using namespace std;
constexpr int N = 5e5 + 5, M = 2e6 + 5;
int n, m;

struct edge {
  int to, nt;
} e[M << 1];

int hd[N], tot = 1;

void add(int u, int v) { e[++tot] = edge{v, hd[u]}, hd[u] = tot; }

void uadd(int u, int v) { add(u, v), add(v, u); }

int ans;
int dfn[N], low[N], bcc_cnt;
int sta[N], top, cnt;
bool cut[N];
vector<int> dcc[N];
int root;

void tarjan(int u) {
  dfn[u] = low[u] = ++bcc_cnt, sta[++top] = u;
  if (u == root && hd[u] == 0) {
    dcc[++cnt].push_back(u);
    return;
  }
  int f = 0;
  for (int i = hd[u]; i; i = e[i].nt) {
    int v = e[i].to;
    if (!dfn[v]) {
      tarjan(v);
      low[u] = min(low[u], low[v]);
      if (low[v] >= dfn[u]) {
        if (++f > 1 || u != root) cut[u] = true;
        cnt++;
        do dcc[cnt].push_back(sta[top--]);
        while (sta[top + 1] != v);
        dcc[cnt].push_back(u);
      }
    } else
      low[u] = min(low[u], dfn[v]);
  }
}

int main() {
  cin.tie(nullptr)->sync_with_stdio(false);
  cin >> n >> m;
  int u, v;
  for (int i = 1; i <= m; i++) {
    cin >> u >> v;
    if (u != v) uadd(u, v);
  }
  for (int i = 1; i <= n; i++)
    if (!dfn[i]) root = i, tarjan(i);
  cout << cnt << '\n';
  for (int i = 1; i <= cnt; i++) {
    cout << dcc[i].size() << ' ';
    for (int j = 0; j < dcc[i].size(); j++) cout << dcc[i][j] << ' ';
    cout << '\n';
  }
  return 0;
}
```

3.29 割点

```cpp
/*
洛谷 P3388 【模板】割点（割顶）
*/
#include <iostream>
#include <vector>
using namespace std;
int n, m;  // n: 点数 m: 边数
int dfn[100001], low[100001], idx, res;
// dfn: 记录每个点的时间戳
// low: 能不经过父亲到达最小的编号，idx: 时间戳，res: 答案数量
bool vis[100001], flag[100001];  // flag: 答案 vis: 标记是否重复
vector<int> edge[100001];        // 存图用的

void Tarjan(int u, int fa) {  // u 当前点的编号，fa 自己爸爸的编号
  vis[u] = true;              // 标记
  low[u] = dfn[u] = ++idx;    // 打上时间戳
  int child = 0;              // 每一个点儿子数量
  for (const auto &v : edge[u]) {  // 访问这个点的所有邻居 （C++11）
    if (!vis[v]) {
      child++;                        // 多了一个儿子
      Tarjan(v, u);                   // 继续
      low[u] = min(low[u], low[v]);  // 更新能到的最小节点编号
      if (fa != u && low[v] >= dfn[u] && !flag[u]) {  // 主要代码
        // 如果不是自己，且不通过父亲返回的最小点符合割点的要求，并且没有被标记过
        // 要求即为：删了父亲连不上去了，即为最多连到父亲
        flag[u] = true;
        res++;  // 记录答案
      }
    } else if (v != fa) {
      // 如果这个点不是自己的父亲，更新能到的最小节点编号
      low[u] = min(low[u], dfn[v]);
    }
  }
  // 主要代码，自己的话需要 2 个儿子才可以
  if (fa == u && child >= 2 && !flag[u]) {
    flag[u] = true;
    res++;  // 记录答案
  }
}

int main() {
  cin >> n >> m;                  // 读入数据
  for (int i = 1; i <= m; i++) {  // 注意点是从 1 开始的
    int x, y;
    cin >> x >> y;
    edge[x].push_back(y);
    edge[y].push_back(x);
  }  // 使用 vector 存图
  for (int i = 1; i <= n; i++)  // 因为 Tarjan 图不一定连通
    if (!vis[i]) {
      idx = 0;       // 时间戳初始为 0
      Tarjan(i, i);  // 从第 i 个点开始，父亲为自己
    }
  cout << res << endl;
  for (int i = 1; i <= n; i++)
    if (flag[i]) cout << i << " ";  // 输出结果
  return 0;
}
```

4 数学

4.1 线性筛

```cpp
vector<int> sieve(int n)
{
  vector<bool> vis(n+1);
    vector<int> mP(n+1,1);
  vector<int> prime;
  vis[1] = 1;
  for (int i = 2; i <= n; ++i) {
    if (!vis[i]) {
            prime.push_back(i);
            mP[i] = i;
        }
    for (auto p: prime) {
      if (i*p > n) { break; }
      vis[i*p] = 1;
            mP[i*p] = p;
      if (i%p == 0) { break; }  // 线性筛关键优化
    }
  }
  return prime;
}

// 分解质因数
map<int,int> cnt;
for (int y = x; y > 1; y /= mP[y]) {
    ++cnt[mP[y]];
}
```

4.2 因数

```
// const int X=1e5;
// vector<int> fac[X+5];
// for (int i = 1; i <= X; ++i) {
//     for (int j = i; j <= X; j += i) {
//         fac[j].push_back(i);
//     }
// }
vector<int> getFactors(int x){
    vector<pair<int,int>> fac;

    for(int p=2;p*p<=x;p++){
        if(x%p!=0) continue;

        int cnt=0;
        while(x%p==0){
            x/=p;
            cnt++;
        }

        fac.push_back({p,cnt});
    }

    if(x>1)
        fac.push_back({x,1});

    vector<int> factors={1};

    for(auto [p,cnt]:fac){
        int sz=factors.size();
        int mul=1;

        for(int i=1;i<=cnt;i++){
            mul*=p;

            for(int j=0;j<sz;j++)
                factors.push_back(factors[j]*mul);
        }
    }

    return factors;
}
```

4.3 拉格朗日插值

```
#include <iostream>
#include <vector>
constexpr int MOD = 998244353;
using LL = long long;

int inv(int k) {
  int res = 1;
  for (int e = MOD - 2; e; e /= 2) {
    if (e & 1) res = (LL)res * k % MOD;
    k = (LL)k * k % MOD;
  }
  return res;
}

// 返回 f 满足 f(x_i) = y_i
// 不考虑乘法逆元的时间，显然为 O(n^2)
std::vector<int> lagrange_interpolation(const std::vector<int> &x,
                                        const std::vector<int> &y) {
  const int n = x.size();
  std::vector<int> M(n + 1), px(n, 1), f(n);
  M[0] = 1;
  // 求出 M(x) = prod_(i=0..n-1)(x - x_i)
  for (int i = 0; i < n; ++i) {
    for (int j = i; j >= 0; --j) {
      M[j + 1] = (M[j] + M[j + 1]) % MOD;
      M[j] = (LL)M[j] * (MOD - x[i]) % MOD;
    }
  }
  // 求出 px_i = prod_(j=0..n-1, j!=i) (x_i - x_j)
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j)
      if (i != j) {
        px[i] = (LL)px[i] * (x[i] - x[j] + MOD) % MOD;
      }
  }
  // 组合出 f(x) = sum_(i=0..n-1)(y_i / px_i)(M(x) / (x - x_i))
  for (int i = 0; i < n; ++i) {
    LL t = (LL)y[i] * inv(px[i]) % MOD, k = M[n];
    for (int j = n - 1; j >= 0; --j) {
      f[j] = (f[j] + k * t) % MOD;
      k = (M[j] + k * x[i]) % MOD;
    }
  }
  return f;
}

int main() {
  std::ios::sync_with_stdio(false);
  std::cin.tie(nullptr);
  int n, k;
  std::cin >> n >> k;
  //求f（k）
  std::vector<int> x(n), y(n);
  for (int i = 0; i < n; ++i) std::cin >> x[i] >> y[i];
  const auto f = lagrange_interpolation(x, y);
  int v = 0;
  for (int i = n - 1; i >= 0; --i) v = ((LL)v * k + f[i]) %
      MOD;
  std::cout << v << '\n';
  return 0;
}
```

4.4 快速幂与乘法逆元

```
const int MOD=998244353;
ll qpow(ll a, ll b=MOD-2)
{   // 快速幂函数，x 的乘法逆元为 qpow(x)
    ll res=1;
    for (; b; b>>=1) {
        if (b & 1) { res=(res*a)%MOD; }
        a = a*a%MOD;  // 不要少 MOD
    }
    return res;
}
void Exgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) x = 1, y = 0;
    else Exgcd(b, a % b, y, x), y -= a / b * x;
}
int main() {
    ll x, y;
    Exgcd (a, p, x, y);
    x = (x % p + p) % p;
    printf ("%d\n", x); //x是a在mod p下的逆元
}
inv[1] = 1;
for(int i = 2; i < p; ++ i)
    inv[i] = (p - p / i) * inv[p % i] % p;
```

4.5 中国剩余定理

```
LL CRT(int k, LL* a, LL* r) {
  LL n = 1, ans = 0;
  for (int i = 1; i <= k; i++) n = n * r[i];
  for (int i = 1; i <= k; i++) {
    LL m = n / r[i], b, y;
    exgcd(m, r[i], b, y);  // b * m mod r[i] = 1
    ans = (ans + a[i] * m * b % n) % n;
  }
  return (ans % n + n) % n;
}
```

4.6 扩展欧几里得

```
LL CRT(int k, LL* a, LL* r) {
  LL n = 1, ans = 0;
  for (int i = 1; i <= k; i++) n = n * r[i];
  for (int i = 1; i <= k; i++) {
    LL m = n / r[i], b, y;
    exgcd(m, r[i], b, y);  // b * m mod r[i] = 1
    ans = (ans + a[i] * m * b % n) % n;
  }
  return (ans % n + n) % n;
}
```

4.7 排列组合数

```
const int N=1e5;
ll fac[N+5], Inv[N+5];
ll C(ll n, ll k) { return n==0? 1: fac[n] * Inv[k] % MOD * Inv[n-k] % MOD; }
ll P(ll n, ll k) { return n==0? 1: fac[n] * Inv[n-k] % MOD; }
// 预处理阶乘和阶乘的逆元
for (int i = 1; i <= N; ++i) {
  fac[i] = fac[i-1] * i % MOD;
  Inv[i] = qpow(fac[i]);
}
```

4.8 Lucas

```
const int MOD = 1000003;
ll C(ll a, ll b) {
  if (a < b) { return 0; }
  ll down = 1, up = 1;
  for (int i = a, j = 1; j <= b; --i, ++j) {
    up = up * i % MOD;
    down = down * j % MOD;
  }
  return up * qpow(down) % MOD;
}
ll lucas(ll a, ll b) {
  if (a < p && b < p) { return C(a, b); }
  return C(a%MOD, b%MOD) * lucas(a/MOD, b/MOD) % MOD;
}
```

4.9 卡特兰数

```
ll Cat(ll n) { return C(2*n, n) * qpow(n+1) % MOD; }
```

4.10 数论分块

```
ll l=1, r=0;
while (l <= n) {
    r = n / (n/l);
    // 统计 ans
    l = r + 1;
}
```

4.11 高斯消元

```
int Guess(vector<vector<db>> &a)
{
  int n = a.size();
    int c, r;
    for (c = 0, r = 0; c < n; ++c) {
        // 选主元
        int t = r;
        for (int i = r + 1; i < n; ++i) {
            if (fabs(a[i][c]) > fabs(a[t][c])) { t = i; }
        }
        if (sgn(a[t][c]) == 0) { continue; }

        // 行交换
        for (int i = c; i <= n; ++i) { swap(a[t][i], a[r][i]); }

        // 归一化
        for (int i = n; i >= c; --i) { a[r][i] /= a[r][c]; }

        // 前向消元
        for (int i = r + 1; i < n; ++i) {
            if (sgn(a[i][c]) != 0) {
                for (int j = n; j >= c; --j) {
                    a[i][j] -= a[r][j] * a[i][c];
                }
            }
        }
        ++r;
    }

    if (r < n) {
        for (int i = r; i < n; ++i) {
            if (sgn(a[i][n]) != 0) { return 2; }  // 无解
        }
        return 1;  // 无穷多解
    }

    // 回代
    for (int i = n - 1; i >= 0; --i) {
        for (int j = i + 1; j < n; ++j) {
            a[i][n] -= a[i][j] * a[j][n];
        }
    }
    return 0;
}
```

4.12 矩阵快速幂

```
using matrix = vector<vector<ll>>;
matrix IM(int n) {
    matrix I(n, vector<ll>(n));
    for (int i = 0; i < n; ++i) { I[i][i] = 1; }
    return I;
}
matrix operator* (matrix A, matrix B) {
    assert(A.front().size() == B.size());
    int n = A.size(), m = B.size(), k = B.front().size();
    matrix C(n, vector<ll>(m));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < k; ++j) {
            for (int x = 0; x < m; ++x) {
                C[i][j] = add(C[i][j], mul(A[i][x], B[x][j]));
            }
        }
    }
    return C;
}
matrix qpow(matrix a, ll b) {
    matrix res = IM(a.size());
    for (; b; b>>=1, a=a*a) {
        if (b & 1) { res = res * a; }
    }
    return res;
}
```

4.13 自适应辛普森法

```
double simpson(double l, double r) {
    double mid = (l + r) / 2;
    return (r - l) * (f(l) + 4 * f(mid) + f(r)) / 6;  // 辛普森公式
}

double asr(double l, double r, double eps, double ans,
           int step) {  // step是递归的下限
    double mid = (l + r) / 2;
    double fl = simpson(l, mid), fr = simpson(mid, r);
    if (abs(fl + fr - ans) <= 15 * eps && step < 0)
        return fl + fr + (fl + fr - ans) / 15;  // 足够相似的话就直接返回
    return asr(l, mid, eps / 2, fl, step - 1) +
           asr(mid, r, eps / 2, fr, step - 1);  // 否则分割成两段递归求解
}

double calc(double l, double r, double eps) {
    return asr(l, r, eps, simpson(l, r), 12);
}
```

4.14 多项式

```
struct cp {
    db x, y;
    cp(db real=0, db imag=0): x(real), y(imag) { };
    cp operator+ (cp b) const { return {x+b.x, y+b.y}; }
    cp operator- (cp b) const { return {x-b.x, y-b.y}; }
    cp operator* (cp b) const { return {x*b.x - y*b.y, x*b.y + y*b.x}; }
};
using vcp = vector<cp>;
using Poly = vector<int>;
namespace FFT {
    const db PI = acos(-1.0);
    vcp Omega(int L) {
        vcp w(L);
        w[1] = 1;
```

```
        for (int i = 2; i < L; i <<= 1) {
            auto w0 = w.begin() + i / 2;
            auto w1 = w.begin() + i;
            cp wn(cos(PI/i), sin(PI/i));
            for (int j = 0; j < i; j += 2) {
                w1[j] = w0[j>>1];
                w1[j+1] = w1[j] * wn;
            }
        }
        return w;
    }
    auto W = Omega(1 << 21);  // NOLINT
    void DIF(cp *a, int n) {
        cp x, y;
        for (int k = n>>1; k; k >>= 1) {
            for (int i = 0; i < n; i += k<<1) {
                for (int j = 0; j < k; ++j) {
                    x = a[i+j];
                    y = a[i+j+k];
                    a[i+j+k] = (a[i+j]-y) * W[k+j];
                    a[i+j] = x+y;
                }
            }
        }
    }
    void IDIT(cp *a, int n) {
        cp x, y;
        for (int k = 1; k < n; k <<= 1) {
            for (int i = 0; i < n; i += k << 1) {
                for (int j = 0; j < k; ++j) {
                    x = a[i+j];
                    y = a[i+j+k] * W[k+j];
                    a[i+j+k] = x-y;
                    a[i+j] = x+y;
                }
            }
        }
        const db Inv = 1.0 / n;
        for (int i = 0; i < n; ++i) {
            a[i].x *= Inv;
            a[i].y *= Inv;
        }
        reverse(a+1, a+n);
    }
}

namespace Polynomial {
    void DFT(vcp &a) { FFT::DIF(a.data(), a.size()); }
    void IDFT(vcp &a) { FFT::IDIT(a.data(), a.size()); }
    int norm(int n) { return 1 << (__lg(n-1) + 1); }
    vcp &dot(vcp &a, vcp &b) {
        for (int i = 0; i < int(a.size()); ++i) {
            a[i] = a[i] * b[i];
        }
        return a;
    }
    Poly operator+ (Poly a, Poly b) {
        int maxlen = max(a.size(), b.size());
        Poly ans(maxlen + 1);
        a.resize(maxlen + 1);
        b.resize(maxlen + 1);
        for (int i = 0; i < maxlen; ++i) { ans[i] = a[i] + b[i
            ]; }
        return ans;
    }
    Poly operator* (ll k, Poly a) {
        Poly ans;
        for (auto i: a) { ans.push_back(k*i); }
        return ans;
    }
    Poly operator* (Poly a, Poly b) {
        int n = a.size() + b.size() - 1;
        vcp c(norm(n));
        for (int i = 0; i < int(a.size()); ++i) { c[i].x = a[i
            ]; }
        for (int i = 0; i < int(b.size()); ++i) { c[i].y = b[i
            ]; }
        DFT(c);
        dot(c, c);
        IDFT(c);
        a.resize(n);
        for (int i = 0; i < n; ++i) {
            a[i] = int(c[i].y * 0.5 + 0.5);
        }
        return a;
    }
}
using namespace Polynomial;
```

4.15 快速幂

```
int qmi(int m, int k, int p)
{
    int res = 1 % p, t = m;
    while (k)
    {
        if (k&1) res = res * t % p;
        t = t * t % p;
        k >>= 1;
    }
    return res;
}
```

4.16 乘法逆元

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;

ll fpm(ll x, ll power, ll mod) {
    x %= mod;
    ll ans = 1;
    for (; power; power >>= 1, (x *= x) %= mod)
      if(power & 1) (ans *= x) %= mod;
    return ans;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,p;
    cin>>n>>p;
    for(int a=1;a<=n;a++){
  cout<< fpm(a, p - 2, p)<<endl;
    }//x为a在mod p意义下的逆元
}
```

4.17 组合数求法

```
#include <iostream>

using namespace std;

typedef long long LL;

LL qmi(LL a,int k, int p)
{
    LL res = 1;
    while(k)
    {
        if (k & 1) res = res * a % p;
        k >>= 1;
        a = a * a % p;
    }
    return res;
}

int C(int a,int b,int p)
{
    if(a < b) return 0;

    LL res = 1;
    for(int i = 1, j = a; i <= b; i ++ , j -- )
    {
        res = res * j % p;
        res = res * qmi(i, p - 2, p) % p;
    }
    return res;
}

LL lucas(LL a, LL b, int p)
{
    if(a < p && b < p) return C(a, b, p);
    return C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}

```

```cpp
int main()
{
    int n;
    cin >> n;
    while(n --)
    {
        LL a,b;
        int p;
        cin >> a >> b >> p;
        cout << lucas(a, b, p) << endl;
    }
    return 0;
}
```

4.18 线性基

```cpp
//普通消元法
void insert(long long x)
{
    for(int i=63;~i;i--) if((x>>i)&1ll)
    {
        if(!place[i]) {place[i]=x;return;}
        else x^=place[i];
    }
}
void insert(int *num)
{
    for(int j=63;~j;j--)
    {
        int p=0;
        for(int i=1;(!p)&&i<=n;i++)
            if((!v[i])&&((num[i]>>j)&1ll)) p=i;
        if(!p) {place[j]=0;continue;}
        v[p]=true,place[j]=num[p];
        for(int i=1;i<=n;i++) if(i!=p&&((num[i]>>j)&1ll))
            num[i]^=num[p];
    }
}
bool check(long long x)
{
    for(int i=63;~i;i--)
        if((x>>i)&1ll) x^=place[i];
    return x==0;
}
struct Base
{
    long long place[N];
    void insert(long long x)
    {
        for(int i=63;~i;i--) if((x>>i)&1ll)
        {
            if(!place[i]) {place[i]=x;return;}
            else x^=place[i];
        }
    }
};
void merge(Base A,Base &B)
{
    for(int i=63;~i;i--) if(A.place[i])
        B.insert(A.place[i]);
}
long long mx()
{
    long long ans=0;
    for(int i=63;~i;i--)
        if((ans^place[i])>ans)
            ans^=place[i];
    return ans;
}
long long mn()
{
    for(int i=0;i<=63;i++)
        if(place[i]) return place[i];
    return -1;
}
```

4.19 矩阵快速幂及高斯消元

```cpp
// 1.矩阵快速幂
#include <bits/stdc++.h>
using namespace std;
#define int long long
using ll = long long;
const int MOD = 1e9 + 7;
// 矩阵结构体
struct Matrix {
    int n, m; // n: 行数, m: 列数
    vector<vector<ll>> a;
    Matrix(int _n, int _m) : n(_n), m(_m), a(_n + 1, vector<ll
        >(_m + 1, 0)) {}

    static Matrix identity(int n) {
        Matrix I(n, n);
        for (int i = 1; i <= n; ++i) {
            I.a[i][i] = 1;
        }
        return I;
    }
};
// 矩阵乘法: C = A * B
// 时间复杂度: O(n^3) 或 O(A.n * A.m * B.m)
Matrix operator*(const Matrix& A, const Matrix& B) {
    Matrix C(A.n, B.m);
    for (int i = 1; i <= A.n; ++i) {
        for (int j = 1; j <= B.m; ++j) {
            for (int k = 1; k <= A.m; ++k) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j
                    ]) % MOD;
            }
        }
    }
    return C;
}

// 矩阵快速幂: A^p, 时间复杂度: O(n^3 * log p) , A必须是方阵
Matrix power(Matrix A, ll p) {
    Matrix res = Matrix::identity(A.n);
    while (p) {
        if (p & 1) res= res * A;
        A = A * A;
        p >>= 1;
    }
    return res;
}
signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int n, k; cin >> n >> k;
    Matrix A(n, n);
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n;j++){
            cin >> A.a[i][j];
        }
    }
    Matrix res = power(A, k);
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            cout << res.a[i][j] << " ";
        }
        cout << endl;
    }
}
```

4.20 线性筛求质因子个数（Omega 函数）

```cpp
const int MAXN = 1e7 + 10;
vector<int> primes;
vector<int> omega(MAXN, 0);
vector<bool> is_comp(MAXN, false);
void sieve_omega(int n) {
    omega[1] = 0;
    for (int i = 2; i <= n; ++i) {
        if (!is_comp[i]) { primes.push_back(i); omega[i] = 1;
            }
        for (int p : primes) {
            long long nxt = 1LL * i * p;
            if (nxt > n) break;
            is_comp[nxt] = true;
            omega[nxt] = omega[i] + 1;
            if (i % p == 0) break;
        }
    }
}
```

4.21 线性筛 (O(n))

```
int n = 100;
vector<int> minp(n + 1, 0), prime;
//minp存的是该数的最小质因数
for(int i = 2; i <= n; i++){
 if(minp[i] == 0){
 minp[i] = i;
 prime.push_back(i);
 }
 for(auto p : prime){
    if(i * p > n) break;
    minp[i * p] = p;
    if(minp[i] == p) break;
 }
}
```

4.22 位运算的部分性质

```
1. 基本代数性质
交换律
c
a & b == b & a
a | b == b | a
a ^ b == b ^ a
结合律
c
(a & b) & c == a & (b & c)
(a | b) | c == a | (b | c)
(a ^ b) ^ c == a ^ (b ^ c)
幂等律
c
a & a == a
a | a == a
a ^ a == 0
注意：异或不是幂等的。

单位元、零元
运算  单位元  零元
& M 全 1  0
| 0 M 全 1
^ 0 无零元，每个数的逆元是自身
例如：

c
a & M == a
a | 0 == a
a ^ 0 == a
a ^ a == 0
互补律
c
a & ~a == 0
a | ~a == M
a ^ ~a == M
双重非
c
~~a == a
吸收律
c
a & (a | b) == a
a | (a & b) == a
分配律
c
a & (b | c) == (a & b) | (a & c)
a | (b & c) == (a | b) & (a | c)
a & (b ^ c) == (a & b) ^ (a & c)
注意：^ 对 & 一般没有分配律，即：

c
a ^ (b & c) != (a ^ b) & (a ^ c)
德摩根定律
c
~(a & b) == ~a | ~b
~(a | b) == ~a & ~b
异或取反也有常用转换：

c
~(a ^ b) == ~a ^ b == a ^ ~b
~a ^ ~b == a ^ b
2. 与、或、非、异或之间的转换
与和或互相转换
c
a | b == ~(~a & ~b)
a & b == ~(~a | ~b)
异或用与、或、非表示
c
a ^ b == (a | b) & ~(a & b)
a ^ b == (a & ~b) | (~a & b)
a ^ b == (a | b) ^ (a & b)
a ^ b == (a | b) - (a & b)
或、与用异或表示
c
a | b == a ^ b ^ (a & b)
a & b == a ^ b ^ (a | b)
非与加减法的关系
c
~a == -a - 1
-a == ~a + 1
a + ~a == M
a ^ M == ~a
3. 与算术运算的关系
加法
c
a + b == (a ^ b) + 2 * (a & b)
a + b == (a | b) + (a & b)
其中：

a ^ b 是不考虑进位的和；

a & b 是进位。

减法
c
a - b == a + ~b + 1
-a == ~a + 1
与、或、异或的算术关系
c
a | b == a + b - (a & b)
a & b == a + b - (a | b)
a ^ b == a + b - 2 * (a & b)

a & b == (a + b - (a ^ b)) / 2
a | b == (a + b + (a ^ b)) / 2
不重叠位的加法
如果：

c
a & b == 0
那么：

c
a + b == a | b == a ^ b
4. 移位与乘除的关系
左移
c
a << n == a * 2^n
在固定位宽中相当于对 2^w 取模。

右移
对于无符号数或非负数：

c
a >> n == a / 2^n   // 向下取整
对于有符号补码数，算术右移等价于向下取整：

c
a >> n == floor(a / 2^n)
例如 -3 >> 1 == -2，而不是 C/C++ 中 -3 / 2 == -1，因为 C 语言
     除法向零取整。

取模 2^n
c
a % 2^n == a & ((1 << n) - 1)
清零低 n 位
c
a & ~((1 << n) - 1)
循环移位
以位宽 w 为例：

c
// 循环左移 n 位
(a << n) | (a >>> (w - n))

// 循环右移 n 位
(a >>> n) | (a << (w - n))
5. 位运算与集合表示
一个整数的二进制位可以表示一个集合：第 i 位为 1 表示元素 i 在
```

```
    集合中。

c
交集:   a & b
并集:   a | b
对称差: a ^ b
补集:   ~a & M
差集:   a & ~b
判断子集:

c
a 是 b 的子集:  (a & b) == a
            或:  (a | b) == b
            或:  (a & ~b) == 0
集合操作:

c
判断元素 i:    (a >> i) & 1
添加元素 i:    a | (1 << i)
删除元素 i:    a & ~(1 << i)
翻转元素 i:    a ^ (1 << i)
枚举一个集合的所有子集:

c
for (int s = a; s; s = (s - 1) & a) {
    // s 是 a 的一个非空子集
}
// 最后还要处理空集 0
6. 常用位操作恒等式
目标   表达式
判断奇偶  x & 1
乘 2  x << 1
除 2  x >> 1
判断是否为 2 的幂 x > 0 && (x & (x - 1)) == 0
获取第 k 位 (x >> k) & 1
设置第 k 位为 1 x | (1 << k)
清除第 k 位 x & ~(1 << k)
翻转第 k 位 x ^ (1 << k)
取最低位 1  x & -x
清除最低位 1  x & (x - 1)
取最低位 0  ~x & (x + 1)
将最低位 0 变为 1 x | (x + 1)
将末尾连续 1 清零 x & (x + 1)
将末尾连续 0 置 1 x | (x - 1)
得到最低位 1 及以下全 1 掩码  x ^ (x - 1)
判断相等  (a ^ b) == 0
交换 a、b a ^= b; b ^= a; a ^= b;
平均值防溢出  (a & b) + ((a ^ b) >> 1)
相反数  -x == ~x + 1
绝对值, w 位有符号数  mask = x >> (w - 1); abs = (x ^ mask) -
    mask;
统计 1 的个数 while (x) { x &= x - 1; count++; }
7. 注意事项
有符号与无符号
右移分算术右移和逻辑右移。算术右移补符号位, 逻辑右移补 0。

移位位数
C/C++ 中移位位数超过等于位宽是未定义行为; Java 等语言会取模。

优先级
位运算优先级容易出错, 建议始终加括号, 例如:

c
if ((x & 1) == 0)
补码性质
补码下:

-x == ~x + 1
~x == -x - 1
逻辑运算与位运算不同
&&、||、! 是逻辑运算, 结果只有 0 或 1, 且会短路;
&、|、^、~ 是按位运算。
如果 x,y 只取 0/1, 则:

x && y == x & y
x || y == x | y
!x == x ^ 1 == 1 - x
```

4.23 builtin

```
函数   功能   竞赛用途   典型代码
__builtin_popcount(x) 统计 32 位整数中 1 的个数 状压 DP、枚举
    子集、判断 2 的幂  int c = __builtin_popcount(mask);
__builtin_popcountll(x) 统计 64 位整数中 1 的个数 64 位掩码统
    计 int c = __builtin_popcountll(mask);
__builtin_clz(x)  统计 32 位前导零个数  求 log2(x)  int lg =
    31 - __builtin_clz(x);
__builtin_clzll(x)  统计 64 位前导零个数  求 64 位 log2(x)
    int lg = 63 - __builtin_clzll(x);
__builtin_ctz(x)  统计 32 位末尾零个数  求 lowbit、遍历掩码中
    的 1 int i = __builtin_ctz(mask);
__builtin_ctzll(x)  统计 64 位末尾零个数  64 位 lowbit  int i
    = __builtin_ctzll(mask);
__builtin_ffs(x)  返回最低位 1 的位置, 从 1 开始  找最低位 1
    的下标 int pos = __builtin_ffs(mask);
__builtin_ffsll(x)  64 位版 ffs 64 位找最低位 1 int pos =
    __builtin_ffsll(mask);
__builtin_parity(x) 判断 1 的个数奇偶性 奇偶校验  int p =
    __builtin_parity(mask);
__builtin_parityll(x) 64 位版 parity  64 位奇偶校验 int p =
    __builtin_parityll(mask);
```

5 字符串

5.1 前缀函数与 KMP

```
vector<int> prefixFunction(string s)
{   // fail : 1-index
    int n = s.length();
    vector<int> fail(n + 1);
    for (int i = 1, j = 0; i < n; ++i) {
        while (j && s[i] != s[j]) { j = fail[j]; }
        j += (s[i] == s[j]);
        fail[i + 1] = j;
    }
    return fail;
}
void kmp()
{
  string s, t;
  cin >> s >> t;
  int n = s.length();
  int m = t.length();
  auto fail = prefixFunction(t + "#" + s);
  for (int i = m + 2; i <= n + m + 1; ++i) {
      if (fail[i] == m) {
          cout << i - 2*m << "\n";
      }
  }
  for (int i = 1; i <= m; ++i) { cout << fail[i] << " \n"[i ==
      m]; }
}
```

5.2 Manacher

```
vector<int> manacher(string s)
{
    vector<int> d(s.length());
    d[0] = 1;
    int r=0, mid=0;
    for (int i = 1; i < s.length()-1; ++i) {  // 最后一个字符
        为防越界使用, 无需访问
        d[i] = 1;
        if (r >= i) { d[i] = min(d[2*mid-i], r-i+1); }
        while (s[i-d[i]] == s[i+d[i]]) { ++d[i]; }  // 需要保
            证不会越界
        if (i + d[i] > r) {
            r = i + d[i] - 1;
            mid = i;
        }
    }
    return d;
}
void solve()
{
    string s, t;
    cin >> s;
    for (auto c: s) {
        t.push_back('#');
        t.push_back(c);
    }
    t = "."+t+"#!";  // 首位添加两个不同的字符保证不会越界
    vector<int> d=manacher(t);
    cout << (*max_element(d.begin(), d.end()))-1 << endl;
}
```

5.3 字符串哈希

```
typedef pair<ll,ll> hs;
const ll MOD1=1e9+7, MOD2=1e9+9;
const hs p = { 117, 131 };
hs operator+ (hs a, hs b) {
    return hs{ (a.first+b.first)%MOD1, (a.second+b.second)%
        MOD2 };
}
hs operator- (hs a, hs b) {
    return hs{ (a.first-b.first+MOD1)%MOD1, (a.second-b.second
        +MOD2)%MOD2 };
}
hs operator* (hs a, hs b) {
    return hs{ a.first*b.first%MOD1, a.second*b.second%MOD2 };
}
struct Hash {
    int n;
    vector<hs> hs1, hs2, Pow;
    Hash(string s) {  // 0-index
        n = s.length();
        hs1.resize(n + 2);
        hs2.resize(n + 2);
        Pow.resize(n + 2);
        Pow[0] = { 1LL, 1LL };
        for (int i = 1; i <= n; ++i) {
            Pow[i] = Pow[i-1] * p;
        }
        for (int i = 1; i <= n; ++i) {
            hs1[i] = hs1[i-1] * p + hs{s[i-1]-'a'+1, s[i-1]-'a
                '+1};
        }
        for (int i = n; i; --i) {
            hs2[i] = hs2[i+1] * p + hs{s[i-1]-'a'+1, s[i-1]-'a
                '+1};
        }
    }
    hs get1(int l, int r) {
        return hs1[r] - hs1[l-1] * Pow[r-l+1];
    }
    hs get2(int l, int r) {
        return hs2[l] - hs2[r+1] * Pow[r-l+1];
    }
};
```

5.4 AC 自动机

```
struct AhoCorasick {
    static constexpr int ALPHABET = 26;
    struct Node {
        int len;
        int link;
        array<int,ALPHABET> next;
        Node(): len{0}, link{0}, next{} { }
    };

    vector<Node> t;

    AhoCorasick() {
        init();
    }

    void init() {
        t.assign(2, Node());
        t[0].next.fill(1);
        t[0].len = -1;
    }

    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }

    int add(const string &a) {
        int p = 1;
        for (auto c: a) {
            int x = c - 'a';
            if (t[p].next[x] == 0) {
                t[p].next[x] = newNode();
                t[t[p].next[x]].len = t[p].len + 1;
            }
            p = t[p].next[x];
        }
        return p;
    }

    void work() {
        queue<int> q;
        q.push(1);

        while (!q.empty()) {
            int x = q.front();
            q.pop();

            for (int i = 0; i < ALPHABET; ++i) {
                if (t[x].next[i] == 0) {
                    t[x].next[i] = t[t[x].link].next[i];
                } else {
                    t[t[x].next[i]].link = t[t[x].link].next[i
                        ];
                    q.push(t[x].next[i]);
                }
            }
        }
    }

    int next(int p, int x) {
        return t[p].next[x];
    }

    int link(int p) {
        return t[p].link;
    }

    int len(int p) {
        return t[p].len;
    }

    int size() {
        return t.size();
    }
};

constexpr int N = 2e6+5;
vector<int> G[N];
int cnt[N];

void dfs(int u)
{
    for (auto v: G[u]) {
        dfs(v);
        cnt[u] += cnt[v];
    }
}

void solve()
{
  int n;
  cin >> n;

    AhoCorasick AC;
    vector<int> pos(n + 1);
    for (int i = 1; i <= n; ++i) {
        string s;
        cin >> s;
        pos[i] = AC.add(s);
    }
    AC.work();

    string T;
    cin >> T;
    int p = 1;
    for (int i = 0; i < int(T.size()); ++i) {
        int x = T[i] - 'a';
        p = AC.next(p, x);
        ++cnt[p];
    }

    for (int i = 1; i < AC.size(); ++i) {
        G[AC.link(i)].push_back(i);
    }
    dfs(1);

    for (int i = 1; i <= n; ++i) {
        cout << cnt[pos[i]] << "\n";
    }
}
```

5.5 后缀自动机 SAM

```
//edited by piaoyun from some other's code
//必须#define int long long

struct SAM {
    static const int N=1e6+5, S=28;
    int tot=1, last=1, link[N<<1], ch[N<<1][S], len[N<<1],
        endpos[N<<1];
  // 总点数 tot，点的 index 属于 [1-tot]，空串/根为 1
  // last 为上一次插入的点
  // link 为点的 parent 树父节点 / 最长 出现位置与自己不同 的后缀
  // ch[n][s] 指节点 n 末尾加字符 s 所转移到的点
  // len 指该节点的串的 最长长度，注意到 最短长度 等于 len[link[n]] + 1 即父节点最长 + 1
  // endpos[n] 参考 get_endpos() 的注释
    void clear() {
        for (int i = 0; i <= tot; ++i) {
            link[i] = len[i] = endpos[i] = 0;
            for (int k = 0; k < S; ++k) { ch[i][k] = 0; }
        }
        tot = 1;
        last = 1;
    }

    // 延长一个字符，通常为 [1-26]
    void extend(int w) {
        int p = ++tot, x = last, r, q;
        endpos[p] = 1;
        for (len[last=p]=len[x]+1; x&&!ch[x][w]; x=link[x]) {
            ch[x][w]=p; }
        if (!x) { link[p] = 1; }
        else if (len[x] + 1 == len[q=ch[x][w]]) { link[p]=q; }
        else {
            link[r=++tot] = link[q];
            memcpy(ch[r], ch[q], sizeof(ch[r]));
            len[r] = len[x] + 1;
            link[p] = link[q] = r;
            for (; x && ch[x][w] == q; x = link[x]) { ch[x][w]
                = r; }
        }
    }

    // 注意 vector 占用的空间
    vector<int> p[N<<1];  //建立 parent 树，以便从上到下 dfs
    void dfs(int u) {
        int v;
        for (int i = 0; i < int(p[u].size()); ++i) {
            v = p[u][i];
            dfs(v);
            endpos[u] += endpos[v];
        }
    }

    //注意！在使用该方法前，endpos[] 代表每个点作为“终结点”的次数
  //使用该方法后，endpos[] 指在串中出现总次数，即原数组的子树求和
    void get_endpos() {
        for (int i = 1; i <= tot; ++i) { p[i].clear(); }
        for (int i = 2; i <= tot; ++i) {
            p[link[i]].push_back(i);
        }
        dfs(1);
        for (int i = 1; i <= tot; ++i) { p[i].clear(); }
    }

    // 在您不确定是否有抄写错误时再使用该方法
    // 必须在输入任何数据前自检，此前的数据会被清空
    static const int STC = 998244353;
    void self_test() {
        clear();
        for (int i = 1; i <= 1000; ++i) { extend(i * i % 26 +
            1); }
        ll tmp = 107 * last + 301 * tot;
        for (int i = 1; i <= tot; ++i) {
            tmp = (tmp * 33 + link[i] * 101 + len[i] * 97) %
                STC;
            for (int k = 1; k < S; ++k) { tmp = (tmp + k * ch[
                i][k]) % STC; }
        }
        assert("stage 1" && tmp == 393281314);   // stage1 :
            检查建树是否正确
        tmp = 0;
        get_endpos();
        for (int i = 1; i <= tot; ++i) { tmp = (tmp * 33 +
            endpos[i]) % STC; }
        assert("stage 2" && tmp == 178417668);   // stage2 :
            检查endpos计算是否正确，如果您修改了endpos[]的含义则会报错
        cout << "Self Test Passed. Remember to delete this
            function's use. \n";
        clear();
    }

  // 调试时可调用
    void debug_print() {
        for (int i = 1; i <= tot; ++i) {
            cerr << "node : " << i << " father : " << link[i]
                << " endpos : " << endpos[i] << " len : " <<
                len[i] << "\n";
        }
    }

    ll solve() {
    // 在这里输入你自己的解题逻辑
        ll ans = 0;
        get_endpos();
        for (int i = 1; i <= tot; ++i) {
            if (endpos[i] >= 2) { ans = max(ans, (ll)endpos[i
                ]*len[i]); }
        }
        return ans;
    }
} sam;

void solve()
{
    // sam.self_test();
    string s; cin >> s;
    for (auto c: s) { sam.extend(c-'a'+1); }
    cout << sam.solve() << "\n";
}
```

5.6 KMP 算法

```
#include<bits/stdc++.h>
using namespace std;
void get_nxt(string s, int nxt[]){
    int j = 0;
    nxt[0] = 0;
    for (int i = 1; i < s.length(); i++)
    {
        while(j>0&&s[i]!=s[j]) j=nxt[j-1];
        if(s[i]==s[j])j++;
        nxt[i]=j;
    }
}
void solve(){
    int cnt=0;
    string s, t;
    cin >> s >> t;
    int* nxt = new int[t.length()];
    get_nxt(t, nxt);
    int j=0;
    for (int i = 0; i < s.length(); i++) //这里的s代表主串
    {
        while(j > 0 && s[i] != t[j]) j = nxt[j-1];
        if(s[i]==t[j]) j++;
        if(j==t.length()){
            cout<<i-t.length()+2<<endl;
        }
    }
    for (int i = 0; i < t.length(); i++){
        cout<<nxt[i]<<' ';
    }
}
int main(){
    solve();
}
```

5.7 Trie 树

```
#include<bits/stdc++.h>
using namespace std;
```

```
int tree[3000005][70],pass[3000005],endp[3000005],cnt=1;
int getidx(char ch){
// 映射函数
if(ch>='a'&&ch<='z') return ch-'a';
if(ch>='A'&&ch<='Z') return ch-'A'+26;
return ch-'0'+52;
}
void insert(string s){//插入操作
 int cur=1;
 pass[cur]++;
    for(int i=0;i<s.length();i++){
        int idx=getidx(s[i]);
        if(tree[cur][idx]==0) {
            tree[cur][idx]=++cnt;
        }
        cur=tree[cur][idx];
        pass[cur]++;
    }
    endp[cur]++;
}
int search_whole_s(string s){
    //查询整个字符串出现次数
    int cur=1;
    for(int i=0;i<s.length();i++){
        int idx=getidx(s[i]);
        if(tree[cur][idx]==0) return 0;
        cur=tree[cur][idx];
    }
    return endp[cur];
}
int search_prefix_s(string s){
    //查询s作为前缀字符串出现次数
    int cur=1;
    for(int i=0;i<s.length();i++){
        int idx=getidx(s[i]);
        if(tree[cur][idx]==0) return 0;
        cur=tree[cur][idx];
    }
    return pass[cur];
}
void delete1(string s)
{
    if(search_whole_s(s)>0)
    {
    int cur=1;
    for(int i=0;i<s.length();i++)
    {
    int idx=getidx(s[i]);
    if(--pass[tree[cur][idx]]==0)
    {
    tree[cur][idx]==0;
    return;
    }
    cur=tree[cur][idx];
    }
    endp[cur]--;
}
}
void clear()
{
    for(int i=1;i<=cnt;i++)
    {
    for(int j=0;j<=69;j++) tree[i][j]=0;
    pass[i]=0;
    endp[i]=0;
    }
    cnt=1;
}
```

5.8 string 常用函数

```
三、基本属性
s.size();      // 长度（常用）
s.length();    // 同 size()
s.empty();     // 是否为空
四、访问与修改
s[i];          // 访问（不检查越界）
s.at(i);       // 访问（会检查越界）

s.front();     // 第一个字符
s.back();      // 最后一个字符
五、拼接与修改
1. 拼接
s += "abc";
s = s + "abc";
2. 插入
s.insert(2, "abc");   // 在下标2插入
3. 删除
s.erase(2, 3);        // 从下标2开始删除3个字符
4. 替换
s.replace(2, 3, "abc");   // 替换区间
六、子串操作
string t = s.substr(pos, len);  // 从pos开始，长度len

示例：

string s = "abcdef";
s.substr(2, 3);   // "cde"
七、查找操作（重点）
1. 查找子串
s.find("abc");     // 返回第一次出现的位置

找不到返回：

string::npos

示例：

if (s.find("abc") != string::npos) {
    // 找到了
}
2. 反向查找
s.rfind("abc");    // 从后往前找
3. 查找字符
s.find('a');
八、比较
if (s == t) {}
if (s < t) {}   // 字典序比较

底层是字典序（lexicographical order）

九、排序
#include <algorithm>

sort(s.begin(), s.end());

示例：

string s = "dcba";
sort(s.begin(), s.end());  // "abcd"
十、反转
reverse(s.begin(), s.end());
十一、大小写转换
#include <cctype>

for (char &c : s) c = toupper(c);
for (char &c : s) c = tolower(c);
十二、统计字符
int cnt = count(s.begin(), s.end(), 'a');
```

6 动态规划

6.1 数位 DP

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using db = long double;
using i128 = __int128;

const int L=18;
int a[L+5];
ll f[L+5][15];
ll dfs(int p, bool limit, bool lead0, int last)
{
  if (!p) { return 1; }
  auto& now = f[p][last];
  if (!limit && !lead0 && ~now) {
    return now;
  }
  int up = limit ? a[p] : 9;
  ll res = 0;
  for (int i = 0; i <= up; ++i) {
    if (!lead0 && abs(last-i) < 2) { continue; }
    res += dfs(p-1, limit&&(i==up), lead0&&(i==0), i);
  }
```

```
  if (!limit && !lead0) {
    now = res;
  }
  return res;
}
ll query(ll x)
{
  int len = 0;
  for (; x; x /= 10) {
    a[++len] = x % 10;
  }
  return dfs(len, true, true, 0);
}

void solve()
{
  ll a, b;
  cin >> a >> b;
  cout << query(b) - query(a-1) << "\n";
}

int main()
{
  ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
  memset(f, -1, sizeof(f));  // 不要忘记设为 -1
  int t = 1;
  // cin >> t;
  while (t--) { solve(); }
}
```

6.2 单调队列优化（滑动窗口）

```
auto work = [&](auto&& cmp) -> void {
  deque<pair<int,int>> q;

  auto update = [&](int id, int x) -> void {
    while (!q.empty() && cmp(x, q.back().second)) {
      q.pop_back();
    }
    q.emplace_back(id, x);
  };

  for (int i = 1; i <= k; ++i) {
    update(i, a[i]);
  }
  for (int i = 1, j = i+k-1; j <= n; ++i, ++j) {
    while (!q.empty() && q.front().first < i) { q.pop_front();
          }
    update(j, a[j]);
    cout << q.front().second << " \n"[j == n];
  }
};
```

6.3 无向图上回路计数

```
// CF11D
for (int i = 1; i <= n; ++i) {
  f[1<<(i-1)][i] = 1;
}
for (int s = 0; s < (1<<n); ++s) {
  for (int i = 1; i <= n; ++i) {
    if (!f[s][i]) { continue; }
    for (int j = 1; j <= n; ++j) {
      if (!G[i][j]) { continue; }
      if ((s&-s) > 1<<(j-1)) { continue; }  // 起点不能改
      if(s>>(j-1)&1) {
        if ((s&-s) == 1<<(j-1)) {  // 走回起点
          ans += f[s][i];
        }
      } else {
        f[s|(1<<(j-1))][j] += f[s][i];
      }
    }
  }
}
cout << (ans-m)/2 << "\n";  // 去除无向边，环正走反走算一个
```

6.4 SOSDP

```
for(int k=0;k<=19;k++)
        for(int i=0;i<=maxn;i++)
            if((i>>k)&1) g[i] += g[i^(1<<k)];   // i 是超集，有第 k 位
for(int k=0;k<=19;k++)
        for(int i=0;i<=maxn;i++)
            if(!((i>>k)&1)) g[i] += g[i^(1<<k)];   // i 是子集，无第 k 位
```

6.5 树形 DP

```
#include<bits/stdc++.h>
//#define int long long
#define in(a) a = read_int()
using namespace std;
const int N = 2e5 + 5;
const int inf = INT_MAX;
inline int read_int() {
  int x = 0;
  char ch = getchar();
  bool f = 0;
  while('9' < ch || ch < '0') f |= ch == '-' , ch = getchar();
  while('0' <= ch && ch <= '9') x = (x << 3) + (x << 1) + ch -
       '0' , ch = getchar();
  return f ? -x : x;
}
struct Edge {
  int n , v , w;
} edge[N << 1];
int head[N] , eid;
void eadd(int u , int v , int w) {
  edge[++ eid].n = head[u] , edge[eid].v = v , edge[head[u] =
      eid].w = w;
}
int n;
int l[N] , r[N] , ans[N];
void dfs(int u , int fa) { // 树形 DP 部分
  l[u] = 1; // 初始化左边界
  for(int i = head[u] ; i ; i = edge[i].n) {
    int v = edge[i].v , w = edge[i].w;
    if(v != fa) {
      r[v] = w - 1; // 初始化右边界
      dfs(v , u);
      r[u] = min(r[u] , w - l[v]); // 转移过程
      l[u] = max(l[u] , w - r[v]);
    }
  }
}
queue<int>q;
signed main() {
  in(n);
  for(int i = 1 ; i < n ; i ++) {
    int u , v , w;
    in(u) , in(v) , in(w);
    eadd(u , v , w) , eadd(v , u , w);
  }
  r[1] = 1e9; // 初始化根节点的右边界
  dfs(1 , 0); // 求每个点权的取值范围
  for(int i = 1 ; i <= eid ; i += 2) { // 判无解部分
    int u = edge[i].v , v = edge[i + 1].v , w = edge[i].w;
    if(r[u] + r[v] < w || l[u] > r[u] || l[v] > r[v]) {
            cout<<"NO";
      return 0;
    }
  }
  ans[1] = l[1]; // 若有解，根节点点权取左边界
  q.push(1);
  while(!q.empty()) { // 广搜部分
    int u = q.front();
    q.pop();
    for(int i = head[u] ; i ; i = edge[i].n) {
      int v = edge[i].v , w = edge[i].w;
      if(ans[v]) continue;
      ans[v] = w - ans[u]; // 递推求解
    }
  }
  cout<<"YES\n"; // 打印答案
  for(int i = 1 ; i <= n ; i ++) cout<<ans[i]<<' ';
  return 0;

}
/*
#include<bits/stdc++.h>
using namespace std;
const int maxn=10000;
```

```
int cnt=1,n,head[maxn],yes[maxn],no[maxn],num[maxn],b[maxn];
struct graph{
    int next,to;
}g[maxn];
void add(int u,int v){
    g[cnt].next=head[u];
    g[cnt].to=v;
    head[u]=cnt++;
}
void dfs(int i){
    no[i]=0;
    yes[i]=num[i];
    for(int ei=head[i],v;ei;ei=g[ei].next){
        v=g[ei].to;
        dfs(v);
        no[i]+=max(yes[v],no[v]);
        yes[i]+=no[v];
    }
}
int main(){
    cin>>n;
    for(int i=1;i<=n;i++) cin>>num[i];
    for(int i=1;i<n;i++){
        int u,v;
        cin>>u>>v;
        b[u]=1;
        add(v,u);

    }
    for(int i=1;i<=n;i++){
        if(!b[i]){
            dfs(i);
            cout<<max(yes[i],no[i])<<endl;
            break;
        }
    }
}
*/
```

6.6 最长不下降子序列

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1e5+9;
int n,k,m,a[N],b[N],dp[N],t[N<<2];

void updata(int i,int l,int r,int n,int val){
    t[i]=max(val,t[i]);
    if(l==r) return;
  int mid=(l+r)>>1;
  if(n<=mid) updata(i<<1,l,mid,n,val);
  else updata(i<<1|1,mid+1,r,n,val);
  return;
}
int getmax(int i,int l,int r,int nl,int nr){
  if(l==nl&&r==nr){
    return t[i];
  }
  int mid=(l+r)>>1;
  if(mid>=nr) return getmax(i<<1,l,mid,nl,nr);
  else if(mid<nl) return getmax(i<<1|1,mid+1,r,nl,nr);
  else return max(getmax(i<<1,l,mid,nl,mid),getmax(i<<1|1,mid
      +1,r,mid+1,nr));
}
void solve(){
    cin>>n>>k;
    for(int i=1;i<=n;i++)cin>>a[i],b[i]=a[i];
    // 离散化
    sort(b+1,b+n+1);
    m=unique(b+1,b+n+1)-b-1;
    for(int i=1;i<=n;i++)a[i]=lower_bound(b+1,b+m+1,a[i])-b;
    for(int i=1;i<=n;i++){
        dp[i]=getmax(1,1,m,1,a[i])+1;
        updata(1,1,m,a[i],dp[i]);
    }
    //到这里dp 中存储的是以a[i]结尾的最长不下降子序列
    int ans=0;
    memset(t,0,sizeof t);
    for(int i=n;i>k;i--){
        ans=max(ans,dp[i-k]+k-1+getmax(1,1,m,a[i-k],m)+1); //
            通过再次利用线段树找到往后最长（题目要求）
        int tem=getmax(1,1,m,a[i],m)+1;
        updata(1,1,m,a[i],tem);
    }
    ans=max(ans,k+getmax(1,1,m,a[k+1],m));
    cout<<ans<<'\n';
}

// 1 2 3 ... i-k i-k+1 i-k+2 i-k+3 ... i ...

int main(){
    ios::sync_with_stdio(false),cin.tie(nullptr),cout.tie(
        nullptr);
    int q=1;
    //cin>>q;
    while(q--){
        solve();
    }
    return 0;
}
```

7 计算几何

7.1 define

```
using ll = long long;
using db = double;
using T = ll;   // 或者 double

// 如果 T 是整数
int sgn(ll x) {
    return (x > 0) - (x < 0);
}

// 如果 T 是浮点数，用这个版本
/*
const double eps = 1e-9;
int sgn(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}
*/
```

7.2 向量

```
using T = ll;
struct Point {
  T x, y;
    Point(T x=0, T y=0): x(x), y(y) {}
  friend istream& operator>> (istream& is, Point& p) { return
      is>>p.x>>p.y; }
  friend ostream& operator<< (ostream& os, Point& p) { return
      os<<p.x<<" "<<p.y; }
  db ang() { return atan2(y, x); }
  bool operator== (const Point &p) const { return sgn(x-p.x)
      ==0 && sgn(y-p.y)==0; }
  bool operator< (const Point &p) const { return x<p.x || (x==
      p.x && y<p.y); }
  Point operator+ (const Point &p) const { return Point(x+p.x,
       y+p.y); }
    Point operator- (const Point &p) const { return Point(x-p.
       x, y-p.y); }
  Point operator* (const T &k) const { return Point(k*x, k*y);
       }
  Point operator/ (const T &k) const { return Point(x/k, y/k);
       }
  T operator* (const Point &p) const { return x*p.x + y*p.y; }
  T operator^ (const Point &p) const { return x*p.y - y*p.x; }
        // 叉乘时打括号
  T len2() { return (*this)*(*this); }
  db len() { return sqrtl(len2()); }  // 等价于 hypotl(x, y)
  Point rot(db ang) { return Point(x*cos(ang)-y*sin(ang), x*
      sin(ang)+y*cos(ang)); }
  Point trunc(db l) { return (*this) * (l/len()); }
};
using Vector = Point;

// 判断 c 是否在 ab 的逆时针方向
int toLeft(Point a, Point b, Point c) { return sgn((b-a)^(c-a)
     ); }

// 二维向量夹角
db getAngle(Vector a, Vector b) { return fabs(atan2(fabs(a^b),
      a*b)); }
```

7.3 点线操作

```
struct Line {
    Point a, b;
    Vector v;
  Line(Point a, Point b): a(a), b(b) { v = b - a; }
  int toLeft(Point p) { return sgn(v^(p-a)); }
  bool onSegment(Point p) { return (toLeft(p)==0) && ((p-a)*(p
      -b)<=0); }
  db distToLine(Point p) { return fabs((v^(p-a))/v.len()); }
  Point proj(Point p) { return a+v*((v*(p-a))/(v*v)); }
  // 用整数操作判断直线与点 C 的距离是否小于（等于）某个值 r
  // 常见于判断直线与圆的位置关系
  // 注意叉乘再相乘可能会爆 ll，所以这里用 __int128
  bool disLess(Point p, i128 r) {
      i128 xmult = (v^(p-a));
      return xmult*xmult <= r*r*v.len2();
  }
};

// 判断两条直线是否平行
bool isParallel(Line l1, Line l2)
{
  return sgn(l1.v^l2.v)==0 && sgn(l1.v^(l1.a-l2.a))!=0;
}

// 判断两条线段是否相交
// 先做两次跨立实验判断是否规范相交，再特判至少三点共线的情况
bool hasIntersection(Line L1, Line L2)
{
  // 特判三点共线的情况
    if (L1.onSegment(L2.a)) { return true; }
    if (L1.onSegment(L2.b)) { return true; }
    if (L2.onSegment(L1.a)) { return true; }
    if (L2.onSegment(L1.b)) { return true; }
    // 跨立实验判断两直线是否规范相交
    // 线段端点分布在直线两侧，“线段与线段/线段与直线/直线与
        直线”是否相交对应修改即可
  function<bool(Line,Point,Point)> crossStanding = [](Line L,
      Point a, Point b){
       return L.toLeft(a) * L.toLeft(b) == -1;
    };
    return crossStanding(L1, L2.a, L2.b) && crossStanding(L2,
        L1.a, L1.b);
}

// 求两直线交点
// 求之前应该先判断有没有交点，避免精度误差，同时还要特判共线
    的情况
Point getIntersection(Line L1, Line L2)
{
    Point P1=L1.a, P2=L2.a;
    Point v1=L1.v, v2=L2.v;
    return P1 + v1 * ( (v2^(P1-P2)) / (v1^v2) );
}

//点P到线段AB的距离公式
double distToSeg(Point p, Point a, Point b) {
    if(a == b) return (p-a).len();
    Point v1=b-a, v2=p-a, v3=p-b;
    if(sgn(v1*v2) < 0) return v2.len();
    if(sgn(v1*v3) > 0) return v3.len();
    return Line(a,b).distToLine(p);
}
```

7.4 多边形与凸包

```
struct Polygon: vector<Point> {
    using vector<Point>::vector;  // 直接使用 vector 的构造函
        数
    size_t nxt(size_t i) { return i+1==size()? 0: i+1; }
    size_t pre(size_t i) { return i==0? size()-1: i-1; }
  // 求多边形面积
  // 为避免精度误差，可以返回 2*S，即去掉 "0.5*()"
  db area() {
        db res = 0;
        for (size_t i = 0; i < size(); ++i) {
            res += 0.5 * ((*this)[i] ^ (*this)[nxt(i)]);
        }
        return fabs(res);
    }
    // Sunday's algorithm 光线回转法
    // 在整数范围内判断一个点是否在多边形（不保证凸包）内部
    // 亦可用于求某点相对于该多边形的回转数
    pair<bool,int> winding(Point p) {
        int wn = 0, n = this->size();
        for (int i = 0; i < n; ++i) {
            if (Line((*this)[i], (*this)[(i+1)%n]).onSegment(p
                )) {
              return { true, -inf };
            }
            int k = Line((*this)[i], (*this)[(i+1)%n]).toLeft(
                p);
            int d1 = sgn((*this)[i].y - p.y);
            int d2 = sgn((*this)[(i + 1) % n].y - p.y);
            if (k>0 && d1<=0 && d2>0) { wn++; }
            if (k<0 && d2<=0 && d1>0) { wn--; }
        }
        return { wn!=0, wn };
    }
};
```

7.5 计算几何模板

```
#include <bits/stdc++.h>
using namespace std;

using point_t=long double;  //全局数据类型，可修改为 long long
    等

constexpr point_t eps=1e-8;
constexpr long double PI=3.14159265358979323846l;

// 点与向量
template<typename T> struct point
{
    T x,y;

    bool operator==(const point &a) const {return (abs(x-a.x)
        <=eps && abs(y-a.y)<=eps);}
    bool operator<(const point &a) const {if (abs(x-a.x)<=eps)
         return y<a.y-eps; return x<a.x-eps;}
    bool operator>(const point &a) const {return !(*this<a ||
        *this==a);}
    point operator+(const point &a) const {return {x+a.x,y+a.y
        };}
    point operator-(const point &a) const {return {x-a.x,y-a.y
        };}
    point operator-() const {return {-x,-y};}
    point operator*(const T k) const {return {k*x,k*y};}
    point operator/(const T k) const {return {x/k,y/k};}
    T operator*(const point &a) const {return x*a.x+y*a.y;}
        // 点积
    T operator^(const point &a) const {return x*a.y-y*a.x;}
        // 叉积，注意优先级
    int toleft(const point &a) const {const auto t=(*this)^a;
        return (t>eps)-(t<-eps);}  // to-left 测试
    T len2() const {return (*this)*(*this);}  // 向量长度的平
        方
    T dis2(const point &a) const {return (a-(*this)).len2();}
         // 两点距离的平方

    // 涉及浮点数
    long double len() const {return sqrtl(len2());}  // 向量长
        度
    long double dis(const point &a) const {return sqrtl(dis2(a
        ));}  // 两点距离
    long double ang(const point &a) const {return acosl(max
        (-1.0l,min(1.0l,((*this)*a)/(len()*a.len()))));}  //
        向量夹角
    point rot(const long double rad) const {return {x*cos(rad)
        -y*sin(rad),x*sin(rad)+y*cos(rad)};}  // 逆时针旋转
        （给定角度）
    point rot(const long double cosr,const long double sinr)
        const {return {x*cosr-y*sinr,x*sinr+y*cosr};}  // 逆
        时针旋转（给定角度的正弦与余弦）
};

using Point=point<point_t>;

// 极角排序
struct argcmp
{
    bool operator()(const Point &a,const Point &b) const
    {
        const auto quad=[](const Point &a)
```

```
        {
            if (a.y<-eps) return 1;
            if (a.y>eps) return 4;
            if (a.x<-eps) return 5;
            if (a.x>eps) return 3;
            return 2;
        };
        const int qa=quad(a),qb=quad(b);
        if (qa!=qb) return qa<qb;
        const auto t=a^b;
        return t>eps;
    }
};

// 直线
template<typename T> struct line
{
    point<T> p,v;  // p 为直线上一点，v 为方向向量

    bool operator==(const line &a) const {return v.toleft(a.v)==0 && v.toleft(p-a.p)==0;}
    int toleft(const point<T> &a) const {return v.toleft(a-p);}  // to-left 测试
    bool operator<(const line &a) const  // 半平面交算法定义的排序
    {
        if (abs(v^a.v)<=eps && v*a.v>=-eps) return toleft(a.p)==-1;
        return argcmp()(v,a.v);
    }

    // 涉及浮点数
    point<T> inter(const line &a) const {return p+v*((a.v^(p-a.p))/(v^a.v));}  // 直线交点
    long double dis(const point<T> &a) const {return abs(v^(a-p))/v.len();}  // 点到直线距离
    point<T> proj(const point<T> &a) const {return p+v*((v*(a-p))/(v*v));}  // 点在直线上的投影
};

using Line=line<point_t>;

//线段
template<typename T> struct segment
{
    point<T> a,b;

    bool operator<(const segment &s) const {return make_pair(a,b)<make_pair(s.a,s.b);}

    // 判定性函数建议在整数域使用

    // 判断点是否在线段上
    // -1 点在线段端点 | 0 点不在线段上 | 1 点严格在线段上
    int is_on(const point<T> &p) const
    {
        if (p==a || p==b) return -1;
        return (p-a).toleft(p-b)==0 && (p-a)*(p-b)<-eps;
    }

    // 判断线段直线是否相交
    // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
    int is_inter(const line<T> &l) const
    {
        if (l.toleft(a)==0 || l.toleft(b)==0) return -1;
        return l.toleft(a)!=l.toleft(b);
    }

    // 判断两线段是否相交
    // -1 在某一线段端点处相交 | 0 两线段不相交 | 1 两线段严格相交
    int is_inter(const segment<T> &s) const
    {
        if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
        const line<T> l{a,b-a},ls{s.a,s.b-s.a};
        return l.toleft(s.a)*l.toleft(s.b)==-1 && ls.toleft(a)*ls.toleft(b)==-1;
    }

    // 点到线段距离
    long double dis(const point<T> &p) const
    {
        if ((p-a)*(b-a)<-eps || (p-b)*(a-b)<-eps) return min(p.dis(a),p.dis(b));
        const line<T> l{a,b-a};
        return l.dis(p);
    }

    // 两线段间距离
    long double dis(const segment<T> &s) const
    {
        if (is_inter(s)) return 0;
        return min({dis(s.a),dis(s.b),s.dis(a),s.dis(b)});
    }
};

using Segment=segment<point_t>;

// 多边形
template<typename T> struct polygon
{
    vector<point<T>> p;  // 以逆时针顺序存储

    size_t nxt(const size_t i) const {return i==p.size()-1?0:i+1;}
    size_t pre(const size_t i) const {return i==0?p.size()-1:i-1;}

    // 回转数
    // 返回值第一项表示点是否在多边形边上
    // 对于狭义多边形，回转数为 0 表示点在多边形外，否则点在多边形内
    pair<bool,int> winding(const point<T> &a) const
    {
        int cnt=0;
        for (size_t i=0;i<p.size();i++)
        {
            const point<T> u=p[i],v=p[nxt(i)];
            if (abs((a-u)^(a-v))<=eps && (a-u)*(a-v)<=eps) return {true,0};
            if (abs(u.y-v.y)<=eps) continue;
            const Line uv={u,v-u};
            if (u.y<v.y-eps && uv.toleft(a)<=0) continue;
            if (u.y>v.y+eps && uv.toleft(a)>=0) continue;
            if (u.y<a.y-eps && v.y>=a.y-eps) cnt++;
            if (u.y>=a.y-eps && v.y<a.y-eps) cnt--;
        }
        return {false,cnt};
    }

    // 多边形面积的两倍
    // 可用于判断点的存储顺序是顺时针或逆时针
    T area() const
    {
        T sum=0;
        for (size_t i=0;i<p.size();i++) sum+=p[i]^p[nxt(i)];
        return sum;
    }

    // 多边形的周长
    long double circ() const
    {
        long double sum=0;
        for (size_t i=0;i<p.size();i++) sum+=p[i].dis(p[nxt(i)]);
        return sum;
    }
};

using Polygon=polygon<point_t>;

//凸多边形
template<typename T> struct convex: polygon<T>
{
    // 闵可夫斯基和
    convex operator+(const convex &c) const
    {
        const auto &p=this->p;
        vector<Segment> e1(p.size()),e2(c.p.size()),edge(p.size()+c.p.size());
        vector<point<T>> res; res.reserve(p.size()+c.p.size());
        const auto cmp=[](const Segment &u,const Segment &v) {return argcmp()(u.b-u.a,v.b-v.a);};
        for (size_t i=0;i<p.size();i++) e1[i]={p[i],p[this->nxt(i)]};
```

```
        for (size_t i=0;i<c.p.size();i++) e2[i]={c.p[i],c.p[c.nxt(i)]};
        rotate(e1.begin(),min_element(e1.begin(),e1.end(),cmp),e1.end());
        rotate(e2.begin(),min_element(e2.begin(),e2.end(),cmp),e2.end());
        merge(e1.begin(),e1.end(),e2.begin(),e2.end(),edge.begin(),cmp);
        const auto check=[](const vector<point<T>> &res,const point<T> &u)
        {
            const auto back1=res.back(),back2=*prev(res.end(),2);
            return (back1-back2).toleft(u-back1)==0 && (back1-back2)*(u-back1)>=-eps;
        };
        auto u=e1[0].a+e2[0].a;
        for (const auto &v:edge)
        {
            while (res.size()>1 && check(res,u)) res.pop_back();
            res.push_back(u);
            u=u+v.b-v.a;
        }
        if (res.size()>1 && check(res,res[0])) res.pop_back();
        return {res};
    }

    // 旋转卡壳
    // func 为更新答案的函数，可以根据题目调整位置
    template<typename F> void rotcaliper(const F &func) const
    {
        const auto &p=this->p;
        const auto area=[](const point<T> &u,const point<T> &v,const point<T> &w){return (w-u)^(w-v);};
        for (size_t i=0,j=1;i<p.size();i++)
        {
            const auto nxti=this->nxt(i);
            func(p[i],p[nxti],p[j]);
            while (area(p[this->nxt(j)],p[i],p[nxti])>=area(p[j],p[i],p[nxti]))
            {
                j=this->nxt(j);
                func(p[i],p[nxti],p[j]);
            }
        }
    }

    // 凸多边形的直径的平方
    T diameter2() const
    {
        const auto &p=this->p;
        if (p.size()==1) return 0;
        if (p.size()==2) return p[0].dis2(p[1]);
        T ans=0;
        auto func=[&](const point<T> &u,const point<T> &v,const point<T> &w){ans=max({ans,w.dis2(u),w.dis2(v)});};
        rotcaliper(func);
        return ans;
    }

    // 判断点是否在凸多边形内
    // 复杂度 O(logn)
    // -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
    int is_in(const point<T> &a) const
    {
        const auto &p=this->p;
        if (p.size()==1) return a==p[0]?-1:0;
        if (p.size()==2) return segment<T>{p[0],p[1]}.is_on(a)?-1:0;
        if (a==p[0]) return -1;
        if ((p[1]-p[0]).toleft(a-p[0])==-1 || (p.back()-p[0]).toleft(a-p[0])==1) return 0;
        const auto cmp=[&](const Point &u,const Point &v){
            return (u-p[0]).toleft(v-p[0])==1;};
        const size_t i=lower_bound(p.begin()+1,p.end(),a,cmp)-p.begin();
        if (i==1) return segment<T>{p[0],p[i]}.is_on(a)?-1:0;
        if (i==p.size()-1 && segment<T>{p[0],p[i]}.is_on(a)) return -1;
        if (segment<T>{p[i-1],p[i]}.is_on(a)) return -1;
        return (p[i]-p[i-1]).toleft(a-p[i-1])>0;
    }

    // 凸多边形关于某一方向的极点
    // 复杂度 O(logn)
    // 参考资料：https://codeforces.com/blog/entry/48868
    template<typename F> size_t extreme(const F &dir) const
    {
        const auto &p=this->p;
        const auto check=[&](const size_t i){return dir(p[i]).toleft(p[this->nxt(i)]-p[i])>=0;};
        const auto dir0=dir(p[0]); const auto check0=check(0);
        if (!check0 && check(p.size()-1)) return 0;
        const auto cmp=[&](const Point &v)
        {
            const size_t vi=&v-p.data();
            if (vi==0) return 1;
            const auto checkv=check(vi);
            const auto t=dir0.toleft(v-p[0]);
            if (vi==1 && checkv==check0 && t==0) return 1;
            return checkv^(checkv==check0 && t<=0);
        };
        return partition_point(p.begin(),p.end(),cmp)-p.begin();
    }

    // 过凸多边形外一点求凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    // 必须保证点在多边形外
    pair<size_t,size_t> tangent(const point<T> &a) const
    {
        const size_t i=extreme([&](const point<T> &u){return u-a;});
        const size_t j=extreme([&](const point<T> &u){return a-u;});
        return {i,j};
    }

    // 求平行于给定直线的凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    pair<size_t,size_t> tangent(const line<T> &a) const
    {
        const size_t i=extreme([&](...){return a.v;});
        const size_t j=extreme([&](...){return -a.v;});
        return {i,j};
    }
};

using Convex=convex<point_t>;

// 圆
struct Circle
{
    Point c;
    long double r;

    bool operator==(const Circle &a) const {return c==a.c && abs(r-a.r)<=eps;}
    long double circ() const {return 2*PI*r;}  // 周长
    long double area() const {return PI*r*r;}  // 面积

    // 点与圆的关系
    // -1 圆上 | 0 圆外 | 1 圆内
    int is_in(const Point &p) const {const long double d=p.dis(c); return abs(d-r)<=eps?-1:d<r-eps;}

    // 直线与圆关系
    // 0 相离 | 1 相切 | 2 相交
    int relation(const Line &l) const
    {
        const long double d=l.dis(c);
        if (d>r+eps) return 0;
        if (abs(d-r)<=eps) return 1;
        return 2;
    }

    // 圆与圆关系
    // -1 相同 | 0 相离 | 1 外切 | 2 相交 | 3 内切 | 4 内含
    int relation(const Circle &a) const
    {
        if (*this==a) return -1;
        const long double d=c.dis(a.c);
        if (d>r+a.r+eps) return 0;
        if (abs(d-r-a.r)<=eps) return 1;
        if (abs(d-abs(r-a.r))<=eps) return 3;
        if (d<abs(r-a.r)-eps) return 4;
```

```
        return 2;
    }

    // 直线与圆的交点
    vector<Point> inter(const Line &l) const
    {
        const long double d=l.dis(c);
        const Point p=l.proj(c);
        const int t=relation(l);
        if (t==0) return vector<Point>();
        if (t==1) return vector<Point>{p};
        const long double k=sqrt(r*r-d*d);
        return vector<Point>{p-(l.v/l.v.len())*k,p+(l.v/l.v.len())*k};
    }

    // 圆与圆交点
    vector<Point> inter(const Circle &a) const
    {
        const long double d=c.dis(a.c);
        const int t=relation(a);
        if (t==-1 || t==0 || t==4) return vector<Point>();
        Point e=a.c-c; e=e/e.len()*r;
        if (t==1 || t==3)
        {
            if (r*r+d*d-a.r*a.r>=-eps) return vector<Point>{c+e};
            return vector<Point>{c-e};
        }
        const long double costh=(r*r+d*d-a.r*a.r)/(2*r*d),sinth=sqrt(1-costh*costh);
        return vector<Point>{c+e.rot(costh,-sinth),c+e.rot(costh,sinth)};
    }

    // 圆与圆交面积
    long double inter_area(const Circle &a) const
    {
        const long double d=c.dis(a.c);
        const int t=relation(a);
        if (t==-1) return area();
        if (t<2) return 0;
        if (t>2) return min(area(),a.area());
        const long double costh1=(r*r+d*d-a.r*a.r)/(2*r*d),costh2=(a.r*a.r+d*d-r*r)/(2*a.r*d);
        const long double sinth1=sqrt(1-costh1*costh1),sinth2=sqrt(1-costh2*costh2);
        const long double th1=acos(costh1),th2=acos(costh2);
        return r*r*(th1-costh1*sinth1)+a.r*a.r*(th2-costh2*sinth2);
    }

    // 过圆外一点圆的切线
    vector<Line> tangent(const Point &a) const
    {
        const int t=is_in(a);
        if (t==1) return vector<Line>();
        if (t==-1)
        {
            const Point v={-(a-c).y,(a-c).x};
            return vector<Line>{{a,v}};
        }
        Point e=a-c; e=e/e.len()*r;
        const long double costh=r/c.dis(a),sinth=sqrt(1-costh*costh);
        const Point t1=c+e.rot(costh,-sinth),t2=c+e.rot(costh,sinth);
        return vector<Line>{{a,t1-a},{a,t2-a}};
    }

    // 两圆的公切线
    vector<Line> tangent(const Circle &a) const
    {
        const int t=relation(a);
        vector<Line> lines;
        if (t==-1 || t==4) return lines;
        if (t==1 || t==3)
        {
            const Point p=inter(a)[0],v={-(a.c-c).y,(a.c-c).x};
            lines.push_back({p,v});
        }
        const long double d=c.dis(a.c);
        const Point e=(a.c-c)/(a.c-c).len();
        if (t<=2)
        {
            const long double costh=(r-a.r)/d,sinth=sqrt(1-costh*costh);
            const Point d1=e.rot(costh,-sinth),d2=e.rot(costh,sinth);
            const Point u1=c+d1*r,u2=c+d2*r,v1=a.c+d1*a.r,v2=a.c+d2*a.r;
            lines.push_back({u1,v1-u1}); lines.push_back({u2,v2-u2});
        }
        if (t==0)
        {
            const long double costh=(r+a.r)/d,sinth=sqrt(1-costh*costh);
            const Point d1=e.rot(costh,-sinth),d2=e.rot(costh,sinth);
            const Point u1=c+d1*r,u2=c+d2*r,v1=a.c-d1*a.r,v2=a.c-d2*a.r;
            lines.push_back({u1,v1-u1}); lines.push_back({u2,v2-u2});
        }
        return lines;
    }

    // 圆的反演
    tuple<int,Circle,Line> inverse(const Line &l) const
    {
        const Circle null_c={{0.0,0.0},0.0};
        const Line null_l={{0.0,0.0},{0.0,0.0}};
        if (l.toleft(c)==0) return {2,null_c,l};
        const Point v=l.toleft(c)==1?Point{l.v.y,-l.v.x}:Point{-l.v.y,l.v.x};
        const long double d=r*r/l.dis(c);
        const Point p=c+v/v.len()*d;
        return {1,{(c+p)/2,d/2},null_l};
    }

    tuple<int,Circle,Line> inverse(const Circle &a) const
    {
        const Circle null_c={{0.0,0.0},0.0};
        const Line null_l={{0.0,0.0},{0.0,0.0}};
        const Point v=a.c-c;
        if (a.is_in(c)==-1)
        {
            const long double d=r*r/(a.r+a.r);
            const Point p=c+v/v.len()*d;
            return {2,null_c,{p,{-v.y,v.x}}};
        }
        if (c==a.c) return {1,{c,r*r/a.r},null_l};
        const long double d1=r*r/(c.dis(a.c)-a.r),d2=r*r/(c.dis(a.c)+a.r);
        const Point p=c+v/v.len()*d1,q=c+v/v.len()*d2;
        return {1,{(p+q)/2,p.dis(q)/2},null_l};
    }
};

// 圆与多边形面积交
long double area_inter(const Circle &circ,const Polygon &poly)
{
    const auto cal=[](const Circle &circ,const Point &a,const Point &b)
    {
        if ((a-circ.c).toleft(b-circ.c)==0) return 0.0l;
        const auto ina=circ.is_in(a),inb=circ.is_in(b);
        const Line ab={a,b-a};
        if (ina && inb) return ((a-circ.c)^(b-circ.c))/2;
        if (ina && !inb)
        {
            const auto t=circ.inter(ab);
            const Point p=t.size()==1?t[0]:t[1];
            const long double ans=((a-circ.c)^(p-circ.c))/2;
            const long double th=(p-circ.c).ang(b-circ.c);
            const long double d=circ.r*circ.r*th/2;
            if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
            return ans-d;
        }
        if (!ina && inb)
        {
            const Point p=circ.inter(ab)[0];
            const long double ans=((p-circ.c)^(b-circ.c))/2;
            const long double th=(a-circ.c).ang(p-circ.c);
            const long double d=circ.r*circ.r*th/2;
            if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
```

```
            return ans-d;
        }
        const auto p=circ.inter(ab);
        if (p.size()==2 && Segment{a,b}.dis(circ.c)<=circ.r+eps)
        {
            const long double ans=((p[0]-circ.c)^(p[1]-circ.c))/2;
            const long double th1=(a-circ.c).ang(p[0]-circ.c),th2=(b-circ.c).ang(p[1]-circ.c);
            const long double d1=circ.r*circ.r*th1/2,d2=circ.r*circ.r*th2/2;
            if ((a-circ.c).toleft(b-circ.c)==1) return ans+d1+d2;
            return ans-d1-d2;
        }
        const long double th=(a-circ.c).ang(b-circ.c);
        if ((a-circ.c).toleft(b-circ.c)==1) return circ.r*circ.r*th/2;
        return -circ.r*circ.r*th/2;
    };

    long double ans=0;
    for (size_t i=0;i<poly.p.size();i++)
    {
        const Point a=poly.p[i],b=poly.p[poly.nxt(i)];
        ans+=cal(circ,a,b);
    }
    return ans;
}

// 点集的凸包
// Andrew 算法，复杂度 O(nlogn)
Convex convexhull(vector<Point> p)
{
    vector<Point> st;
    if (p.empty()) return Convex{st};
    sort(p.begin(),p.end());
    const auto check=[](const vector<Point> &st,const Point &u)
    {
        const auto back1=st.back(),back2=*prev(st.end(),2);
        return (back1-back2).toleft(u-back1)<=0;
    };
    for (const Point &u:p)
    {
        while (st.size()>1 && check(st,u)) st.pop_back();
        st.push_back(u);
    }
    size_t k=st.size();
    p.pop_back(); reverse(p.begin(),p.end());
    for (const Point &u:p)
    {
        while (st.size()>k && check(st,u)) st.pop_back();
        st.push_back(u);
    }
    st.pop_back();
    return Convex{st};
}

// 半平面交
// 排序增量法，复杂度 O(nlogn)
// 输入与返回值都是用直线表示的半平面集合
vector<Line> halfinter(vector<Line> l, const point_t lim=1e9)
{
    const auto check=[](const Line &a,const Line &b,const Line &c){return a.toleft(b.inter(c))<0;};
    l.push_back({{-lim,0},{0,-1}}); l.push_back({{0,-lim},{1,0}});
    l.push_back({{lim,0},{0,1}}); l.push_back({{0,lim},{-1,0}});
    sort(l.begin(),l.end());
    deque<Line> q;
    for (size_t i=0;i<l.size();i++)
    {
        if (i>0 && l[i-1].v.toleft(l[i].v)==0 && l[i-1].v*l[i].v>eps) continue;
        while (q.size()>1 && check(l[i],q.back(),q[q.size()-2])) q.pop_back();
        while (q.size()>1 && check(l[i],q[0],q[1])) q.pop_front();
        if (!q.empty() && q.back().v.toleft(l[i].v)<=0) return vector<Line>();
        q.push_back(l[i]);
    }
    while (q.size()>1 && check(q[0],q.back(),q[q.size()-2])) q.pop_back();
    while (q.size()>1 && check(q.back(),q[0],q[1])) q.pop_front();
    return vector<Line>(q.begin(),q.end());
}

// 点集形成的最小最大三角形
// 极角序扫描线，复杂度 O(n^2logn)
// 最大三角形问题可以使用凸包与旋转卡壳做到 O(n^2)
pair<point_t,point_t> minmax_triangle(const vector<Point> &vec)
{
    if (vec.size()<=2) return {0,0};
    vector<pair<int,int>> evt;
    evt.reserve(vec.size()*vec.size());
    point_t maxans=0,minans=numeric_limits<point_t>::max();
    for (size_t i=0;i<vec.size();i++)
    {
        for (size_t j=0;j<vec.size();j++)
        {
            if (i==j) continue;
            if (vec[i]==vec[j]) minans=0;
            else evt.push_back({i,j});
        }
    }
    sort(evt.begin(),evt.end(),[&](const pair<int,int> &u,const pair<int,int> &v)
    {
        const Point du=vec[u.second]-vec[u.first],dv=vec[v.second]-vec[v.first];
        return argcmp()({du.y,-du.x},{dv.y,-dv.x});
    });
    vector<size_t> vx(vec.size()),pos(vec.size());
    for (size_t i=0;i<vec.size();i++) vx[i]=i;
    sort(vx.begin(),vx.end(),[&](int x,int y){return vec[x]<vec[y];});
    for (size_t i=0;i<vx.size();i++) pos[vx[i]]=i;
    for (auto [u,v]:evt)
    {
        const size_t i=pos[u],j=pos[v];
        const size_t l=min(i,j),r=max(i,j);
        const Point vecu=vec[u],vecv=vec[v];
        if (l>0) minans=min(minans,abs((vec[vx[l-1]]-vecu)^(vec[vx[l-1]]-vecv)));
        if (r<vx.size()-1) minans=min(minans,abs((vec[vx[r+1]]-vecu)^(vec[vx[r+1]]-vecv)));
        maxans=max({maxans,abs((vec[vx[0]]-vecu)^(vec[vx[0]]-vecv)),abs((vec[vx.back()]-vecu)^(vec[vx.back()]-vecv))});
        if (i<j) swap(vx[i],vx[j]),pos[u]=j,pos[v]=i;
    }
    return {minans,maxans};
}

// 判断多条线段是否有交点
// 扫描线，复杂度 O(nlogn)
bool segs_inter(const vector<Segment> &segs)
{
    if (segs.empty()) return false;
    using seq_t=tuple<point_t,int,Segment>;
    const auto seqcmp=[](const seq_t &u, const seq_t &v)
    {
        const auto [u0,u1,u2]=u;
        const auto [v0,v1,v2]=v;
        if (abs(u0-v0)<=eps) return make_pair(u1,u2)<make_pair(v1,v2);
        return u0<v0-eps;
    };
    vector<seq_t> seq;
    for (auto seg:segs)
    {
        if (seg.a.x>seg.b.x+eps) swap(seg.a,seg.b);
        seq.push_back({seg.a.x,0,seg});
        seq.push_back({seg.b.x,1,seg});
    }
    sort(seq.begin(),seq.end(),seqcmp);
    point_t x_now;
    auto cmp=[&](const Segment &u, const Segment &v)
    {
        if (abs(u.a.x-u.b.x)<=eps || abs(v.a.x-v.b.x)<=eps)
            return u.a.y<v.a.y-eps;
        return ((x_now-u.a.x)*(u.b.y-u.a.y)+u.a.y*(u.b.x-u.a.x
```

```
            ))*(v.b.x-v.a.x)<((x_now-v.a.x)*(v.b.y-v.a.y)+v.a
            .y*(v.b.x-v.a.x))*(u.b.x-u.a.x)-eps;
    };
    multiset<Segment,decltype(cmp)> s{cmp};
    for (const auto [x,o,seg]:seq)
    {
        x_now=x;
        const auto it=s.lower_bound(seg);
        if (o==0)
        {
            if (it!=s.end() && seg.is_inter(*it)) return true;
            if (it!=s.begin() && seg.is_inter(*prev(it)))
                return true;
            s.insert(seg);
        }
        else
        {
            if (next(it)!=s.end() && it!=s.begin() && (*prev(
                it)).is_inter(*next(it))) return true;
            s.erase(it);
        }
    }
    return false;
}

// 多边形面积并
// 轮廓积分，复杂度 O(n^2logn)，n为边数
// ans[i] 表示被至少覆盖了 i+1 次的区域的面积
vector<long double> area_union(const vector<Polygon> &polys)
{
    const size_t siz=polys.size();
    vector<vector<pair<Point,Point>>> segs(siz);
    const auto check=[](const Point &u,const Segment &e){
        return !((u<e.a && u<e.b) || (u>e.a && u>e.b));};

    auto cut_edge=[&](const Segment &e,const size_t i)
    {
        const Line le{e.a,e.b-e.a};
        vector<pair<Point,int>> evt;
        evt.push_back({e.a,0}); evt.push_back({e.b,0});
        for (size_t j=0;j<polys.size();j++)
        {
            if (i==j) continue;
            const auto &pj=polys[j];
            for (size_t k=0;k<pj.p.size();k++)
            {
                const Segment s={pj.p[k],pj.p[pj.nxt(k)]};
                if (le.toleft(s.a)==0 && le.toleft(s.b)==0)
                {
                    evt.push_back({s.a,0});
                    evt.push_back({s.b,0});
                }
                else if (s.is_inter(le))
                {
                    const Line ls{s.a,s.b-s.a};
                    const Point u=le.inter(ls);
                    if (le.toleft(s.a)<0 && le.toleft(s.b)>=0)
                        evt.push_back({u,-1});
                    else if (le.toleft(s.a)>=0 && le.toleft(s.
                        b)<0) evt.push_back({u,1});
                }
            }
        }
        sort(evt.begin(),evt.end());
        if (e.a>e.b) reverse(evt.begin(),evt.end());
        int sum=0;
        for (size_t i=0;i<evt.size();i++)
        {
            sum+=evt[i].second;
            const Point u=evt[i].first,v=evt[i+1].first;
            if (!(u==v) && check(u,e) && check(v,e)) segs[sum
                ].push_back({u,v});
            if (v==e.b) break;
        }
    };

    for (size_t i=0;i<polys.size();i++)
    {
        const auto &pi=polys[i];
        for (size_t k=0;k<pi.p.size();k++)
        {
            const Segment ei={pi.p[k],pi.p[pi.nxt(k)]};
            cut_edge(ei,i);
        }
    }
    vector<long double> ans(siz);
    for (size_t i=0;i<siz;i++)
    {
        long double sum=0;
        sort(segs[i].begin(),segs[i].end());
        int cnt=0;
        for (size_t j=0;j<segs[i].size();j++)
        {
            if (j>0 && segs[i][j]==segs[i][j-1]) segs[i+(++cnt
                )].push_back(segs[i][j]);
            else cnt=0,sum+=segs[i][j].first^segs[i][j].second
                ;
        }
        ans[i]=sum/2;
    }
    return ans;
}

// 圆面积并
// 轮廓积分，复杂度 O(n^2logn)
// ans[i] 表示被至少覆盖了 i+1 次的区域的面积
vector<long double> area_union(const vector<Circle> &circs)
{
    const size_t siz=circs.size();
    using arc_t=tuple<Point,long double,long double,long
        double>;
    vector<vector<arc_t>> arcs(siz);
    const auto eq=[](const arc_t &u,const arc_t &v)
    {
        const auto [u1,u2,u3,u4]=u;
        const auto [v1,v2,v3,v4]=v;
        return u1==v1 && abs(u2-v2)<=eps && abs(u3-v3)<=eps &&
            abs(u4-v4)<=eps;
    };

    auto cut_circ=[&](const Circle &ci,const size_t i)
    {
        vector<pair<long double,int>> evt;
        evt.push_back({-PI,0}); evt.push_back({PI,0});
        int init=0;
        for (size_t j=0;j<circs.size();j++)
        {
            if (i==j) continue;
            const Circle &cj=circs[j];
            if (ci.r<cj.r-eps && ci.relation(cj)>=3) init++;
            const auto inters=ci.inter(cj);
            if (inters.size()==1) evt.push_back({atan2l((
                inters[0]-ci.c).y,(inters[0]-ci.c).x),0});
            if (inters.size()==2)
            {
                const Point dl=inters[0]-ci.c,dr=inters[1]-ci.
                    c;
                long double argl=atan2l(dl.y,dl.x),argr=atan2l
                    (dr.y,dr.x);
                if (abs(argl+PI)<=eps) argl=PI;
                if (abs(argr+PI)<=eps) argr=PI;
                if (argl>argr+eps)
                {
                    evt.push_back({argl,1}); evt.push_back({PI
                        ,-1});
                    evt.push_back({-PI,1}); evt.push_back({
                        argr,-1});
                }
                else
                {
                    evt.push_back({argl,1});
                    evt.push_back({argr,-1});
                }
            }
        }
        sort(evt.begin(),evt.end());
        int sum=init;
        for (size_t i=0;i<evt.size();i++)
        {
            sum+=evt[i].second;
            if (abs(evt[i].first-evt[i+1].first)>eps) arcs[sum
                ].push_back({ci.c,ci.r,evt[i].first,evt[i+1].
                first});
            if (abs(evt[i+1].first-PI)<=eps) break;
        }
    };

    const auto oint=[](const arc_t &arc)
```

```
    {
        const auto [cc,cr,l,r]=arc;
        if (abs(r-l-PI-PI)<=eps) return 2.0l*PI*cr*cr;
        return cr*cr*(r-l)+cc.x*cr*(sin(r)-sin(l))-cc.y*cr*(
             cos(r)-cos(l));
    };

    for (size_t i=0;i<circs.size();i++)
    {
        const auto &ci=circs[i];
        cut_circ(ci,i);
    }
    vector<long double> ans(siz);
    for (size_t i=0;i<siz;i++)
    {
        long double sum=0;
        sort(arcs[i].begin(),arcs[i].end());
        int cnt=0;
        for (size_t j=0;j<arcs[i].size();j++)
        {
            if (j>0 && eq(arcs[i][j],arcs[i][j-1])) arcs[i+(++
                 cnt)].push_back(arcs[i][j]);
            else cnt=0,sum+=oint(arcs[i][j]);
        }
        ans[i]=sum/2;
    }
    return ans;
}
```

8 其它

8.1 二维差分与前缀和

```
void cf(int x1,int y1,int x2,int y2){
    a[x1][y1]++;
    a[x2+1][y2+1]++;
    a[x2+1][y1]--;
    a[x1][y2+1]--;
}

只对四边形四条边差分
cf(i,j,i+l,j);
cf(i,j+1,i,j+l);
cf(i+l,j+1,i+l,j+l);
cf(i+1,j+l,i+l-1,j+l);

前缀和
a[i][j] += a[i-1][j] + a[i][j-1] - a[i-1][j-1];
```

8.2 CDQ 分治

```
//3810

#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+10;
const int maxk=2e5+10;
struct node{
    int idx,a,b,c;
    bool operator==(const node& other) const {
        return a == other.a &&b == other.b &&c == other.c;
    }
}t[maxn];
int f[maxn],n,ans[maxk],k;
struct BIT{
    int n;
    vector<int>t;
    BIT(int n):n(n),t(n+1){};
    int lowbit(int x) {return x&-x;}
    void modify(int x,int d){
        for(;x<=n;x+=lowbit(x)) t[x]+=d;
    }
    int query(int x){
        int res=0;
        for(;x;x-=lowbit(x)) res+=t[x];
        return res;
    }
};
BIT T(maxk);
void prepare(){
    sort(t+1,t+n+1,[](node a,node b){
        if(a.a!=b.a) return a.a<b.a;
        if(a.b!=b.b) return a.b<b.b;
        return a.c<b.c;
    });
    for(int l=1,r=1;l<=n;l=++r){
        while(r+1<=n&&t[l]==t[r+1]) r++;
        for(int i=l;i<=r;i++){
            f[t[i].idx]=r-i;
        }
    }
}
void merge(int l,int mid,int r){
    int p,q;
    for(p=l-1,q=mid+1;q<=r;q++){
        while(p+1<=mid&&t[p+1].b<=t[q].b){
            p++;
            T.modify(t[p].c,1);
        }
        f[t[q].idx]+=T.query(t[q].c);
    }
    for(int i=l;i<=p;i++){
        T.modify(t[i].c,-1);
    }
    sort(t+l,t+r+1,[](node a,node b){
        return a.b<b.b;
    });
}
void cdq(int l,int r){
    if(l==r) return;
    int mid=(l+r)>>1;
    cdq(l,mid);
    cdq(mid+1,r);
    merge(l,mid,r);
}

int main(){
    cin>>n>>k;
    for(int i=1;i<=n;i++){
        cin>>t[i].a>>t[i].b>>t[i].c;
        t[i].idx=i;
    }
    prepare();
    cdq(1,n);
    for(int i=1;i<=n;i++){
        ans[f[i]]++;
    }
    for(int i=0;i<n;i++) cout<<ans[i]<<endl;
}
```

8.3 矩阵加速

```
//1939
// #include <iostream>
// #include <cstdio>
// #include <cstring>

// using namespace std;

// typedef long long LL;
// const int Mod = 1e9+7;
// int T, n;
// struct mat{
//  LL m[5][5];
// }Ans, base;
// inline void init() {
//  memset(Ans.m, 0, sizeof(Ans.m));
//  for(int i=1; i<=3; i++) Ans.m[i][i] = 1;
//  memset(base.m, 0, sizeof(base.m));
//  base.m[1][1] = base.m[1][3] = base.m[2][1] = base.m[3][2]
     = 1;
// }
// inline mat mul(mat a, mat b) {
//  mat res;
//  memset(res.m, 0, sizeof(res.m));
//  for(int i=1; i<=3; i++) {
//    for(int j=1; j<=3; j++) {
//      for(int k=1; k<=3; k++) {
//        res.m[i][j] += (a.m[i][k] % Mod) * (b.m[k][j] % Mod)
     ;
//        res.m[i][j] %= Mod;
//      }
//    }
//  }
//  return res;
```

```
// }
// inline void Qmat_pow(int p) {
//  while (p) {
//    if(p & 1) Ans = mul(Ans, base);
//    base = mul(base, base);
//    p >>= 1;
//  }
// }

// int main() {
//  scanf("%d", &T);
//  while (T--) {
//    scanf("%d", &n);
//    if(n <= 3) {
//      printf("1\n");
//      continue;
//    }
//    init();
//    Qmat_pow(n);
//    printf("%lld\n", Ans.m[2][1]);
//  }
// }
/*
f[i-1]       f[i]
f[i-2] ----> f[i-1]
f[i-3]       f[i-2]
f[i] = f[i-1] * 1 + f[i-2] * 0 + f[i-3] * 1
f[i-1] = f[i-1] * 1 + f[i-2] * 0 + f[i-3] * 0
f[i-2] = f[i-1] * 0 + f[i-2] * 1 + f[i-3] * 0
so
1 0 1
1 0 0
0 1 0
*/
//P6569
#include<iostream>
#include<cstdio>
#include<cstring>
#include<algorithm>
using namespace std;
struct matrix{//用结构体存矩阵敲好使哎~
  long long qwq[105][105];
    int x,y;//x,y就是矩阵的行数和列数
  matrix(){x=0,y=0;memset(qwq,0,sizeof(qwq));};//构造函数，刚听说这玩意，用着玩玩
};
matrix operator * (const matrix &a,const matrix &b){//重载“异或版矩阵乘法”的运算符
  matrix c;
  c.x=a.x,c.y=b.y;
  for(int i=1;i<=a.x;i++){
    for(int j=1;j<=b.y;j++){
      for(int k=1;k<=a.y;k++)
      c.qwq[i][j]^=a.qwq[i][k]*b.qwq[k][j];//和矩阵乘法别无二致，只不过就是改成异或
    }
  }
  return c;
}
matrix wyx[40];//存储e矩阵的2^i次方，取这个变量名是因为想借助神仙的力量AC本题
long long f[105];//每个城市的初始魔法值（提醒：十年OI一场空……）
int main(){
  int n,m,q;
  scanf("%d%d%d",&n,&m,&q);
  for(int i=1;i<=n;i++)
  scanf("%lld",&f[i]);
  wyx[0].x=n,wyx[0].y=n;//初始化一开始的邻接矩阵（也就是2的0次方）的大小
  for(int i=1;i<=m;i++){
    int a,b;
    scanf("%d%d",&a,&b);
    wyx[0].qwq[a][b]=wyx[0].qwq[b][a]=1;//给邻接矩阵连上边
  }
  for(int i=1;i<32;i++)wyx[i]=wyx[i-1]*wyx[i-1];//利用快速幂的思想，处理出邻接矩阵2的i次方
  while(q--){
    long long x;
    scanf("%lld",&x);
        matrix ans;
    for(int i=1;i<=n;i++)ans.qwq[1][i]=f[i];//一开始就是初始f值
    ans.x=1,ans.y=n;
    for(int i=0;i<32;i++){//开始进行二进制拆分
      if((x>>i)&1)//如果这一位是1
      ans=ans*wyx[i];//就乘上对应的2的i次方
    }
    printf("%lld\n",ans.qwq[1][1]);
  }
  return 0;
}
```

8.4 cdq 斜率优化

```
#include<bits/stdc++.h>
using namespace std;

using ll=long long;
using i128=__int128_t;

const int maxn=5e4+10;
const i128 inf=((i128)1<<120);

int n;
ll L;
ll c[maxn],s[maxn];
i128 f[maxn];

struct node{
    ll k;
    i128 b;
};

vector<node> a,hull;

bool cmp(node x,node y){
    if(x.k!=y.k)
        return x.k<y.k;
    return x.b<y.b;
}

i128 get(node x,ll p){
    return (i128)x.k*p+x.b;
}

bool check(node a,node b,node c){
    return (a.b-b.b)*(c.k-b.k)
         >=(b.b-c.b)*(b.k-a.k);
}

void build(int l,int r){
    a.clear();
    hull.clear();

    for(int i=l;i<=r;i++){
        node x;
        x.k=-2*s[i];
        x.b=f[i]+(i128)s[i]*s[i];
        a.push_back(x);
    }

    sort(a.begin(),a.end(),cmp);

    for(auto x:a){
        if(!hull.empty()&&hull.back().k==x.k)
            continue;

        while(hull.size()>=2&&
              check(hull[hull.size()-2],hull.back(),x)){
            hull.pop_back();
        }

        hull.push_back(x);
    }
}

i128 query(ll x){
    int l=0,r=(int)hull.size()-1;

    while(l<r){
        int mid=(l+r)>>1;

        if(get(hull[mid],x)>=get(hull[mid+1],x))
            l=mid+1;
        else
            r=mid;
```

```
    }

    return get(hull[l],x);
}

void CDQ(int l,int r){
    if(l==r)
        return;

    int mid=(l+r)>>1;

    CDQ(l,mid);

    build(l,mid);

    for(int i=mid+1;i<=r;i++){
        ll x=s[i]-L;

        i128 res=(i128)x*x+query(x);

        f[i]=min(f[i],res);
    }

    CDQ(mid+1,r);
}

void print(i128 x){
    if(x==0){
        cout<<0;
        return;
    }

    if(x<0){
        cout<<'-';
        x=-x;
    }

    string s;

    while(x){
        s.push_back('0'+x%10);
        x/=10;
    }

    reverse(s.begin(),s.end());
    cout<<s;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin>>n>>L;

    s[0]=0;

    for(int i=1;i<=n;i++){
        cin>>c[i];
        s[i]=s[i-1]+c[i]+1;
    }

    for(int i=0;i<=n;i++)
        f[i]=inf;

    f[0]=0;

    CDQ(0,n);

    print(f[n]);
    cout<<'\n';

    return 0;
}
```

8.5 并查集

```
//推导部分和
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll N=1e5+10;
struct tree{
  ll n,w,to;
}e[N<<1];
ll head[N<<1],tot,fa[N],vis[N],sum[N];
void add(int u,int v,ll w){
  e[tot].to=v;
  e[tot].n=head[u];
  e[tot].w=w;
  head[u]=tot++;
}
int find(int i){
  if(fa[i]!=i) return fa[i]=find(fa[i]);
  return i;
}
void unite(int x,int y){
  x=find(x);
  y=find(y);
  if(x!=y) fa[x]=y;
}
void bfs(int x){
  vis[x]=1;
  queue<int>q; q.push(x);
  sum[x]=0;
  while(!q.empty()){
    int u=q.front();
    q.pop();
    for(int i=head[u];i!=-1;i=e[i].n){
      int v=e[i].to;
      if(vis[v]) continue;
      vis[v]=1;
      sum[v]=sum[u]+e[i].w;
      q.push(v);
    }
  }
}
int main()
{
  memset(head,-1,sizeof(head));
  int n,m,q;
  cin>>n>>m>>q;
  for(int i=1;i<=n;i++) fa[i]=i;
  for(int i=1;i<=m;i++){
    ll l,r,w;
    cin>>l>>r>>w;
    add(l-1,r,w);
    add(r,l-1,-w);
    unite(l-1,r);
  }
  for(int i=0;i<=n;i++){
    if(!vis[i]) bfs(i);
  }
  for(int i=1;i<=m;i++){
    int l,r;
    cin>>l>>r;
    if(find(l-1)!=find(r)) cout<<"UNKNOWN"<<endl;
    else cout<<sum[r]-sum[l-1]<<endl;
  }
  return 0;
}
```

8.6 FFT（快速傅里叶变换）

```
// 当vector用就可以了
#include <bits/stdc++.h>
#define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_
    ; ++i)
#define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_
    ; --i)
using namespace std;
using ll = int64_t;
using db = double;
/*
    ------------------------------------------------------------
    */
struct cp {
    db x, y;
    cp(db real = 0, db imag = 0) : x(real), y(imag) {};
    cp operator+(cp b) const { return {x + b.x, y + b.y}; }
    cp operator-(cp b) const { return {x - b.x, y - b.y}; }
    cp operator*(cp b) const { return {x * b.x - y * b.y, x *
        b.y + y * b.x}; }
};
using vcp = vector<cp>;
using Poly = vector<int>;
namespace FFT {
```

```
const db pi = acos(-1);
vcp Omega(int L) {
    vcp w(L);
    w[1] = 1;
    for (int i = 2; i < L; i <<= 1) {
        auto w0 = w.begin() + i / 2, w1 = w.begin() + i;
        cp wn(cos(pi / i), sin(pi / i));
        for (int j = 0; j < i; j += 2)
            w1[j] = w0[j >> 1], w1[j + 1] = w1[j] * wn;
    }
    return w;
}
auto W = Omega(1 << 21);  // NOLINT
void DIF(cp *a, int n) {
    cp x, y;
    for (int k = n >> 1; k; k >>= 1)
        for (int i = 0; i < n; i += k << 1)
            for (int j = 0; j < k; ++j)
                x = a[i + j], y = a[i + j + k],
                a[i + j + k] = (a[i + j] - y) * W[k + j], a[i
                     + j] = x + y;
}
void IDIT(cp *a, int n) {
    cp x, y;
    for (int k = 1; k < n; k <<= 1)
        for (int i = 0; i < n; i += k << 1)
            for (int j = 0; j < k; ++j)
                x = a[i + j], y = a[i + j + k] * W[k + j], a[i
                      + j + k] = x - y,
                a[i + j] = x + y;
    const db Inv = 1. / n;
    fp(i, 0, n - 1) a[i].x *= Inv, a[i].y *= Inv;
    reverse(a + 1, a + n);
}
}  // namespace FFT

namespace Polynomial {
// basic operator
void DFT(vcp &a) { FFT::DIF(a.data(), a.size()); }
void IDFT(vcp &a) { FFT::IDIT(a.data(), a.size()); }
int norm(int n) { return 1 << (__lg(n - 1) + 1); }

// Poly mul
vcp &dot(vcp &a, vcp &b) {
    fp(i, 0, a.size() - 1) a[i] = a[i] * b[i];
    return a;
}
Poly operator+(Poly a, Poly b) {
    int maxlen = max(a.size(), b.size());
    Poly ans(maxlen + 1);
    a.resize(maxlen + 1), b.resize(maxlen + 1);
    for (int i = 0; i < maxlen; i++) ans[i] = a[i] + b[i];
    return ans;
}
Poly operator*(ll k, Poly a) {
    Poly ans;
    for (auto i : a) ans.push_back(k * i);
    return ans;
}
Poly operator*(Poly a, Poly b) {
    int n = a.size() + b.size() - 1;
    vcp c(norm(n));
    fp(i, 0, a.size() - 1) c[i].x = a[i];
    fp(i, 0, b.size() - 1) c[i].y = b[i];
    DFT(c), dot(c, c), IDIT(c), a.resize(n);
    fp(i, 0, n - 1) a[i] = int(c[i].y * .5 + .5);
    return a;
}
}  // namespace Polynomial
/*
     ------------------------------------------------------------------
     */
using namespace Polynomial;
```

8.7 NTT（数论变换）

```
#include <bits/stdc++.h>
#define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_
    ; ++i)
#define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_
    ; --i)
using namespace std;
const int maxn = 2e5 + 5, P = 998244353;
using ll = int64_t;
#define ADD(a, b) (((a) += (b)) >= P ? (a) -= P : 0)
#define SUB(a, b) (((a) -= (b)) < 0 ? (a) += P : 0)
#define MUL(a, b) (ll(a) * (b) % P)
int POW(ll a, int b = P - 2, ll x = 1) {
    for (; b; b >>= 1, a = a * a % P)
        if (b & 1) x = x * a % P;
    return x;
}

namespace NTT {
const int g = 3;
vector<int> Omega(int L) {
    int wn = POW(g, P / L);
    vector<int> w(L);
    w[L >> 1] = 1;
    fp(i, L / 2 + 1, L - 1) w[i] = MUL(w[i - 1], wn);
    fd(i, L / 2 - 1, 1) w[i] = w[i << 1];
    return w;
}
auto W = Omega(1 << 21);
void DIF(int *a, int n) {
    for (int k = n >> 1; k; k >>= 1) {
        for (int i = 0, y; i < n; i += k << 1)
            fp(j, 0, k - 1) y = a[i + j + k],
                            a[i + j + k] = MUL(a[i + j] - y +
                                 P, W[k + j]),
                            ADD(a[i + j], y);
    }
}
void IDIT(int *a, int n) {
    for (int k = 1; k < n; k <<= 1) {
        for (int i = 0, x, y; i < n; i += k << 1)
            fp(j, 0, k - 1) x = a[i + j], y = MUL(a[i + j + k
                 ], W[k + j]),
                            a[i + j + k] = x - y < 0 ? x - y +
                                  P : x - y,
                            ADD(a[i + j], y);
    }
    int Inv = P - (P - 1) / n;
    fp(i, 0, n - 1) a[i] = MUL(a[i], Inv);
    reverse(a + 1, a + n);
}
}  // namespace NTT
namespace Polynomial {
using Poly = std::vector<int>;
Poly &operator*=(Poly &a, int b) {
    for (auto &x : a) x = MUL(x, b);
    return a;
}
Poly operator*(Poly a, int b) { return a *= b; }
Poly operator*(int a, Poly b) { return b * a; }
void DFT(Poly &a) { NTT::DIF(a.data(), a.size()); }
void IDFT(Poly &a) { NTT::IDIT(a.data(), a.size()); }
int norm(int n) { return 1 << (32 - __builtin_clz(n - 1)); }
void norm(Poly &a) {
    if (!a.empty()) a.resize(norm(a.size()), 0);
}
Poly &dot(Poly &a, Poly &b) {
    fp(i, 0, a.size() - 1) a[i] = MUL(a[i], b[i]);
    return a;
}
Poly operator*(Poly a, Poly b) {
    int n = a.size() + b.size() - 1, L = norm(n);
    if (a.size() <= 8 || b.size() <= 8) {
        Poly c(n);
        fp(i, 0, a.size() - 1) fp(j, 0, b.size() - 1) c[i + j]
              =
            (c[i + j] + (ll)a[i] * b[j]) % P;
        return c;
    }
    a.resize(L), b.resize(L);
    DFT(a), DFT(b), dot(a, b), IDFT(a);
    return a.resize(n), a;
}
}  // namespace Polynomial
using namespace Polynomial;
```

8.8 扫描线算法

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
```

```
ll arr[15010],n,height[15010];

struct node{
    ll l,r,h;
}a[15010];

struct compare{
    bool operator()(pair<ll,ll> a,pair<ll,ll> b){
        if(a.first!=b.first) return a.first<b.first;
        return a.second>b.second;
        //大根堆
    }
};
bool cmp(node x,node y){
    return x.l<y.l;
}
int main(){
    ll l,r,h,m=0;
    while(scanf("%lld %lld %lld",&l,&h,&r)!=EOF){
        a[++m].l=l;
        a[m].r=r-1;
        a[m].h=h;
        arr[++n]=l;
        arr[++n]=r;
        arr[++n]=r-1;
    }
    sort(arr+1,arr+n+1);
    ll len=unique(arr+1,arr+n+1)-arr-1;
    for(int j=1;j<=m;j++){
        a[j].l=lower_bound(arr+1,arr+len+1,a[j].l)-arr;
        a[j].r=lower_bound(arr+1,arr+len+1,a[j].r)-arr;
    }
    sort(a+1,a+m+1,cmp);
   priority_queue<pair<ll,ll>, vector<pair<ll,ll>>, compare>
        heap;
   for(int i=1,j=0;i<=len;i++){
    for(;j<=m&&a[j].l<=i;j++){
        heap.push(make_pair(a[j].h,a[j].r));
    }
    while(!heap.empty()&&heap.top().second<i)
    {
        heap.pop();
    }

   if(!heap.empty()){
    height[i]=heap.top().first;
   }
}
    ll ansn=0;
    pair<ll,ll>ans[15010];
    for(ll i=1,pre=0;i<=len;i++){
        if(pre!=height[i]){
            ans[++ansn]=make_pair(arr[i],height[i]);
        }
        pre=height[i];
    }
    for(int i=1;i<=ansn;i++){
        cout<<ans[i].first<<' '<<ans[i].second<<' ';
    }
}
```

8.9 最长上升子序列

```
#include<cstdio>
#include<algorithm>
const int MAXN=200001;
int a[MAXN];
int d[MAXN];
int main()
{
    int n;
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
        scanf("%d",&a[i]);
    d[1]=a[1];
    int len=1;
    for(int i=2;i<=n;i++)
    {
        if(a[i]>d[len])
            d[++len]=a[i];
        else
        {
            int j=std::lower_bound(d+1,d+len+1,a[i])-d;
            d[j]=a[i];
        }
    }
    printf("%d\n",len);
    return 0;
}
```

8.10 全排列输出

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> arr = {1, 2, 3};  // 示例数组

    // 先排序（必须步骤，确保生成所有排列）
    std::sort(arr.begin(), arr.end());

    // 输出所有排列组合
    do {
        for (int num : arr) {
            std::cout << num << " ";
        }
        std::cout << "\n";
    } while (std::next_permutation(arr.begin(), arr.end()));

    return 0;
}
```

8.11 位运算用法

```
/*为了更好的理解状压dp，首先介绍位运算相关的知识。

'&'符号，x&y，会将两个十进制数在二进制下进行与运算(都1为1，其余为0） 然后返回其十进制下的值。例如3(11)&2(10)=2(10)。
'|'符号，x|y，会将两个十进制数在二进制下进行或运算（都0为0，其余为1） 然后返回其十进制下的值。例如3(11)|2(10)=3(11)。
'^'符号，x^y，会将两个十进制数在二进制下进行异或运算（不同为1，其余 为0）然后返回其十进制下的值。例如3(11)^2(10)=1(01)。
'~'符号，~x，按位取反。例如~101=010。
'<<'符号，左移操作，x<<2，将x在二进制下的每一位向左移动两位，最右边用0填充，x<<2相当于让x乘以4。 '>>'符号，是右移操作，x>>1相当于给x/2，去掉x二进制下的最右一位
1.判断一个数字x二进制下第i位是不是等于1。（最低第1位）

方法：if(((1<<(i-1))&x)>0) 将1左移i-1位，相当于制造了一个只有第i位 上是1，其他位上都是0的二进制数。然后与x做与运算，如果结果>0， 说明x第i位上是1，反之则是0。

2.将一个数字x二进制下第i位更改成1。

方法：x=x|(1<<(i-1)) 证明方法与1类似。

3.将一个数字x二进制下第i位更改成0。

方法：x=x&~(1<<(i-1))

4.把一个数字二进制下最靠右的第一个1去掉。

方法：x=x&(x-1)
__builtin_popcount() 二进制数中一的个数
*/
#include<iostream>
using namespace std;
struct state {
    int st[1 << 12+1];
    int num;
} a[15];
int m, n;
int f[15][1 << 12+1];
const int MOD = 100000000;
void getstate(int i, int t) {
    a[i].num = 0;
    for (int j = 0; j < (1 << n); j++) {
        if ((j & (j << 1)) || (j & t))
            continue;
        a[i].st[++a[i].num] = j;
    }
}
void dp() {
```

```
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= a[i].num; j++)
            f[i][j] = 0;
    for (int j = 1; j <= a[1].num; j++)
        f[1][j] = 1;
    for (int i = 2; i <= m; i++) {
        for (int j = 1; j <= a[i].num; j++) {
            int s1 = a[i].st[j];
            for (int k = 1; k <= a[i-1].num; k++) {
                int s2 = a[i-1].st[k];
                if (s1 & s2) continue;
                f[i][j] = (f[i][j] + f[i-1][k]) % MOD;
            }
        }
    }
    int ans = 0;
    for (int j = 1; j <= a[m].num; j++)
        ans = (ans + f[m][j]) % MOD;
    cout << ans << endl;
}
int main() {
    cin >> m >> n;
    for (int i = 1; i <= m; i++) {
        int t = 0;
        for (int j = 0; j < n; j++) {
            int x;
            cin >> x;
            t = (t << 1) + (1 - x);
        }
        getstate(i, t);
    }
    dp();
    return 0;
}
```

8.12 Map 的使用

```
#include<bits/stdc++.h>
using namespace std;
int main(){
  //什么是map？map是干什么用的
  map<int,int>mp; //定义了一个名叫mp的map容器
  mp[10001]=82;mp[100002]=82;
  //输出10001信息
  cout<<mp[10001]<<endl;//可能会溢出
  //安全的方法
  if(mp.find(10005)!=mp.end()){
    cout<<mp[10005]<<endl;
  }

    else cout<<"没有此人信息"<<endl;
  //储存学生姓名对应成绩
  map<string,int>stu;
  stu["xiaoming"]=85;
  stu["xiaohong"]=77;
  cout<<stu["xiaohong"]<<endl;
  //map的遍历
    //1.
  for(auto it:stu){//将容器从头到尾遍历了一遍
    //map自动按照首位字典序从小到大排序
    cout<<it.first<<' '<<it.second<<endl;
  }
  //2.迭代器类型
  map<string,int>::iterator it;//定义map<string,int>类型的迭代器it
  for(it=stu.begin();it!=stu.end();it++){
    cout<<(*it).first<<' '<<(*it).second<<endl;
  }
}
```

8.13 优先队列（堆）

```
//自定义类的优先队列
struct Student {
    std::string name;
    int score;
    int age;

    // 重载 < 运算符，用于大根堆（按score从大到小）
    bool operator<(const Student& other) const {
        // 注意：priority_queue默认用 < 比较，但实际表现是最大堆
        // 所以这里返回 true 表示优先级更低
        return score < other.score;  // 大根堆
    }
};
priority_queue<Student>q1;//大根堆
priority_queue<int>q2;//默认大根堆
priority_queue<int,vector<int>,greater<int>>q3;//小根堆
```

8.14 扫描线 + 线段树（矩形面积并）

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll arr[15010],n,height[15010];

struct node{
    ll l,r,h;
}a[15010];

struct compare{
    bool operator()(pair<ll,ll> a,pair<ll,ll> b){
        if(a.first!=b.first) return a.first<b.first;
        return a.second>b.second;
        //大根堆
    }
};
bool cmp(node x,node y){
    return x.l<y.l;
}
int main(){
    ll l,r,h,m=0;
    while(scanf("%lld %lld %lld",&l,&h,&r)!=EOF){
        a[++m].l=l;
        a[m].r=r-1;
        a[m].h=h;
        arr[++n]=l;
        arr[++n]=r;
        arr[++n]=r-1;
    }
    sort(arr+1,arr+n+1);
    ll len=unique(arr+1,arr+n+1)-arr-1;
    for(int j=1;j<=m;j++){
        a[j].l=lower_bound(arr+1,arr+len+1,a[j].l)-arr;
        a[j].r=lower_bound(arr+1,arr+len+1,a[j].r)-arr;
    }
    sort(a+1,a+m+1,cmp);
   priority_queue<pair<ll,ll>, vector<pair<ll,ll>>, compare>
        heap;
   for(int i=1,j=0;i<=len;i++){
    for(;j<=m&&a[j].l<=i;j++){
        heap.push(make_pair(a[j].h,a[j].r));
    }
    while(!heap.empty()&&heap.top().second<i)
    {
        heap.pop();
    }

   if(!heap.empty()){
    height[i]=heap.top().first;
   }
}
    ll ansn=0;
    pair<ll,ll>ans[15010];
    for(ll i=1,pre=0;i<=len;i++){
        if(pre!=height[i]){
            ans[++ansn]=make_pair(arr[i],height[i]);
        }
        pre=height[i];
    }
    for(int i=1;i<=ansn;i++){
        cout<<ans[i].first<<' '<<ans[i].second<<' ';
    }
}
```

8.15 建树参考

```
#include<bits/stdc++.h>
//#define int long long
#define in(a) a = read_int()
using namespace std;
const int N = 2e5 + 5;
const int inf = INT_MAX;
```

```
inline int read_int() {
  int x = 0;
  char ch = getchar();
  bool f = 0;
  while('9' < ch || ch < '0') f |= ch == '-' , ch = getchar();
  while('0' <= ch && ch <= '9') x = (x << 3) + (x << 1) + ch -
       '0' , ch = getchar();
  return f ? -x : x;
}
struct Edge {
  int n , v , w;
} edge[N << 1];
int head[N] , eid;
void eadd(int u , int v , int w) {
  edge[++ eid].n = head[u] , edge[eid].v = v , edge[head[u] =
      eid].w = w;
}
int n;
int l[N] , r[N] , ans[N];
void dfs(int u , int fa) { // 树形 DP 部分
  l[u] = 1; // 初始化左边界
  for(int i = head[u] ; i ; i = edge[i].n) {
    int v = edge[i].v , w = edge[i].w;
    if(v != fa) {
      r[v] = w - 1; // 初始化右边界
      dfs(v , u);
      r[u] = min(r[u] , w - l[v]); // 转移过程
      l[u] = max(l[u] , w - r[v]);
    }
  }
}
queue<int>q;
signed main() {
  in(n);
  for(int i = 1 ; i < n ; i ++) {
    int u , v , w;
    in(u) , in(v) , in(w);
    eadd(u , v , w) , eadd(v , u , w);
  }
  r[1] = 1e9; // 初始化根节点的右边界
  dfs(1 , 0); // 求每个点权的取值范围
  for(int i = 1 ; i <= eid ; i += 2) { // 判无解部分
    int u = edge[i].v , v = edge[i + 1].v , w = edge[i].w;
    if(r[u] + r[v] < w || l[u] > r[u] || l[v] > r[v]) {
            cout<<"NO";
      return 0;
    }
  }
  ans[1] = l[1]; // 若有解，根节点点权取左边界
  q.push(1);
  while(!q.empty()) { // 广搜部分
    int u = q.front();
    q.pop();
    for(int i = head[u] ; i ; i = edge[i].n) {
      int v = edge[i].v , w = edge[i].w;
      if(ans[v]) continue;
      ans[v] = w - ans[u]; // 递推求解
      q.push(v);
    }
  }
  cout<<"YES\n"; // 打印答案
  for(int i = 1 ; i <= n ; i ++) cout<<ans[i]<<' ';
  return 0;
}
```

8.16 Hash 树

```
//树同构

#include <bits/stdc++.h>

 typedef long long LL;
 const int N = 60, MO = 998244353;

 struct Edge {
     int nex, v;
 }edge[N << 1]; int tp;

 int e[N], n, m, turn, fr[N], p[1000010], top, siz[N], h[N], g
      [N];
 std::vector<int> v[N];
 bool vis[1000010];

 inline void add(int x, int y) {
     tp++;
     edge[tp].v = y;
     edge[tp].nex = e[x];
     e[x] = tp;
     return;
 }

 inline bool equal(int a, int b) {
     int len = v[a].size();
     if(len != v[b].size()) return false;
     for(int i = 0; i < len; i++) {
         if(v[a][i] != v[b][i]) return false;
     }
     return true;
 }

 inline void getp(int n) {
     for(int i = 2; i <= n; i++) {
         if(!vis[i]) {
             p[++top] = i;
         }
         for(int j = 1; j <= top && i * p[j] <= n; j++) {
             vis[i * p[j]] = 1;
             if(i % p[j] == 0) {
                 break;
             }
         }
     }
     return;
 }

 void DFS_1(int x, int f) {
     siz[x] = 1;
     h[x] = 1;
     for(int i = e[x]; i; i = edge[i].nex) {
         int y = edge[i].v;
         if(y == f) continue;
         DFS_1(y, x);
         h[x] = (h[x] + 1ll * h[y] * p[siz[y]] % MO) % MO;
         siz[x] += siz[y];
     }
     return;
 }

 void DFS_2(int x, int f, int V) {
     g[x] = (h[x] + 1ll * V * p[n - siz[x]] % MO) % MO;
     v[turn].push_back(g[x]);
     V = (1ll * V * p[n - siz[x]] % MO + 1) % MO;
     for(int i = e[x]; i; i = edge[i].nex) {
         int y = edge[i].v;
         if(y == f) {
             continue;
         }
         DFS_2(y, x, ((LL)V + h[x] - 1 - 1ll * h[y] * p[siz[y
              ]] % MO + MO) % MO);
     }
     return;
 }

 int main() {
     getp(1000009);
     scanf("%d", &m);
     for(turn = 1; turn <= m; turn++) {
         scanf("%d", &n);
         tp = 0;
         memset(e + 1, 0, n * sizeof(int));
         for(int i = 1, x; i <= n; i++) {
             scanf("%d", &x);
             if(x) {
                 add(x, i);
                 add(i, x);
             }
         }
         DFS_1(1, 0);
         DFS_2(1, 0, 0);
         std::sort(v[turn].begin(), v[turn].end());
     }

     for(int i = 1; i <= m; i++) {
         fr[i] = i;
     }
     for(int i = 2; i <= m; i++) {
         for(int j = 1; j < i; j++) {
```

```
99              if(equal(i, j)) {
100                 fr[i] = fr[j];
101                 break;
102             }
103         }
104     }
105     for(int i = 1; i <= m; i++) {
106         printf("%d\n", fr[i]);
107     }
108     return 0;
109 }
```